

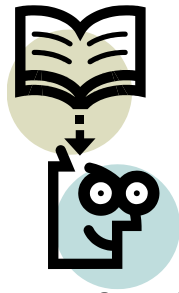
UNIT-1

Basics of Cryptography



Outline

- Information Security understanding
- Security goals
- Security attacks
- Security Services
- Security Mechanism

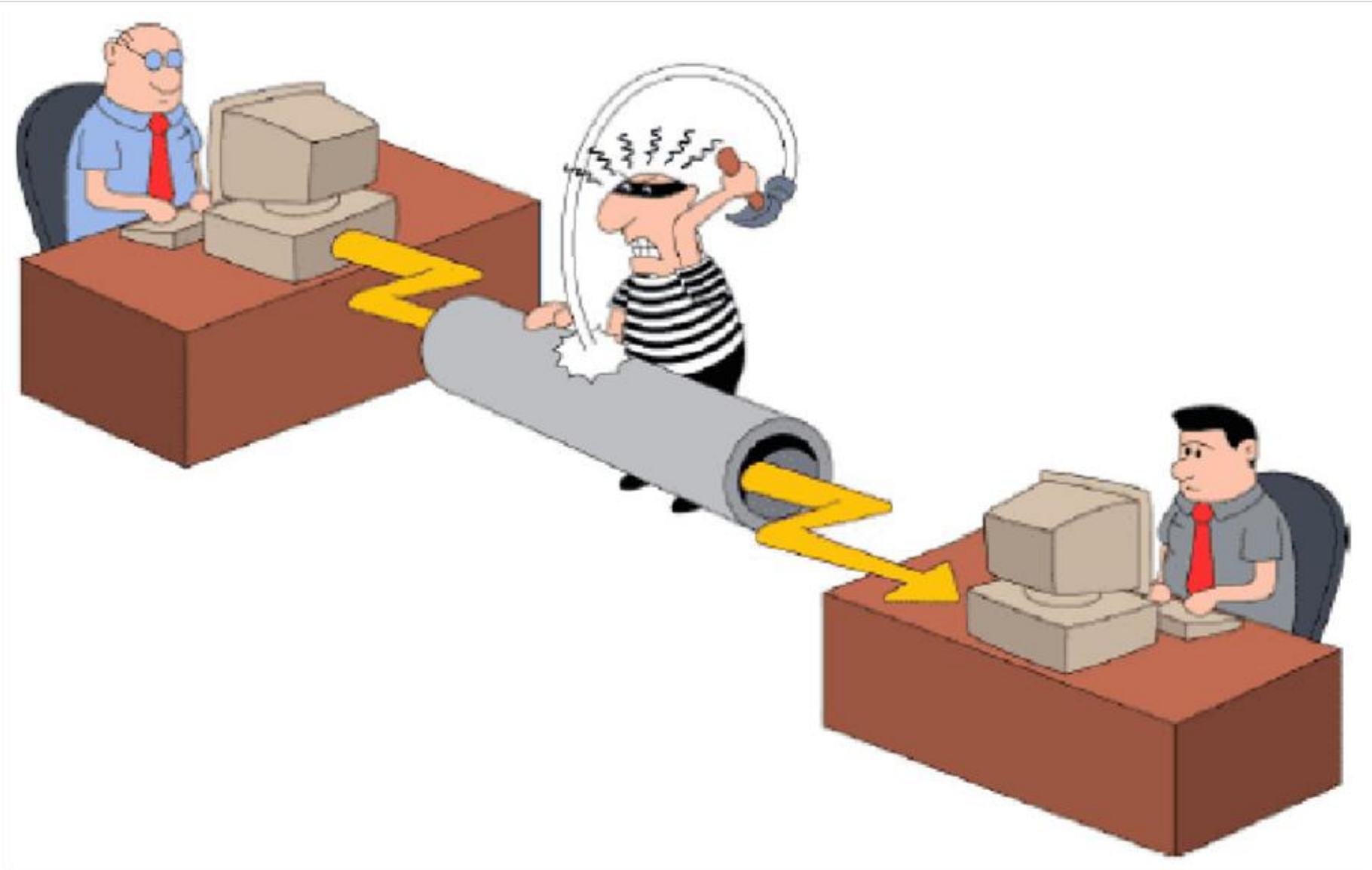


Lecture 1

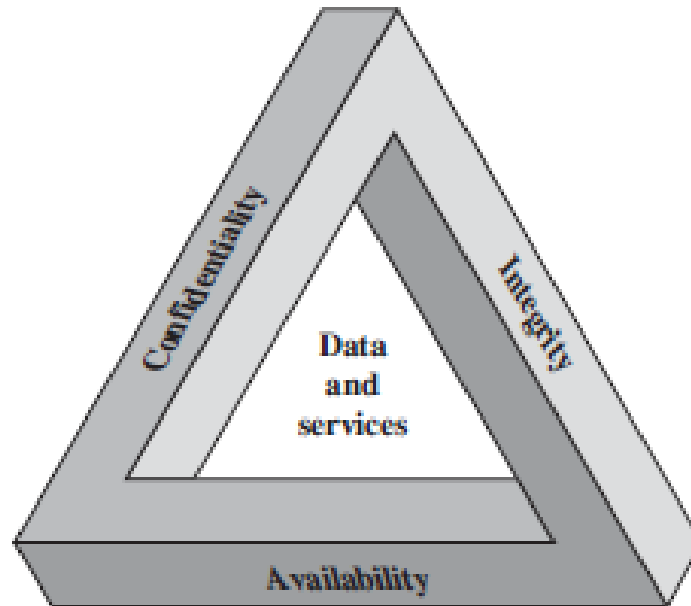


Information Security Understanding

Introduction to Information & N/W Security



Security requirements triad

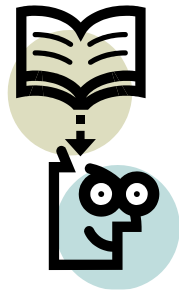


OSI Security Architecture

- The OSI (Open Systems Interconnection) security architecture focuses on Security Attacks, Mechanisms, and Services.
- **Security Attack:** Any action that compromises the security of information owned by an organization.
- **Security Mechanism:** A process that is designed to detect, prevent, or recover from a security attack.
- **Security Service:** A communication service that enhances the security of the data processing systems and the information transfers of an organization.

Main Areas

- **Symmetric encryption:** Used to conceal the contents of blocks or streams of data of any size, including messages, files, encryption keys, and passwords.
- **Asymmetric encryption:** Used to conceal small blocks of data, such as encryption keys and hash function values, which are used in digital signatures.
- **Data integrity algorithms:** Used to protect blocks of data, such as messages from alteration.
- **Authentication Protocols:** These are schemes based on the use of cryptographic algorithms designed to authenticate the identity of entities.



Lecture 2

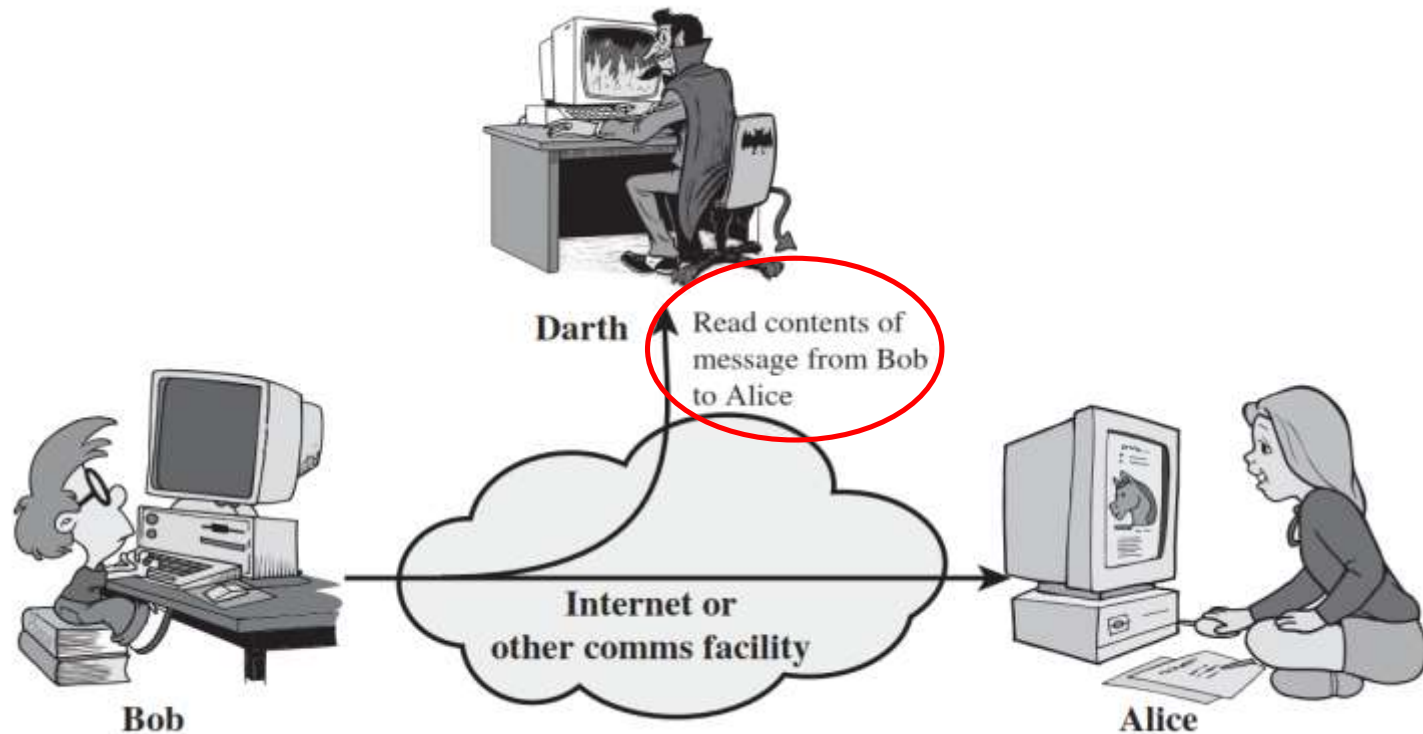


Security Attacks

Security Attacks

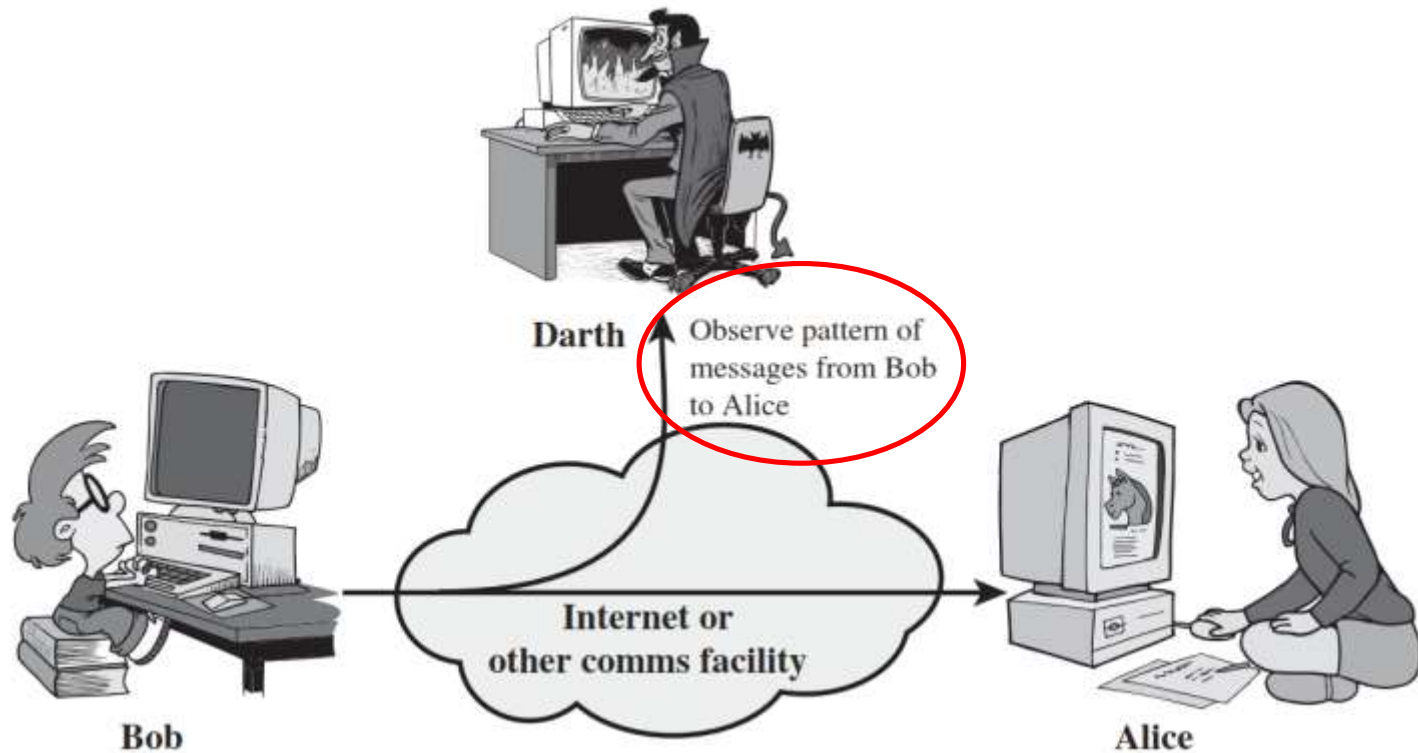
- A **passive attack** attempts to learn or make use of information from the system but does not affect system resources.
 1. Release of message contents
 2. Traffic analysis
- An **active attack** attempts to alter system resources or affect their operation.
 1. Masquerade
 2. Replay
 3. Modification of messages
 4. Denial of service.

1) Release of message contents (Passive Attack)



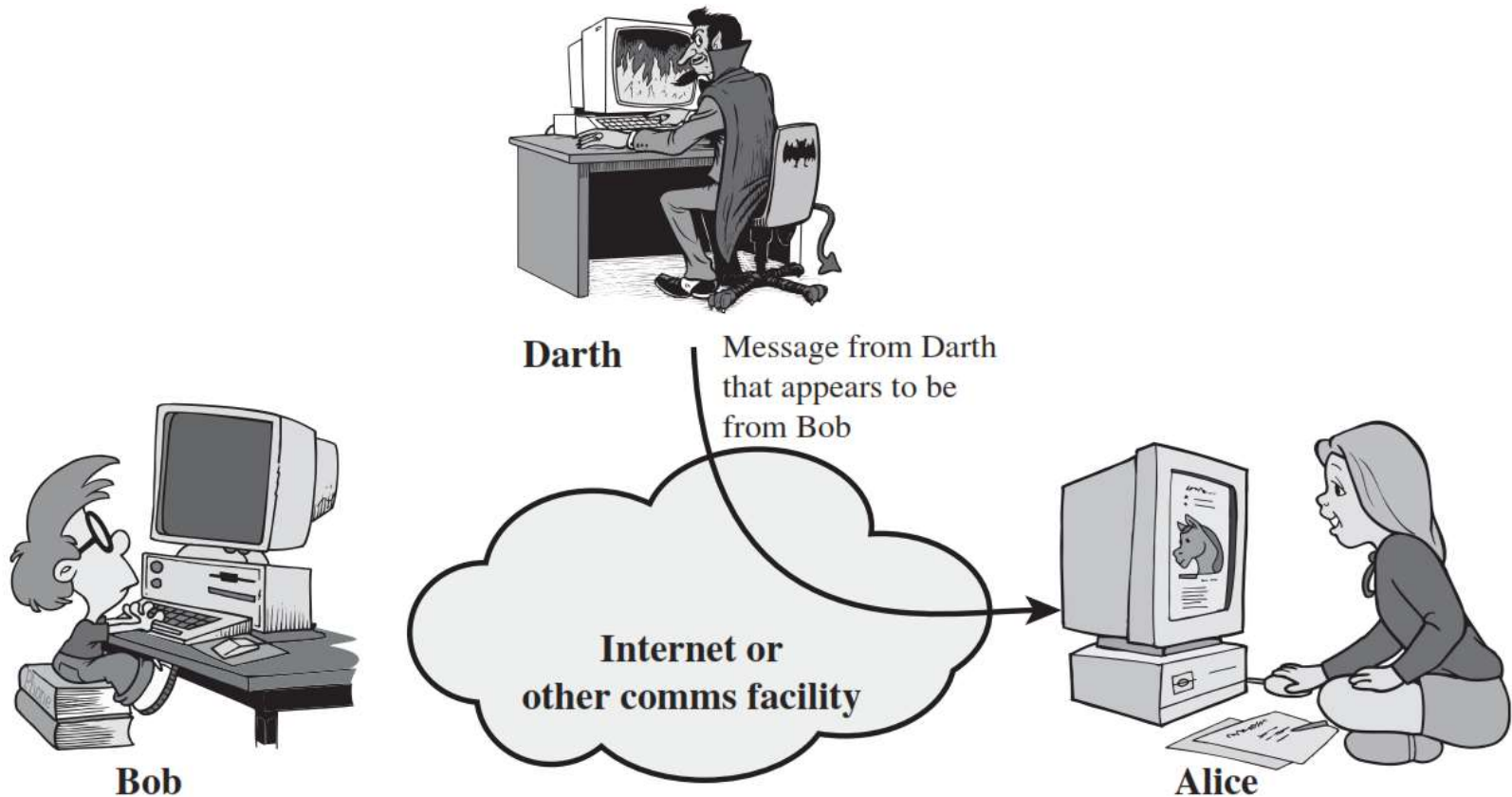
- A telephone conversation, an electronic mail message, and a transferred file may contain sensitive or confidential information.
- We would like to prevent an opponent from learning the contents of these transmissions.

2) Traffic Analysis (Passive Attack)



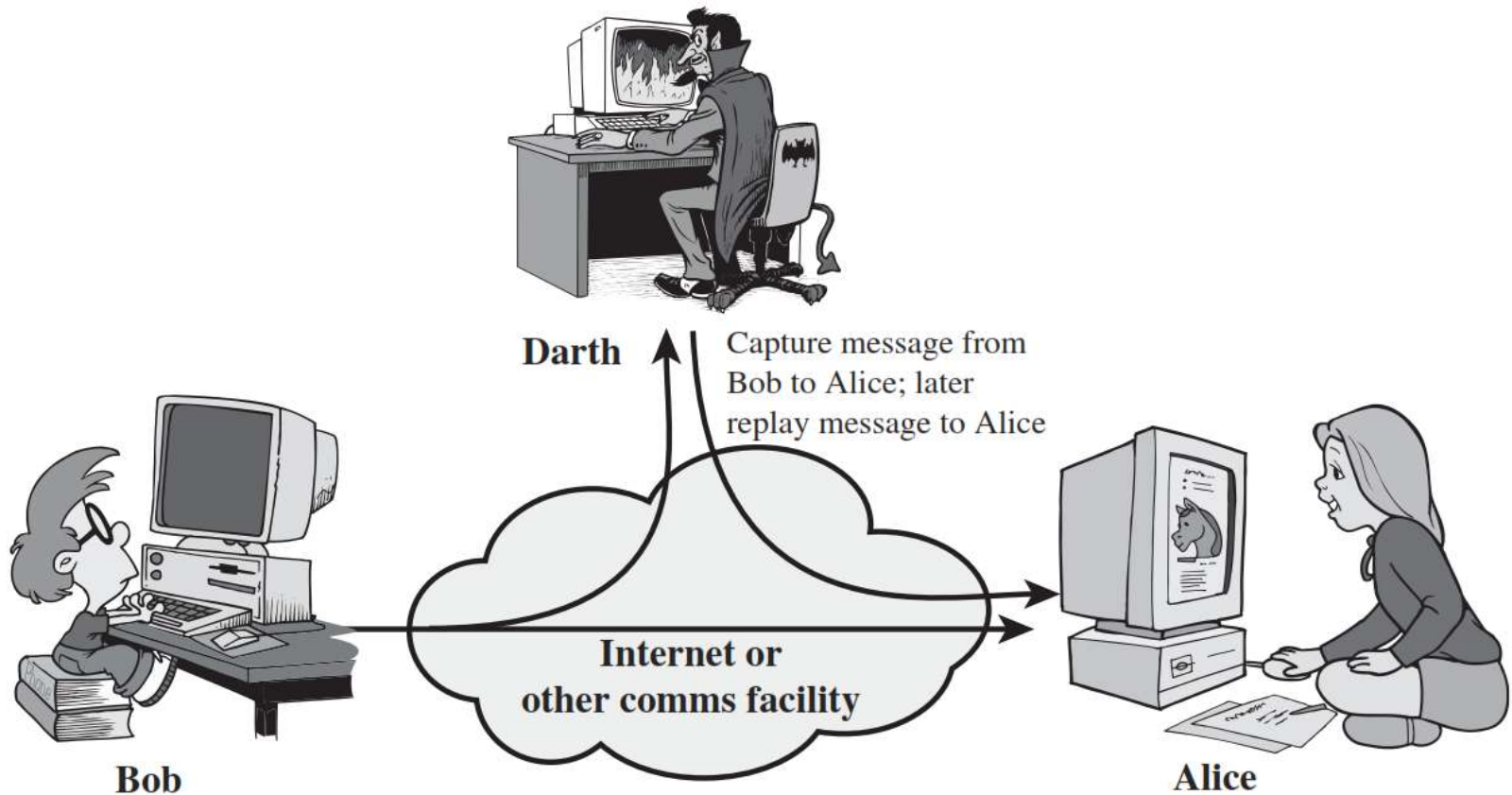
- In such attacks, an adversary, capable of observing network traffic statistics in several different networks, correlates the traffic patterns in these networks.

1) Masquerade Attack (Active Attack)



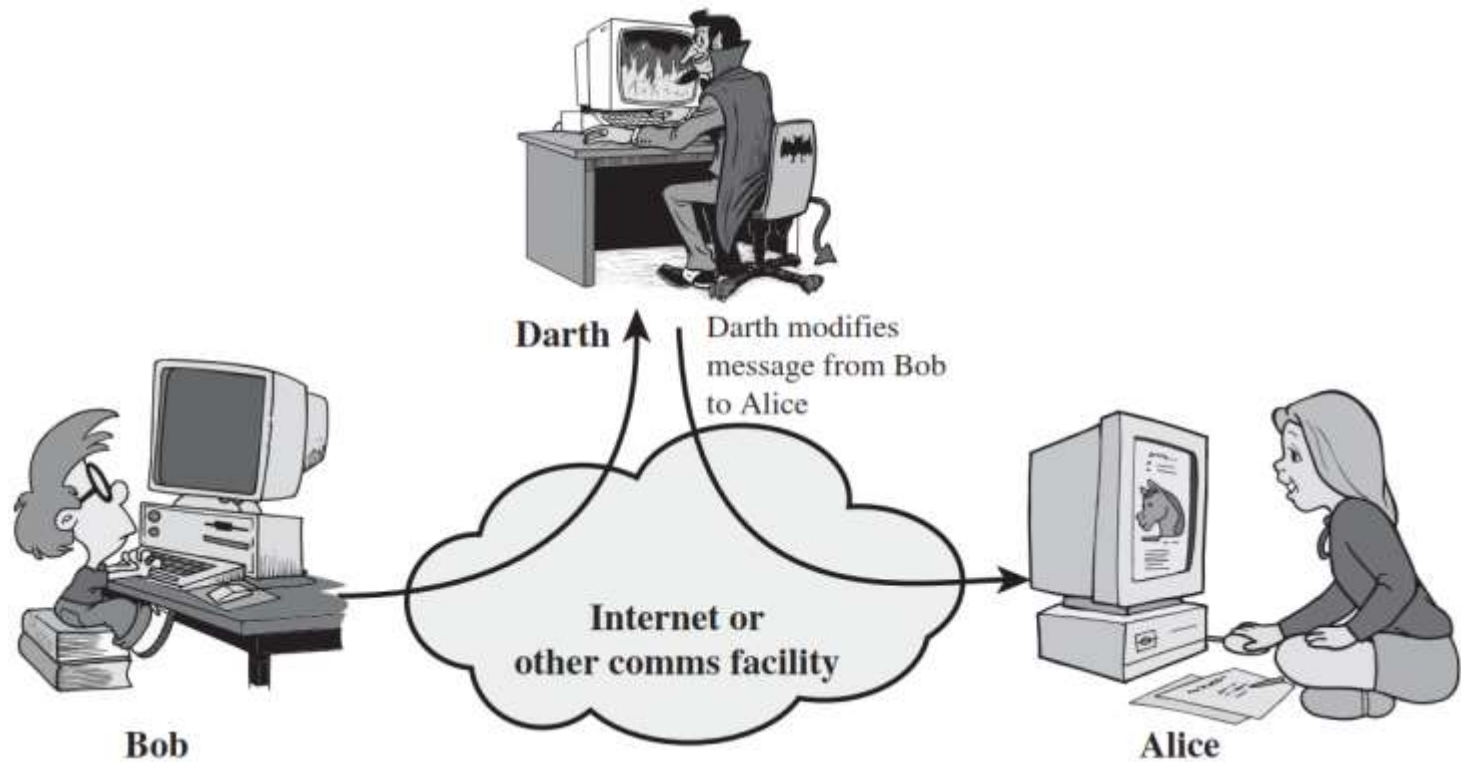
- A **masquerade** takes place when one entity pretends to be a different entity.

2) Replay Attack (Active Attack)



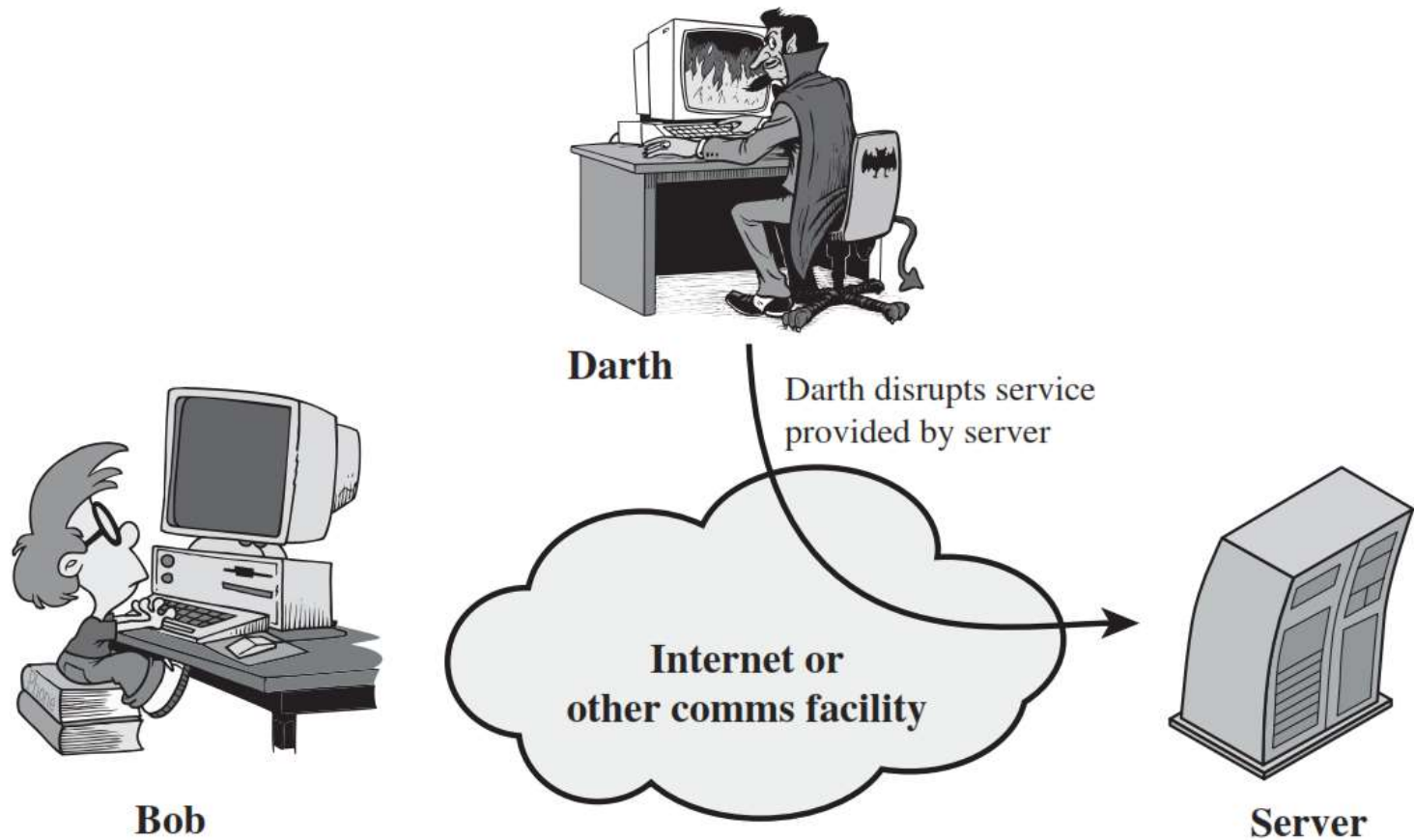
- **Replay attack** involves the passive capture of a data unit and its subsequent retransmission to produce an unauthorized effect.

3) Modification of messages Attack (Active Attack)

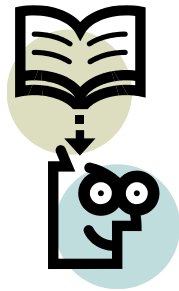


- **Modification of messages** simply means that some portion of a legitimate message is altered, or that messages are delayed or reordered, to produce an unauthorized effect.

4) Denial of Service Attack (Active Attack)



- **The denial of service** attack prevents the normal use or management of communications facilities.



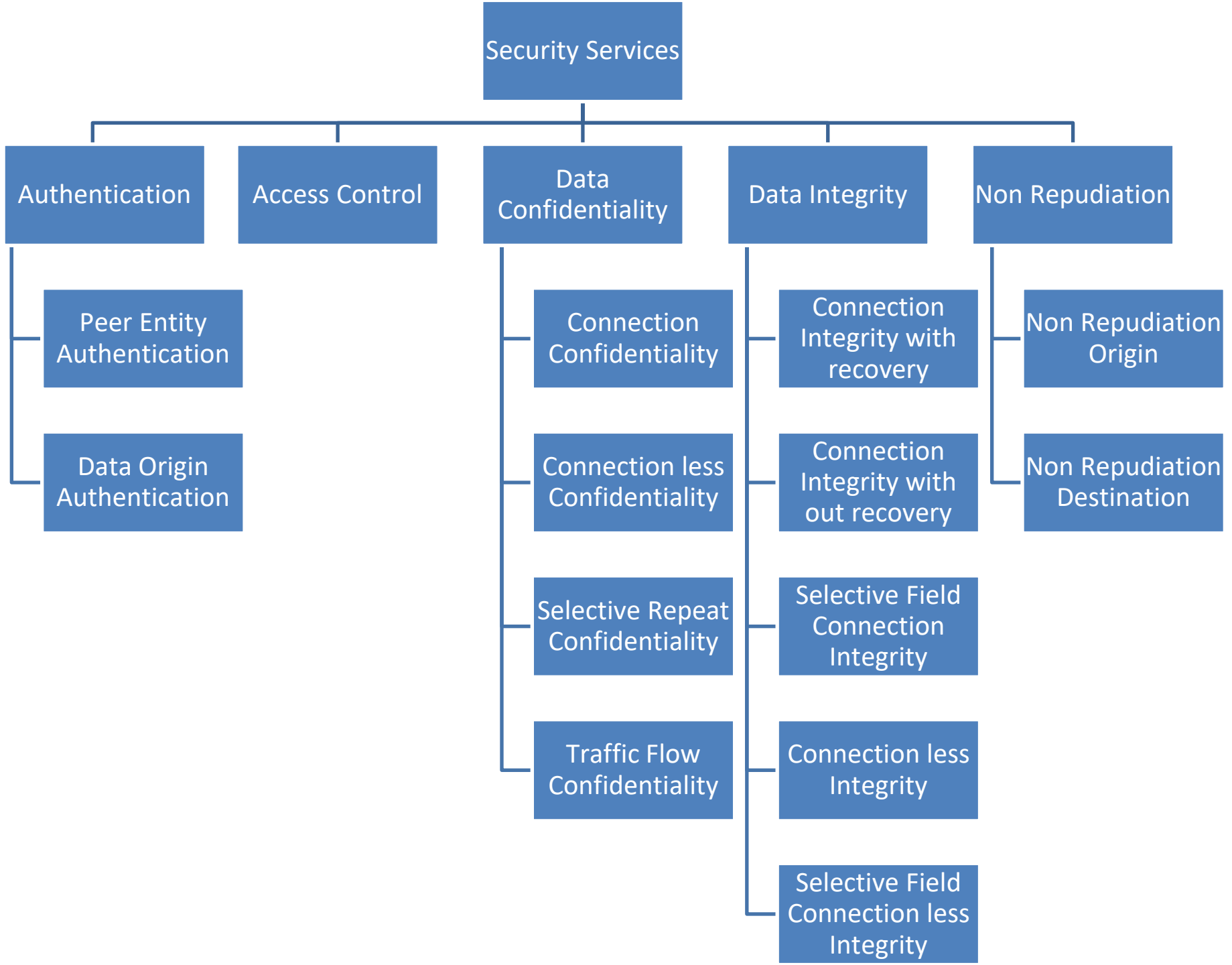
Lecture 3



Security Services

Security Services (X.800)

- X.800 standard defines a security service as a service that is provided by a protocol layer of communicating open systems and that ensures security of the systems or of data transfers.



Authentication

- **Authentication** is the assurance that the communicating entity is the one that it claims to be.

1. **Peer Entity Authentication:**

Used in association with a logical connection to provide confidence in the identity of the entities connected.

2. **Data-Origin Authentication:** In a connectionless transfer, provides assurance that the source of received data is as claimed.

Who you are ?
(biometrics)



Physical
authentication
where you are ?



What you know ?

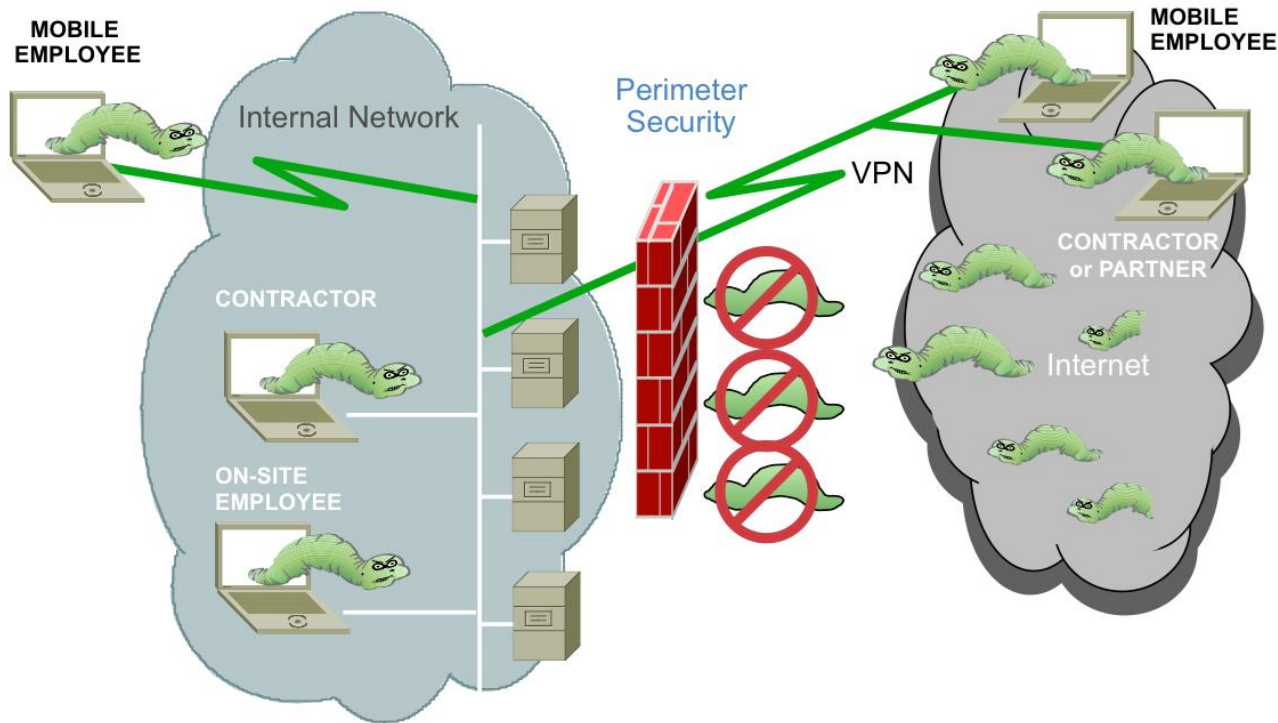
Password

One-time Passwords

Network address

Access Control

- **Access control** is the prevention of unauthorized use of a resource
- This service controls who can have access to a resource, under what conditions access can occur, and what those accessing the resource are allowed to do).



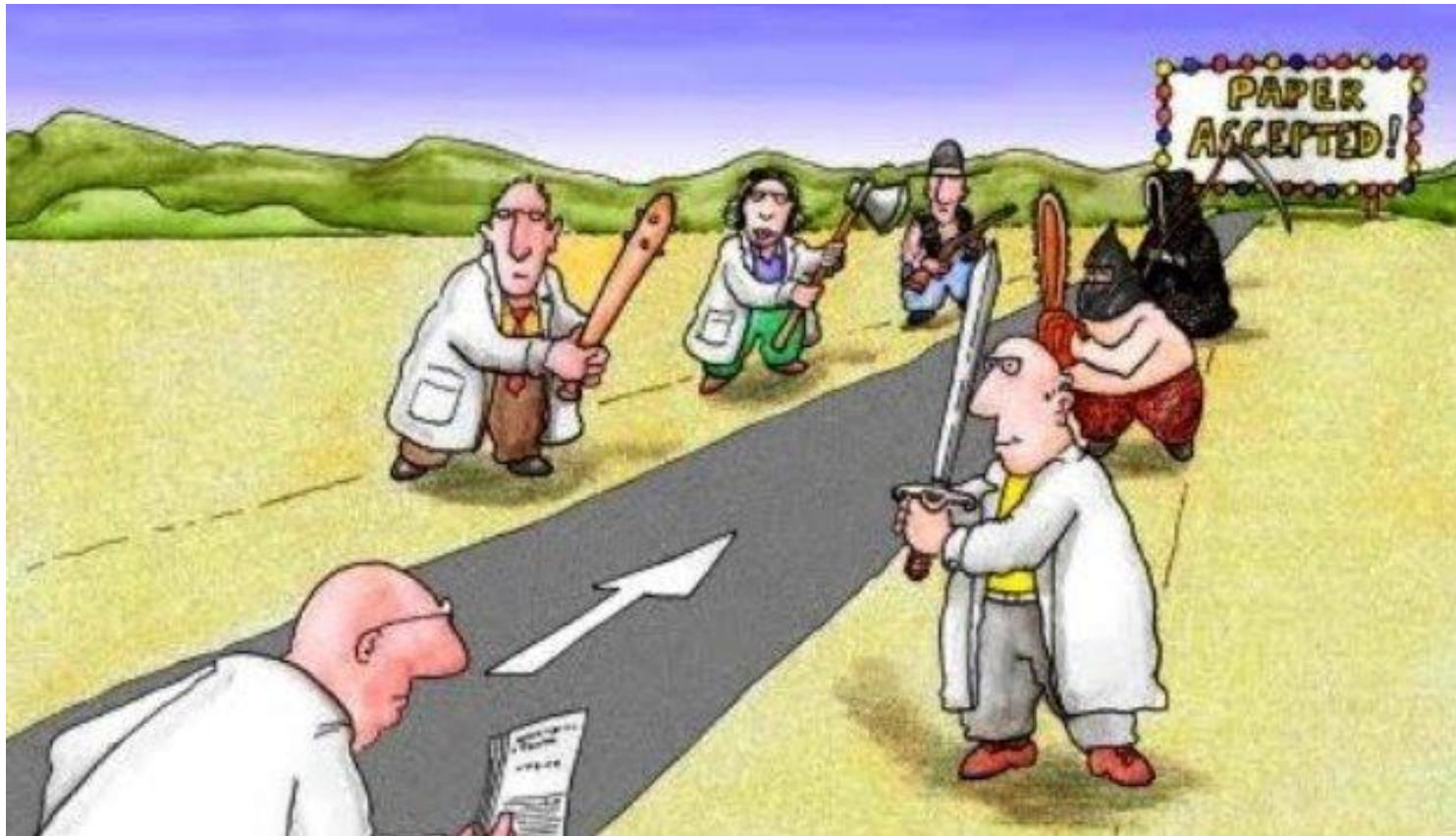
Data Confidentiality

- **Data confidentiality** is the protection of data from unauthorized disclosure.
 1. **Connection Confidentiality:** The protection of all user data on a connection.
 2. **Connectionless Confidentiality:** The protection of all user data in a single data block.
 3. **Selective-Field Confidentiality:** The confidentiality of selected fields within the user data on a connection or in a single data block.
 4. **Traffic-Flow Confidentiality:** The protection of the information that might be derived from observation of traffic flows.



Data Integrity

- Data integrity is the assurance that data received are exactly as sent by an authorized entity (i.e., contain no modification, insertion, deletion, or replay).



Data Integrity (Cont...)

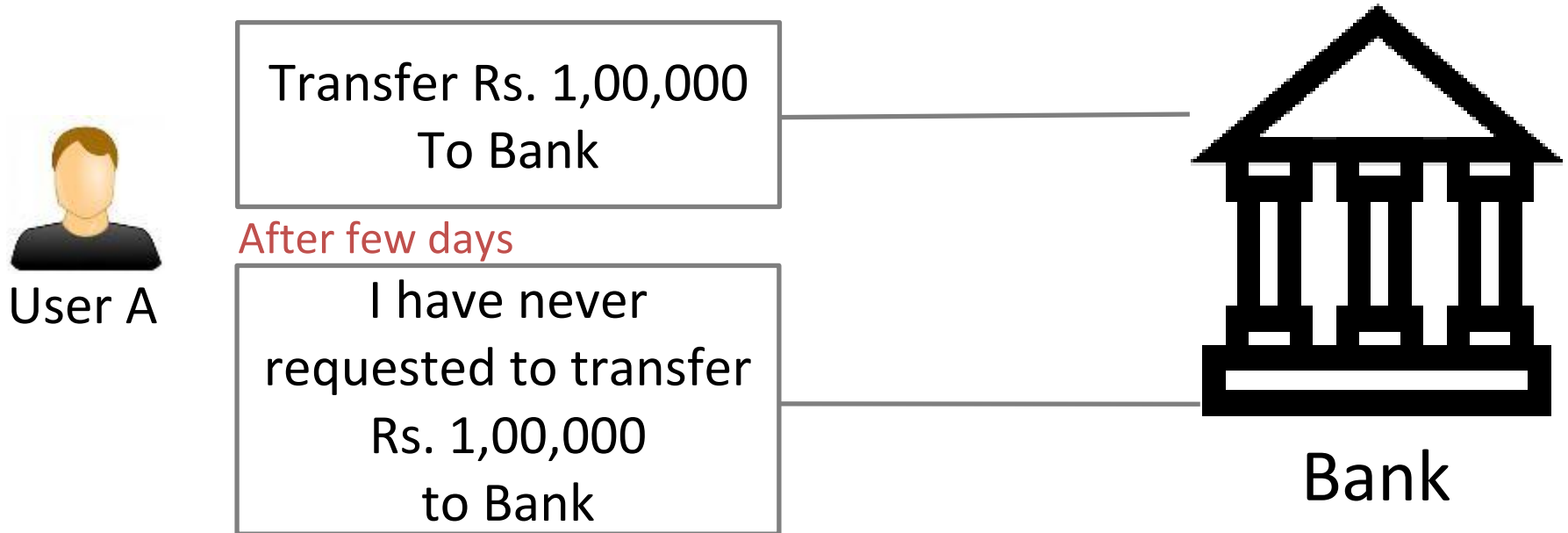
- **Connection Integrity with Recovery:** Provides integrity of all user data on a connection and detects any modification, insertion, deletion, or replay of any data with recovery attempted.
- **Connection Integrity without Recovery:** As above, but provides only detection without recovery.
- **Selective-Field Connection Integrity:** Provides integrity of selected fields within the user data and takes the form of determination of whether the selected fields have been modified, inserted, deleted, or replayed.

Data Integrity (Cont...)

- **Connectionless Integrity:** Provides integrity of a single connectionless data block and may take the form of detection of data modification. Additionally, a limited form of replay detection may be provided.
- **Selective-Field Connectionless Integrity:** Provides integrity of selected fields within a single connectionless data block; takes the form of determination of whether the selected fields have been modified.

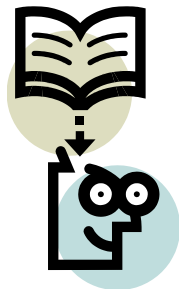
Non Repudiation

- **Nonrepudiation** is the assurance that someone cannot deny something.
- Typically, nonrepudiation refers to the ability to ensure that a communication cannot deny the authenticity of their signature on a document or the sending of a message that they originated.



Non Repudiation (Cont...)

- **Nonrepudiation-Origin:** Proof that the message was sent by the specified party.
- **Nonrepudiation-Destination:** Proof that the message was received by the specified party.



Lecture 4



Security Mechanisms

Security Mechanisms (X.800)

- **Specific security mechanisms:** Integrated into the appropriate protocol layer in order to provide some of the OSI security services.
- **Pervasive security mechanisms:** Not integrated to any particular OSI security service or protocol layer

Security Mechanism (Specific Security)

- **Encipherment:** Hiding or covering data using mathematical algorithms.
- **Digital Signature:** The sender can electronically sign the data and the receiver can electronically verify the signature.
- **Access Control:** A variety of mechanisms that enforce access rights to resources.
- **Data Integrity:** A variety of mechanisms used to assure the integrity of a data unit or stream of data units.
- **Authentication Exchange:** Two entities exchange some messages to prove their identity to each other.

Security Mechanism (Specific security)

- **Traffic Padding:** The insertion of bits into gaps in a data stream to frustrate traffic analysis attempts.
- **Routing Control:** Selecting and continuously changing routes between sender and receiver to prevent opponent from eavesdropping.
- **Notarization:** The use of a trusted third party to assure and control the communication.

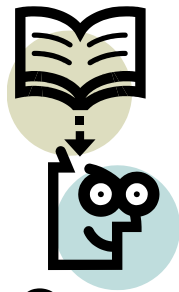
Security Mechanism (Pervasive security)

- **Trusted Functionality**
- **Security Label**
- **Event Detection**
- **Security Audit Trail**
- **Security Recovery**

UNIT-3

Classical Ciphers





Lecture 5

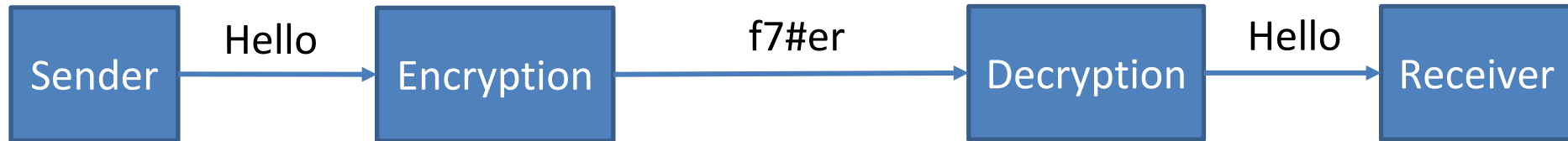


Symmetric cipher models

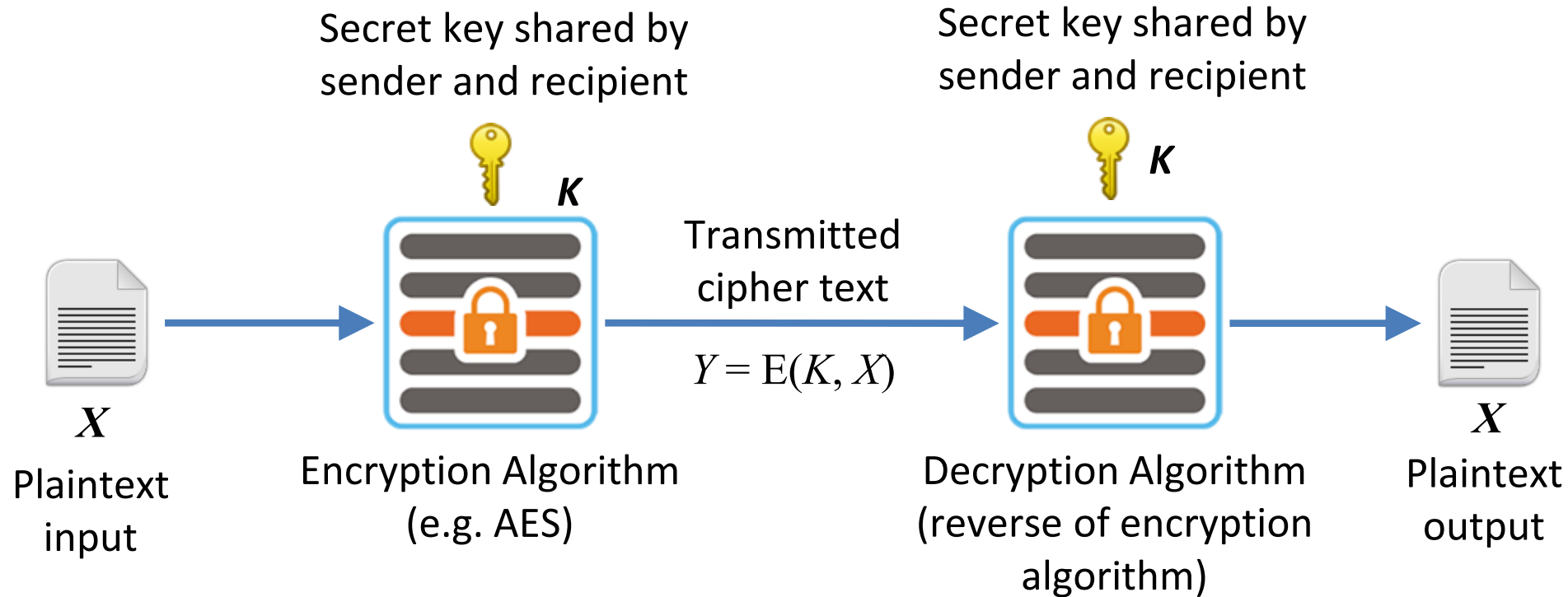
Outline

- Symmetric cipher models
- Substitution ciphers
- Transposition ciphers
- Steganography

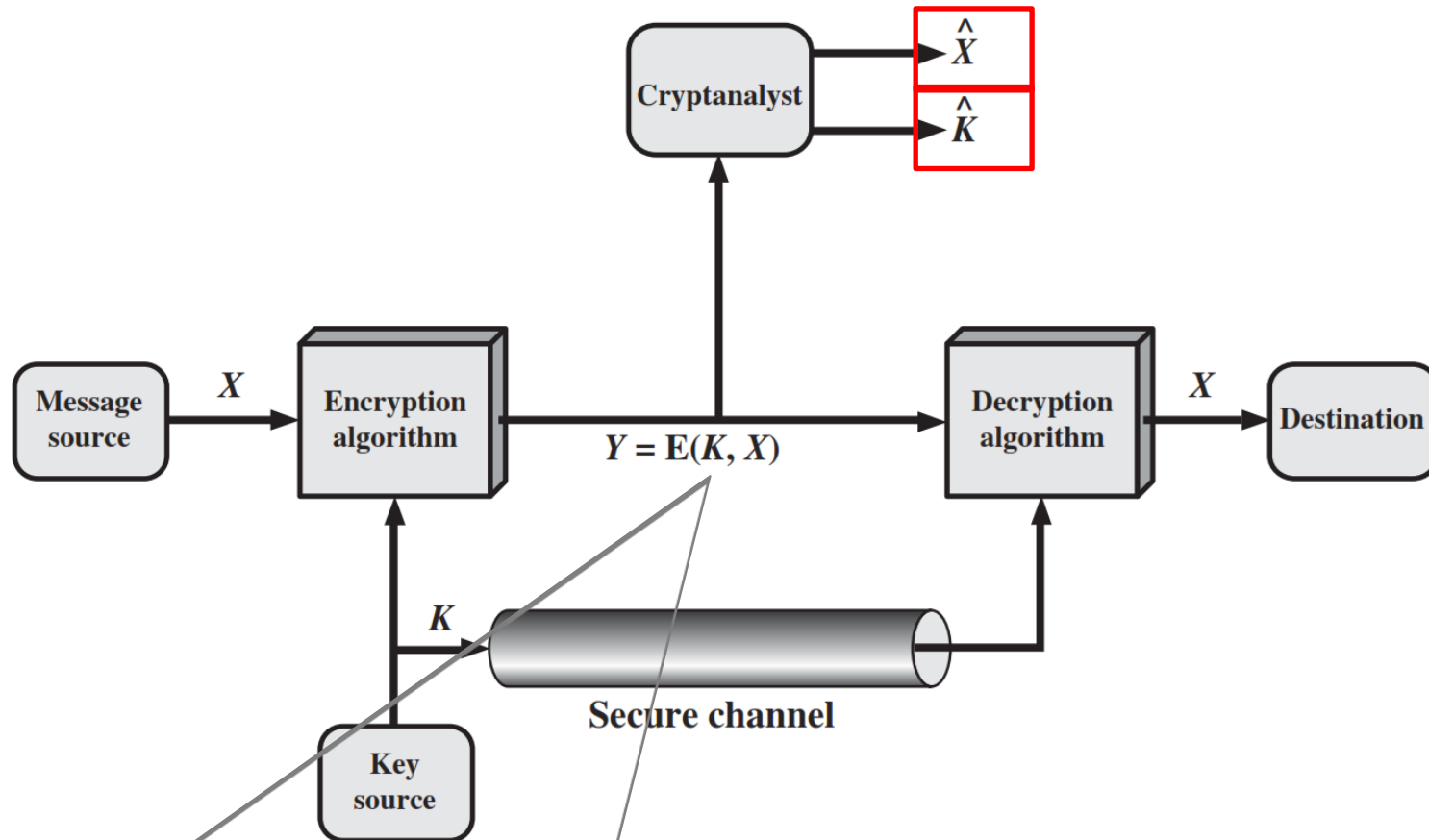
Encryption and Decryption



Symmetric Cipher Model (Conventional Encryption)



- Encryption and decryption use the same key.
- The key is shared between the sender and the receiver.
- The key is used to both encrypt the plaintext and to decrypt the ciphertext.
- The key is used to both encrypt the plaintext and to decrypt the ciphertext.
- The key is used to both encrypt the plaintext and to decrypt the ciphertext.



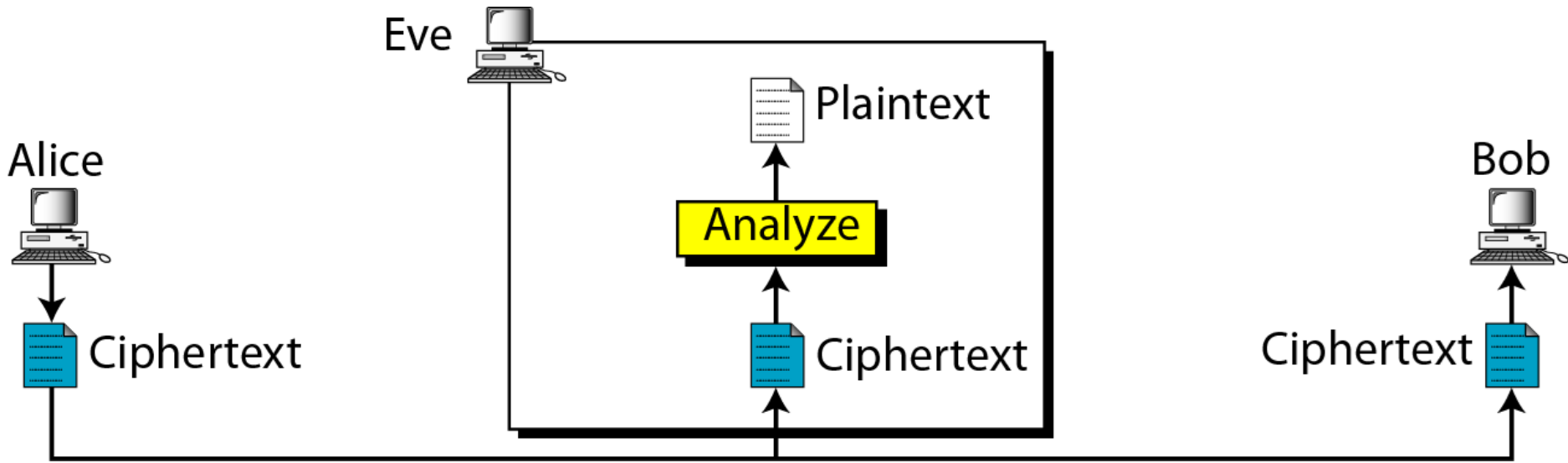
- An opponent, observing Y but not having access to K or X , may attempt to recover X or K or both X and K .
- If the opponent is interested in only this particular message, then he will focus to recover X by generating a plaintext estimate $\hat{X}$.
- Often, however, the opponent is interested in being able to read future messages as well, in which case an attempt is made to recover K by generating an estimate $\hat{K}$.

Cryptanalysis and Brute-Force Attack

- **Cryptanalysis:** Cryptanalytic attacks rely on the nature of the algorithm and some knowledge of the general characteristics of the plaintext or even some sample plaintext–ciphertext pairs.
- This type of attack exploits the characteristics of the algorithm to attempt to derive a specific plaintext or to derive the key being used.
- **Brute-force attack:** The attacker tries every possible key on a piece of ciphertext until an intelligible translation into plaintext is obtained.
- On average, half of all possible keys must be tried to achieve success.

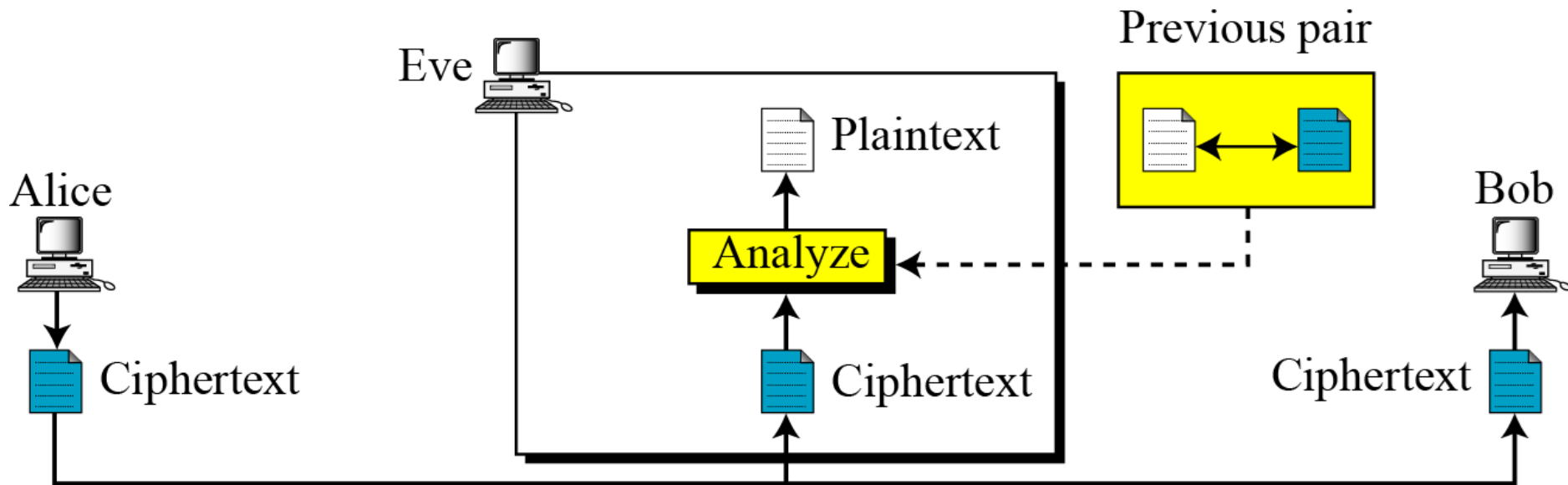
Attacks on Encrypted Messages

Type of Attack	Known to cryptanalyst
Ciphertext Only	Encryption algorithm, Ciphertext



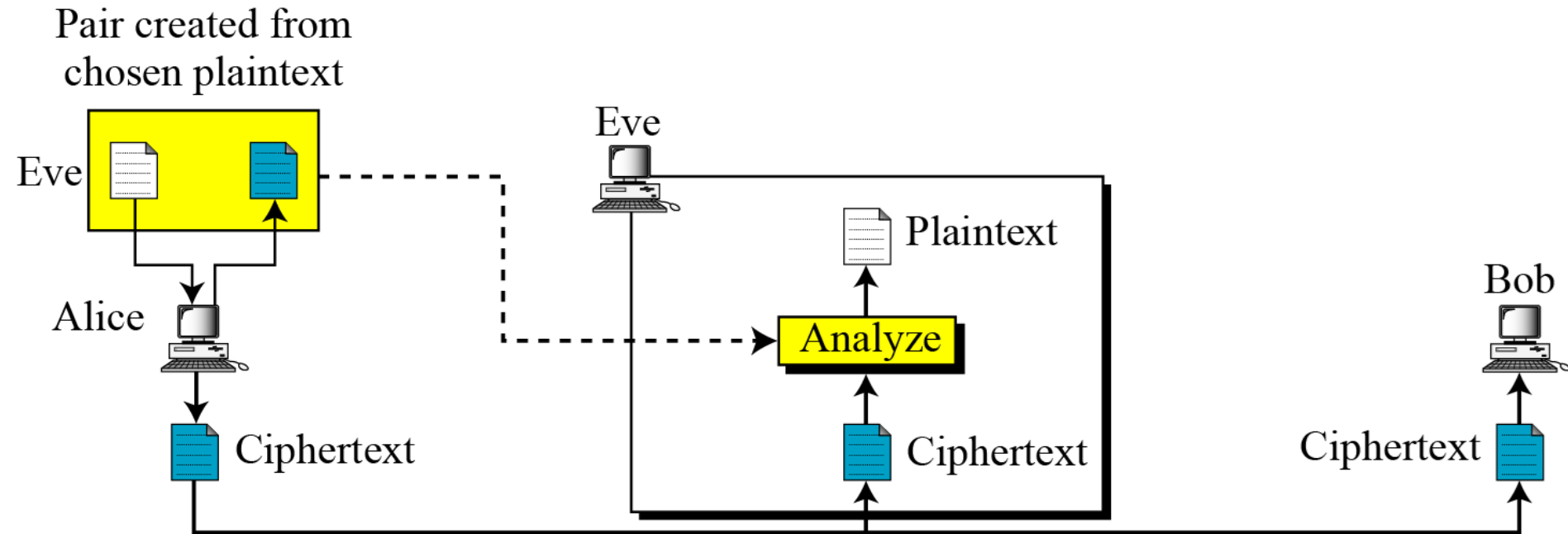
Attacks on Encrypted Messages

Type of Attack	Known to cryptanalyst
Known Plaintext	Encryption algorithm, Ciphertext, One or more plaintext-cipher text pairs formed with the secret key



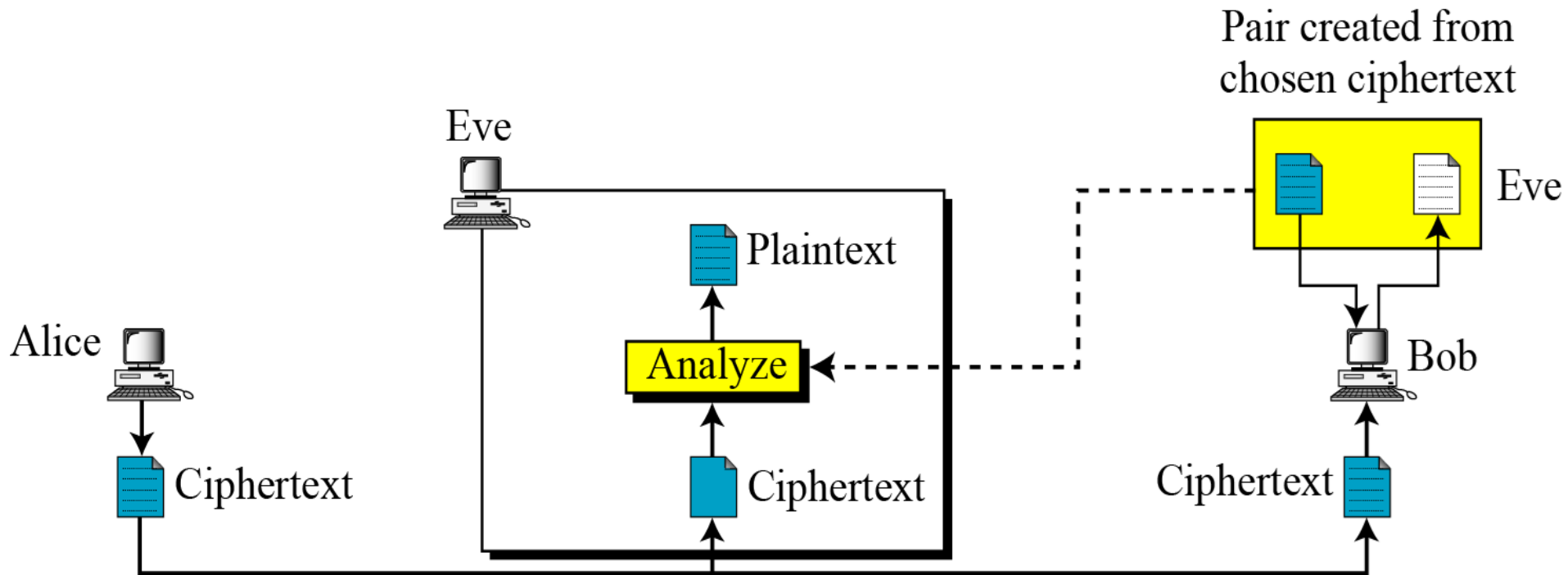
Attacks on Encrypted Messages

Type of Attack	Known to cryptanalyst
Chosen Plaintext	Encryption algorithm, Ciphertext, Plaintext message chosen by cryptanalyst



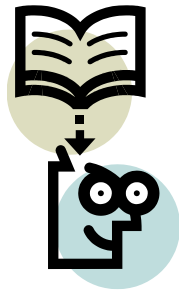
Attacks on Encrypted Messages

Type of Attack	Known to cryptanalyst
Chosen Ciphertext	Encryption algorithm, Ciphertext, Ciphertext chosen by cryptanalyst, with its corresponding decrypted plaintext generated with the secret key



Attacks on Encrypted Messages

Type of Attack	Known to cryptanalyst
Chosen text	Encryption algorithm, Ciphertext, Plaintext chosen by cryptanalyst, with its corresponding ciphertext generated with the secret key , Ciphertext chosen by cryptanalyst, with its corresponding decrypted plaintext generated with the secret key



Lecture 6



Caesar & Monoalphabetic

Substitution Techniques

- A substitution technique is one in which the letters of plaintext are replaced by other letters or by numbers or symbols.
 - 1) Caesar Cipher
 - 2) Monoalphabetic Cipher
 - 3) Playfair Cipher
 - 4) Hill Cipher
 - 5) Polyalphabetic Ciphers
 - 6) One-Time Pad

1) Caesar Cipher

- The **Caesar cipher** involves replacing each letter of the alphabet with the letter standing three places further down the alphabet.
- In encryption each plaintext letter P , substitute the ciphertext letter C :

$$C = E(k, P) = (P + k) \bmod 26$$

$$C = E(3, P) = (P + 3) \bmod 26$$

- For decryption algorithm is:

$$P = D(k, C) = (C - k) \bmod 26$$

Caesar Cipher (Cont...)

- Let us assign a numerical equivalent to each letter

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12
n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25

$$C = E(3, P) = (P + 3) \bmod 26$$

plain: a b c d e f g h i j k l m n o p q r s t u v w x y z
cipher: d e f g h i j k l m n o p q r s t u v w x y z a b c

Example:

Plaintext: THE QUICK BROWN FOX

Ciphertext: WKH TXLFN EURZQ IRA

Brute force attack on Caesar Cipher

- The encryption and decryption algorithms are known.
- There are only 25 keys to try.
- The language of the plaintext is known and easily recognizable.

Brute force attack on Caesar Cipher

Ciphertext: ZNK WAOIQ HXUCT LUD

Key	Transformed text
1	YMJ VZNHP GWTBS KTC
2	XLI UYMGO FVSAR JSB
3	WKH TXLFN EURZQ IRA
4	VJG SWKEM DTQYP HQZ
5	UIF RVJDL CSPXOGPY
6	THE QUICK BROWN FOX
7	SGD PTHBJ AQNVM ENW
8	RFC OSGAI ZPMUL DMV
9	QEB NRFZH YOLTK CLU
10	PDA MQEYG XNKSJ BKT
11	OCZ LPDXF WMJRI AJS
12	NBY KOCWE VLIQH ZIR
13	MAX JNBVD UKHPG YHQ

Key	Transformed text
14	LZW IMAUC TJGOF XGP
15	KYV HLZTB SIFNE WFO
16	JXU GKYSR RHEMD VEN
17	IWT FJXRZ QGDLC UDM
18	HVS EIWQY PFCKB TCL
19	GUR DHVPX OEBJA SBK
20	FTQ CGUOW NDAIZ RAJ
21	ESP BFTNV MCZHY QZI
22	DRO AESMU LBYGX PYH
23	CQN ZDRLT KAXFW OXG
24	BPM YCQKS JZWEV NWF
25	AOL XBPJR IYVDU MVE

Substitution Techniques

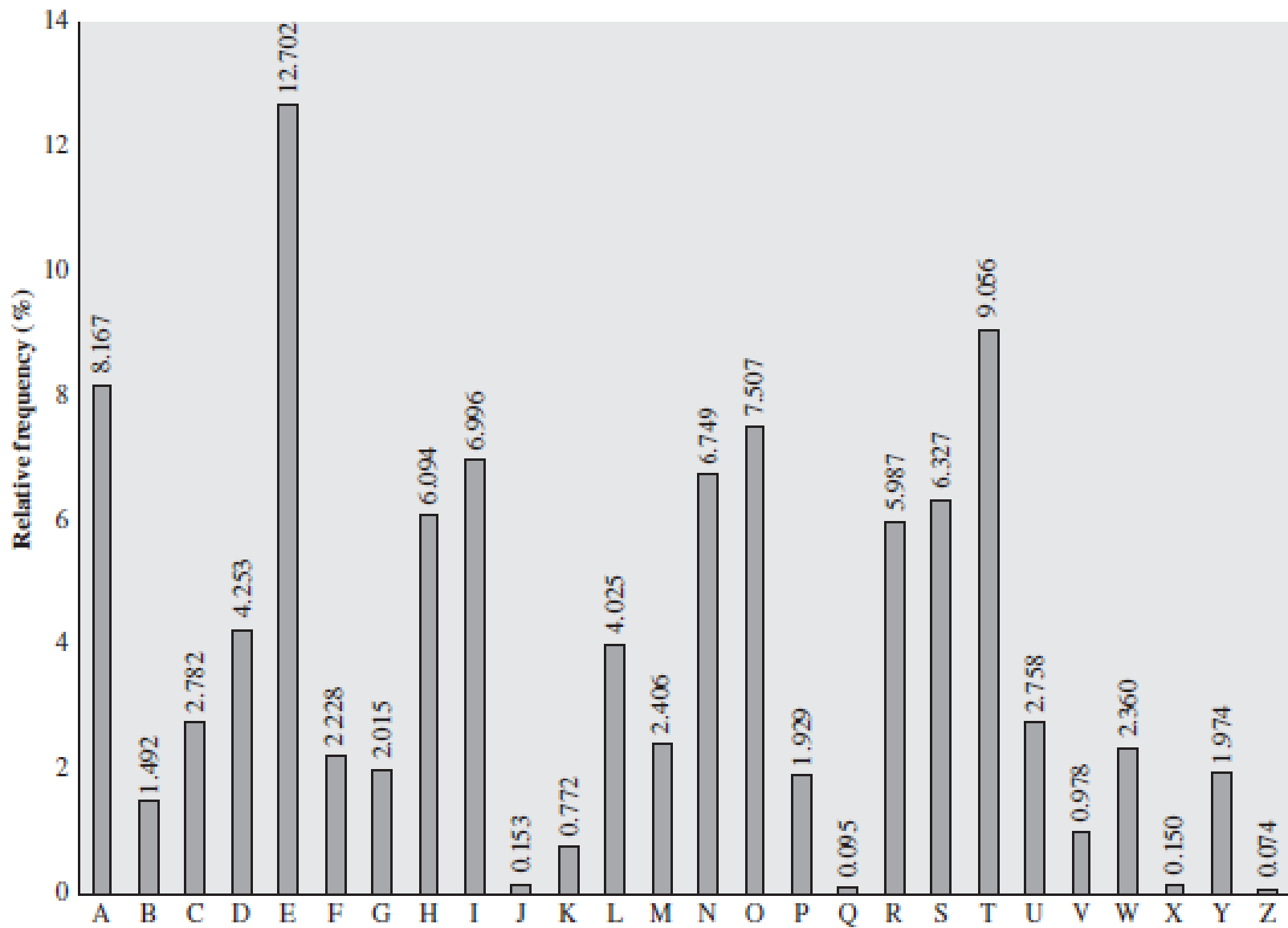
- 1) Caesar Cipher
- 2) **Monoalphabetic Cipher**
- 3) Playfair Cipher
- 4) Hill Cipher
- 5) Polyalphabetic Ciphers
- 6) One-Time Pad

2) Monoalphabetic Cipher (Simple substitution)

- It is an improvement to the Caesar Cipher.
- Instead of shifting the alphabets by some number, this scheme uses some permutation of the letters in alphabet.
- The sender and the receiver decide on a randomly selected permutation of the letters of the alphabet.
- With 26 letters in alphabet, the possible permutations are $26!$ which is equal to 4×10^{26} .

plain:	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
cipher:	y	n	l	k	x	b	s	h	m	i	w	d	p	j	r	o	q	v	f	e	a	u	g	t	z	c

Relative Frequency of Letters in English Text



Attack on Monoalphabetic Cipher

- The relative frequencies of the letters in the ciphertext (in percentages) are

P 13.33	H 5.83	F 3.33	B 1.67	C 0.00
Z 11.67	D 5.00	W 3.33	G 1.67	K 0.00
S 8.33	E 5.00	Q 2.50	Y 1.67	L 0.00
U 8.33	V 4.17	T 2.50	I 0.83	N 0.00
O 7.50	X 4.17	A 1.67	J 0.83	R 0.00
M 6.67				

Ciphertext:

uzqsovuohxmopvgpozpevsg**zw**szopfpesxudbmetsxaizvuephzhmdzshzowsf
pappdtsvpqu**zw**ymxuzuhxsxepyepopdzszufpomb**zwp**fupzhmdjudtmohmq

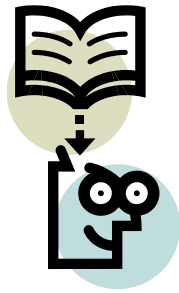
- In our ciphertext, the most common digram is ZW, which appears three times. So equate Z with t, W with h and P with e.
- Now notice that the sequence ZWP appears in the ciphertext, and we can translate that sequence as “the.”

Actual Message

it was disclosed yesterday that several informal but direct contacts have been made with political representatives of the viet cong in moscow

Attack on Monoalphabetic Cipher (Cont...)

- If the cryptanalyst knows the nature of the plaintext, then the analyst can exploit the regularities of the language.
- The relative frequency of the letters can be determined and compared to a standard frequency distribution for English.
- If the message were long enough, this technique alone might be sufficient, but because this is a relatively short message, we cannot expect an exact match.



Lecture 7



Playfair Cipher

Substitution Techniques

- 1) Caesar Cipher
- 2) Monoalphabetic Cipher
- 3) **Playfair Cipher**
- 4) Hill Cipher
- 5) Polyalphabetic Ciphers
- 6) One-Time Pad

3) Playfair Cipher

- The Playfair algorithm is based on a 5×5 matrix (**key**) of letters.
- The matrix is constructed by filling in the letters of the **keyword** (minus duplicates) from left to right and from top to bottom, and then filling in the remainder of the matrix with the remaining letters in alphabetic order. **The letters I and J count as one letter.**

Example:

Keyword= OCCURRENCE

Plaintext= TALL TREES

Playfair Cipher - Encrypt Plaintext

- Playfair, treats digrams (two letters) in the plaintext as single units and translates these units into ciphertext digrams.
- Make Pairs of letters add filler letter “X” if same letter appears in a pair.

Plaintext= TALL TREES

Plaintext= TA LX LT RE ES

- If there is an odd number of letters, then add uncommon letter to complete digram, a X/Z may be added to the last letter.

Playfair Cipher - Encrypt Plaintext

- Map each pair in key matrix

Plaintext= TA LX LT RE ES

Ciphertext= PF IZ TZ EO RT

O	C	U	R	E
N	A	B	D	F
G	H	I/J	K	L
M	P	Q	S	T
V	W	X	Y	Z

- If the letters appear in the same row, replace them with the letters immediately to the right of the original two (wrapping around to the first if necessary)
- If the letters appear in the same column, replace them with the letters immediately below the original two (wrapping around to the top if necessary)
- For example, using the table above, the letter pair **RE** would be encoded as **EO**.
- Encoded as **PF**, using the table above, the letter pair **TA** would be encoded as **PF**.

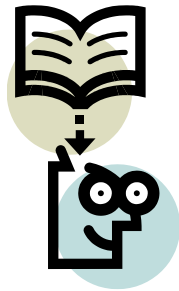
Playfair Cipher Examples

1. Key= “ engineering ” Plaintext=“ test this process ”
2. Key= “ keyword ” Plaintext=“ come to the window ”
3. Key= “ moonmission ” Plaintext=“ greet ”

E N G I R	Encrypted Message: pi tu pm gt ue lf gp xg
A B C D F	
H K L M O	
P Q S T U	
V W X Y Z	

K E Y W O	Encrypted Message: lc nk zk vf yo gq ce bw
R D A B C	
F G H I L	
M N P Q S	
T U V X Z	

M O N I S	Encrypted Message: hq cz du
A B C D E	
F G H K L	
P Q R T U	
V W X Y Z	



Lecture 8

Hill Cipher



Substitution Techniques

- 1) Caesar Cipher
- 2) Monoalphabetic Cipher
- 3) **Playfair Cipher**
- 4) Hill Cipher
- 5) Polyalphabetic Ciphers
- 6) One-Time Pad

4) Hill Cipher

- Hill cipher is based on linear algebra
- Each letter is represented by numbers from 0 to 25 and calculations are done modulo 26.
- Encryption and decryption can be given by the following formula:

Encryption:

$$C = PK \bmod 26$$

Decryption:

$$P = CK^{-1} \bmod 26$$

$$(c_1 \ c_2 \ c_3) = (p_1 \ p_2 \ p_3) \begin{pmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{pmatrix} \bmod 26$$

Hill Cipher Encryption

- To encrypt a message using the Hill Cipher we must first turn our keyword and plaintext into a matrix (a 2 x 2 matrix or a 3 x 3 matrix, etc).

Example: Key = “HILL”, Plaintext =

“EXAM”

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12
n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25

$$\text{Key Matrix } \begin{pmatrix} H & I \\ L & L \end{pmatrix} = \begin{pmatrix} 7 & 8 \\ 11 & 11 \end{pmatrix}$$

$$\text{Plaintext } \begin{pmatrix} E \\ X \end{pmatrix} \begin{pmatrix} A \\ M \end{pmatrix} = \begin{pmatrix} 4 \\ 23 \end{pmatrix} \begin{pmatrix} 0 \\ 12 \end{pmatrix}$$

Hill Cipher Encryption (Cont...)

$$\text{Key Matrix} = \begin{pmatrix} H & I \\ L & L \end{pmatrix} = \begin{pmatrix} 7 & 8 \\ 11 & 11 \end{pmatrix}$$

$$\text{Plaintext} \begin{pmatrix} E \\ X \end{pmatrix} \begin{pmatrix} A \\ M \end{pmatrix} = \begin{pmatrix} 4 \\ 23 \end{pmatrix} \begin{pmatrix} 0 \\ 12 \end{pmatrix}$$

$$C = PK \bmod 26$$

$$\begin{pmatrix} 7 & 8 \\ 11 & 11 \end{pmatrix} \begin{pmatrix} 4 \\ 23 \end{pmatrix}$$

$$7 \times 4 + 8 \times 23 = 212$$

$$11 \times 4 + 11 \times 23 = 297$$

$$\begin{pmatrix} 7 & 8 \\ 11 & 11 \end{pmatrix} \begin{pmatrix} 4 \\ 23 \end{pmatrix} = \begin{pmatrix} 212 \\ 297 \end{pmatrix}$$

$$\begin{pmatrix} 212 \\ 297 \end{pmatrix} = \begin{pmatrix} 4 \\ 11 \end{pmatrix} \bmod 26 = \begin{pmatrix} E \\ L \end{pmatrix}$$

$$\begin{pmatrix} 7 & 8 \\ 11 & 11 \end{pmatrix} \begin{pmatrix} 0 \\ 12 \end{pmatrix}$$

$$7 \times 0 + 8 \times 12 = 96$$

$$11 \times 0 + 11 \times 12 = 132$$

$$\begin{pmatrix} 7 & 8 \\ 11 & 11 \end{pmatrix} \begin{pmatrix} 0 \\ 12 \end{pmatrix} = \begin{pmatrix} 96 \\ 132 \end{pmatrix}$$

$$\begin{pmatrix} 96 \\ 132 \end{pmatrix} = \begin{pmatrix} 18 \\ 2 \end{pmatrix} \bmod 26 = \begin{pmatrix} S \\ C \end{pmatrix}$$

Ciphertext = "ELSC"

THE HILL ALGORITHM This encryption algorithm takes m successive plaintext letters and substitutes for them ciphertext letters.

For , m=3 the system can be described as

$$\mathbf{C} = \mathbf{PK} \bmod 26$$

$$(c_1 \ c_2 \ c_3) = (p \ p_2 \ p_3) \begin{pmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{pmatrix} \bmod 26$$

$$c_1 = (k_{11}p_1 + k_{12}p_2 + k_{13}p_3) \bmod 26$$

$$c_2 = (k_{21}p_1 + k_{22}p_2 + k_{23}p_3) \bmod 26$$

$$c_3 = (k_{31}p_1 + k_{32}p_2 + k_{33}p_3) \bmod 26$$

For example, consider the plaintext “paymoremoney” and use the encryption key

$$\mathbf{K} = \begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix}$$

p	a	y	m	o	r	e	m	o	n	e	y
15	0	24	12	14	17	4	12	14	13	4	24

Key = 3 x 3 matrix.

PT = pay mor emo ney

pay : (15 0 24)

$$(C_1 \ C_2 \ C_3) = (P_1 \ P_2 \ P_3) \begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix} \bmod 26$$

$$\begin{aligned} (C_1 \ C_2 \ C_3) &= (15 \ 0 \ 24) \begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix} \bmod 26 \\ &= (15 \times 17 + 0 \times 21 + 24 \times 2 \quad 15 \times 17 + 0 \times 18 + 24 \times 2 \quad 15 \times 5 + 0 \times 21 + 24 \times 19) \bmod 26 \\ &= (303 \ 303 \ 531) \bmod 26 \\ &= (17 \ 17 \ 11) \\ &= (R \ R \ L) \end{aligned}$$

mor : (12 14 17)

$$(C_1 \ C_2 \ C_3) = (P_1 \ P_2 \ P_3) \begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix} \text{ mod } 26$$

$$\begin{aligned} (C_1 \ C_2 \ C_3) &= (12 \ 14 \ 17) \begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix} \text{ mod } 26 \\ &= (12 \times 17 + 14 \times 21 + 17 \times 2 \quad 12 \times 17 + 14 \times 18 + 17 \times 2 \quad 12 \times 5 + 14 \times 21 + 17 \times 19) \text{ mod } 26 \\ &= (532 \ 490 \ 677) \text{ mod } 26 \\ &= (12 \ 22 \ 1) \\ &= (M \ W \ B) \end{aligned}$$

emo : (4 12 14)

$$(C_1 \ C_2 \ C_3) = (P_1 \ P_2 \ P_3) \begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix} \bmod 26$$

$$(C_1 \ C_2 \ C_3) = (4 \ 12 \ 14) \begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix} \bmod 26$$

$$= (4 \times 17 + 12 \times 21 + 14 \times 2 \quad 4 \times 17 + 12 \times 18 + 14 \times 2 \quad 4 \times 5 + 12 \times 21 + 14 \times 19) \bmod 26$$

$$= (348 \ 312 \ 538) \bmod 26$$

$$= (10 \ 0 \ 18)$$

$$= (K \ A \ S)$$

key : (13 4 24)

$$(C_1 \ C_2 \ C_3) = (P_1 \ P_2 \ P_3) \begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix} \text{mod } 26$$

$$(C_1 \ C_2 \ C_3) = (13 \ 4 \ 24) \begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix} \text{mod } 26$$

$$= (13 \times 17 + 4 \times 21 + 24 \times 2 \quad 13 \times 17 + 4 \times 18 + 24 \times 2 \quad 13 \times 5 + 4 \times 21 + 24 \times 19) \text{mod } 26$$

$$= (348 \ 312 \ 538) \text{mod } 26$$

$$= (15 \ 3 \ 7)$$

$$= (P \ D \ H)$$

For example, consider the plaintext “paymoremoney” and use the encryption key

$$\mathbf{K} = \begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix}$$

p	a	y	m	o	r	e	m	o	n	e	y
15	0	24	12	14	17	4	12	14	13	4	24

Key = 3 x 3 matrix.

PT = pay mor emo ney

PT	p	a	y	m	o	r	e	m	o	n	e	y
CT	R	R	L	M	W	B	K	A	S	P	D	H

Hill Cipher Decryption

$$P = CK^{-1} \bmod 26$$

Step:1 Find Inverse of key matrix

Step:2 Multiply the Multiplicative Inverse of the Determinant by the Adjoin Matrix

Step:3 Multiply inverse key matrix with ciphertext matrix to obtain plaintext matrix

Step: 1 Inverse of key matrix

2 X 2 inverse of matrix

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - cb} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

3 X 3 inverse of matrix

$$A^{-1} = \frac{1}{\text{determinant}(A)} \cdot \text{adjoin}(A)$$

Step: 1 Inverse of key matrix

$$\text{Inverse Key Matrix} = \begin{pmatrix} 7 & 8 \\ 11 & 11 \end{pmatrix}^{-1} = \frac{1}{77 - 88} \begin{pmatrix} 11 & -8 \\ -11 & 7 \end{pmatrix}$$

$$= \frac{1}{-11} \begin{pmatrix} 11 & -8 \\ -11 & 7 \end{pmatrix}$$

$$= \frac{1}{15} \begin{pmatrix} 11 & 18 \\ 15 & 7 \end{pmatrix} \text{ mod } 26$$

- $-11 \text{ mod } 26 = 15$
- Because, modulo for negative number is $= N - (B\%N)$
 $= 26 - (11\%26)$

Step: 2 Modular (Multiplicative) inverse

- The inverse of a number A is $1/A$ since $A * 1/A = 1$
e.g. the inverse of 5 is $1/5$
- In modular arithmetic we do not have a division operation.
- The modular inverse of $A \pmod C$ is A^{-1}
- **$(A * A^{-1}) \equiv 1 \pmod C$**

Example:

- The modular inverse of $A \pmod C$ is the A^{-1} value that makes

$$A * A^{-1} \pmod C = 1$$

$$A = 3, C = 11$$

Since $(3 * 4) \pmod{11} = 1$, **4** is modulo inverse of **3**

$$A = 10, C = 17, A^{-1} = ?$$

Step 2: Modular (Multiplicative) inverse

Determinants' multiplicative inverse Modulo 26												
Determinant	1	3	5	7	9	11	15	17	19	21	23	25
Inverse Modulo 26	1	9	21	15	3	19	7	23	11	5	17	25

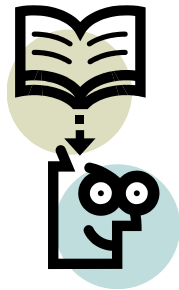
$$= \frac{1}{15} \begin{pmatrix} 11 & 18 \\ 15 & 7 \end{pmatrix} \text{ mod } 26$$

- Multiplicative inverse of $\frac{1}{15}$ is 7

Step 2: Multiply with adjoin of matrix

$$= 7 \begin{pmatrix} 11 & 18 \\ 15 & 7 \end{pmatrix} = \begin{pmatrix} 77 & 126 \\ 105 & 49 \end{pmatrix} = \begin{pmatrix} 25 & 22 \\ 1 & 23 \end{pmatrix} \pmod{26}$$

$$= \text{thus, if } K = \begin{pmatrix} 7 & 8 \\ 11 & 11 \end{pmatrix} \text{ then } K^{-1} = \begin{pmatrix} 25 & 22 \\ 1 & 23 \end{pmatrix}$$



Lecture 9



Polyalphabetic & One time pad

Substitution Techniques

- 1) Caesar Cipher
- 2) Monoalphabetic Cipher
- 3) Playfair Cipher
- 4) Hill Cipher
- 5) **Polyalphabetic Ciphers**
- 6) One-Time Pad

5) Polyalphabetic Cipher

- Monoalphabetic cipher encoded using only one fixed alphabet
- **Polyalphabetic cipher** is a substitution cipher in which the cipher alphabet for the plain alphabet may be different at different places during the encryption process.
 1. **Vigenere cipher**
 2. **Vernam cipher**

Vigenere Cipher

Keyword : **DECEPTIVE**

Key : **DECEPTIVE**

Plaintext : **WEAREDISCOVEREDSAVEYOURSELF**

$$C = (P_1 + K_1, P_2 + K_2, \dots P_m + K_m) \bmod 26$$

$$P = (C_1 - K_1, C_2 - K_2, \dots C_m - K_m) \bmod 26$$

Vignere Cipher

Key : deceptivedeceptivedeceptive

Plaintext : wearediscoveredsaveyourself

$$C = (P_1 + K_1, P_2 + K_2, \dots, P_m + K_m) \bmod 26$$

Vignere Cipher

Key : deceptive deceptive deceptive

Plaintext : wearediscoveredsaveyourself

$$C = (P_1 + K_1, P_2 + K_2, \dots, P_m + K_m) \bmod 26$$

[illegible][illegible]

Vigenere Cipher

Key : deceptivedeceptivedeceptive

Plaintext : wearediscoveredsaveyourself

$$C = (P_1 + K_1, P_2 + K_2, \dots P_m + K_m) \bmod 26$$

Key	3	4	2	4	15	19	8	21	4	3	4	2	4
PT	22	4	0	17	4	3	8	18	2	14	21	4	17
CT	25	8	2	21	19	22	16	13	6	17	25	6	21

Key	15	19	8	21	4	3	4	2	4	15	19	8	21	4
PT	4	3	18	0	21	4	24	14	20	17	18	4	11	5
CT	19	22	0	21	25	7	2	16	24	6	11	12	6	9

Vigenere Cipher

Key : deceptivedeceptivedeceptive

Plaintext : wearediscoveredsaveyourself

$$C = (P_1 + K_1, P_2 + K_2, \dots P_m + K_m) \bmod 26$$

Key : deceptivedeceptivedeceptive

Plaintext : wearediscoveredsaveyourself

Ciphertext : ZICVTWQNGRZGVTWAVZHCQYGLMGJ

$$P = (C_1 - K_1, C_2 - K_2, \dots C_m - K_m) \bmod 26$$

Vigenere Cipher

Keyword : **DECEPTIVE**

Key : **DECEPTIVE**

Plaintext : **WEAREDISCOVEREDSAVEYOURSELF**

Ciphertext : **ZICVTWQNGRZGVTWAVZHCQYGLMGJ**

$$C = (P_1 + K_1, P_2 + K_2, \dots P_m + K_m) \bmod 26$$

$$P = (C_1 - K_1, C_2 - K_2, \dots C_m - K_m) \bmod 26$$

An analyst looking at only the ciphertext would detect the repeated sequences VTW at a displacement of 9 and make the assumption that the keyword is either three or nine letters in length.

Keyword : **DECEPTIVE**

Key : **DECEPTIVE**

Plaintext : **WEAREDISCOVEREDSAVEYOURSELF**

This system
is referred as
an **autokey**
system

Vigenere Cipher Autokey System

Key : deceptivewarediscoveredsav

Plaintext : wearediscoveredsaveyourself

$$C = (P_1 + K_1, P_2 + K_2, \dots P_m + K_m) \bmod 26$$

Key : deceptivewarediscoveredsav

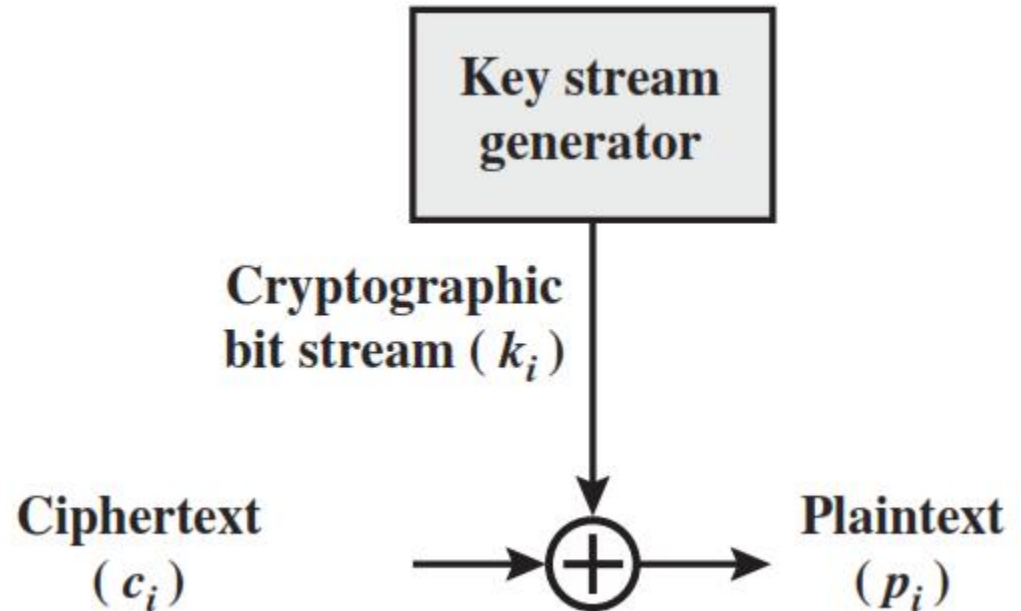
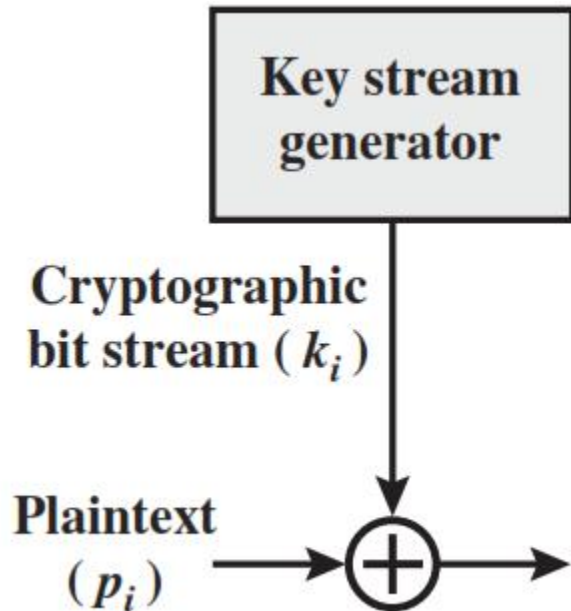
Plaintext : wearediscoveredsaveyourself

Ciphertext : ZICVTWQNGKZEIIGASXSTSLVWLA

$$P = (C_1 - K_1, C_2 - K_2, \dots C_m - K_m) \bmod 26$$

Vernam Cipher

- The ciphertext is generated by applying the logical XOR operation to the individual bits of plaintext and the key stream.



Vernam Cipher

$$c_i = p_i \oplus k_i$$

where

p_i = i th binary digit of plaintext

k_i = i th binary digit of key

c_i = i th binary digit of ciphertext

$\oplus$ = exclusive-or (XOR) operation

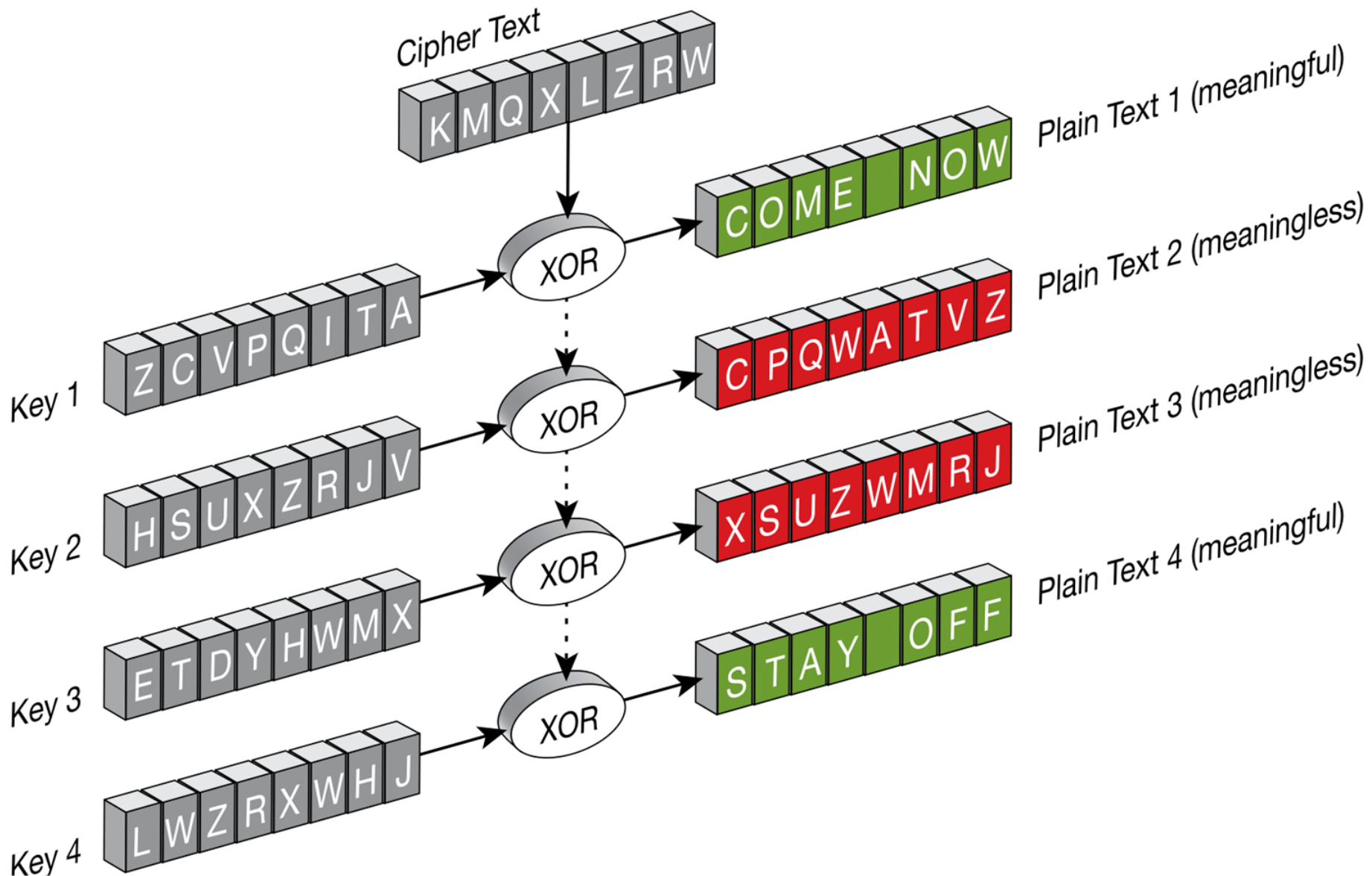
The essence of this technique is the means of construction of the key. Vernam proposed the use of a running loop of tape that eventually repeated the key, so that in fact the system worked with a very long but repeating keyword. Although such a scheme, with a long key, presents formidable cryptanalytic difficulties, it can be broken with sufficient ciphertext, the use of known or probable plaintext sequences, or both.

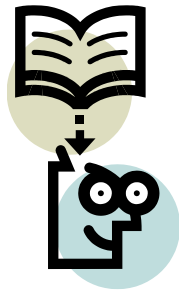
Substitution Techniques

- 1) Caesar Cipher
- 2) Monoalphabetic Cipher
- 3) Playfair Cipher
- 4) Hill Cipher
- 5) Polyalphabetic Ciphers
- 6) **One-Time Pad**

One time pad

- The one-time pad, which is a provably secure cryptosystem, was developed by Gilbert Vernam in 1918.
- The message is represented as a binary string (a sequence of 0's and 1's using a coding mechanism such as ASCII coding).
- **The key is a truly random sequence of 0's and 1's of the same length as the message.**
- message = 'IF'
- then its ASCII code =(1001001 1000110)
- key = (1010110 0110001)
- *Encryption:*
 - 1001001 1000110 plaintext
 - 1010110 0110001 key
 - 0011111 1110110 ciphertext





Lecture 10



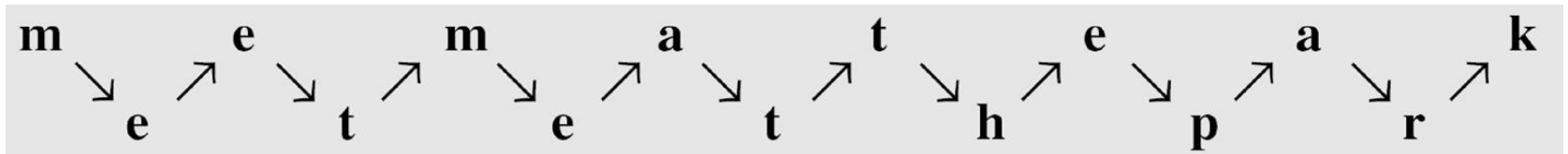
Transposition Techniques

Transposition Techniques

- 1) Rail Fence Transposition
- 2) Row Column Transposition

Rail Fence Transposition

- A transposition cipher does not substitute one symbol for another, instead it changes the location of the symbols.
- The simplest such cipher is the **rail fence technique**, in which the plaintext is written down as a sequence of diagonals and then read off as a sequence of rows.
- For example, to send the message **“Meet me at the park”** to Bob, Alice writes



- She then creates the ciphertext **“MEMATEAKETETHPR”**.

Rail Fence Transposition

- For example, to send the message **“Thank you very much”** to Alice, What Bob writes ? Use depth 3
- He then creates the ciphertext **“TKVMHNYUEYUHAORE”**.

Transposition Techniques

- 1) Rail Fence Transposition
- 2) Row Column Transposition

Row Column Transposition

- A more complex scheme is to write the message in a rectangle, row by row, and read the message off, column by column, but permute the order of the columns.
- The order of the columns then becomes the key to the algorithm.

Key:

4 3 1 2 5 6 7

Plaintext:

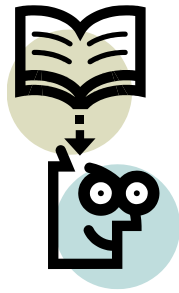
a	t	t	a	c	k	p
o	s	t	p	o	n	e
d	u	n	t	i	l	t
w	o	a	m	x	y	z

Ciphertext: TTNAAPTMTSUOAODWCOIXKNLYPETZ

Row Column Transposition

Ciphertext: “Kill corona virus at twelve am tomorrow”

Ciphertext: “LATARLVTMOINAERKOSVOCIW TWOREOY”.



Lecture 11



Steganography

Steganography

A plaintext message may be hidden in one of two ways.

The methods of **steganography** conceal the existence of the message, whereas the methods of cryptography render the message unintelligible to outsiders by various transformations of the text

A simple form of steganography, but one that is time-consuming to construct, is one in which an arrangement of words or letters within an apparently innocuous text spells out the real message.

Steganography

Example:

Simply encrypt correct reading exactly twice

Simply encrypt correct reading exactly twice

Secret

Steganography

- 1) Character marking
- 2) Invisible ink
- 3) Pin punctures
- 4) Typewriter correction ribbon

Character marking

Selected letters of printed or typewritten text are over written in pencil.

The marks are ordinarily not visible unless the paper is held at an angle to bright light.

Invisible ink

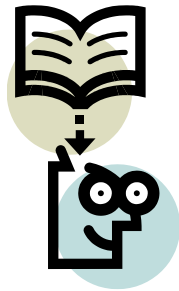
A number of substances can be used for writing but leave no visible trace until heat or some chemical is applied to the paper.

Pin punctures

Small pin punctures on selected letters are ordinarily not visible unless the paper is held up in front of a light.

Typewriter correction ribbon

Used between lines typed with a black ribbon, the results of typing with the correction tape are visible only under a strong light.



Lecture 12



Modular arithmetic

Congruence

★ In cryptography, congruence($\equiv$) instead of equality(=).

Examples:

$$15 \equiv 3 \pmod{12}$$



Congruence

★ In cryptography, congruence($\equiv$) instead of equality(=).

Examples:

$$15 \equiv 3 \pmod{12}$$

$$23 \equiv 11 \pmod{12}$$



Congruence

★ In cryptography, congruence($\equiv$) instead of equality(=).

Examples:

$$15 \equiv 3 \pmod{12}$$

$$23 \equiv 11 \pmod{12}$$

$$33 \equiv 3 \pmod{10}$$



Congruence

$$10 \equiv -2 \pmod{12}$$

$$\therefore a \equiv b \pmod{m}$$

$$\text{i.e. } a = km + b$$

	0	k
12	10	a
	0	
	10	b
	(-2)	

Modular arithmetic

1. $[(a \bmod n) + (b \bmod n)] \bmod n = (a + b) \bmod n$
2. $[(a \bmod n) - (b \bmod n)] \bmod n = (a - b) \bmod n$
3. $[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$

Modular arithmetic

1. $[(a \bmod n) + (b \bmod n)] \bmod n = (a + b) \bmod n$
2. $[(a \bmod n) - (b \bmod n)] \bmod n = (a - b) \bmod n$
3. $[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$

Example:

$$\begin{aligned} [(15 \bmod 8) + (11 \bmod 8)] \bmod 8 &= (15 + 11) \bmod 8 \\ &= 26 \bmod 8 \\ &= 2 \end{aligned}$$

Modular arithmetic

1. $[(a \bmod n) + (b \bmod n)] \bmod n = (a + b) \bmod n$
2. $[(a \bmod n) - (b \bmod n)] \bmod n = (a - b) \bmod n$
3. $[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$

Example:

$$\begin{aligned} [(15 \bmod 8) - (11 \bmod 8)] \bmod 8 &= (15 - 11) \bmod 8 \\ &= 4 \bmod 8 \\ &= 4 \end{aligned}$$

Modular arithmetic

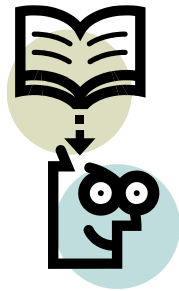
1. $[(a \bmod n) + (b \bmod n)] \bmod n = (a + b) \bmod n$
2. $[(a \bmod n) - (b \bmod n)] \bmod n = (a - b) \bmod n$
3. $[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$

Example:

$$\begin{aligned} [(15 \bmod 8) \times (11 \bmod 8)] \bmod 8 &= (15 \times 11) \bmod 8 \\ &= 165 \bmod 8 \\ &= 5 \end{aligned}$$

Modular arithmetic

Property	Expression
Commutative Laws	$(a + b) \bmod n = (b + a) \bmod n$ $(a \times b) \bmod n = (b \times a) \bmod n$
Associative Laws	$[(a + b) + c] \bmod n = [a + (b + c)] \bmod n$ $[(a \times b) \times c] \bmod n = [a \times (b \times c)] \bmod n$
Distributive Laws	$[a \times (b + c)] \bmod n = [(a \times b) + (a \times c)] \bmod n$
Identities	$(0 + a) \bmod n = a \bmod n$ $(1 \times a) \bmod n = a \bmod n$
Additive Inverse	For each $a \in \mathbb{Z}_n$, there exists a $'-a'$ such that $a + (-a) \equiv 0 \bmod n$



Lecture 14

Algebraic Structure



Algebraic structures

The combination of the set and the operations that are applied to the elements of the set is called an algebraic structure

The study of three algebraic structures allows us to use sets in which operations similar to addition/subtraction and multiplication/division can be used with the set.

Group

Ring

Field

Group

A group is a set equipped with an operation (e.g., addition or multiplication) that satisfies the following properties:

A1 Closure

A3 Identity

A2 Associativity

A4 Inverse Element

Closure: Performing the operation on two elements of the set results in an element within the set.

Associativity: The operation is associative.

Identity Element: There is an element in the set such that operation with any element leaves it unchanged.

Inverse Element: Every element has an inverse.

Group

Applications in Cryptography

Elliptic Curve Cryptography (ECC)

Diffie-Hellman Key Exchange

RSA Encryption

Finite Group & Infinite Group

If a group has a finite number of elements, it is referred to as a finite group, and the order of the group is equal to the number of elements in the group. Otherwise, the group is an infinite group.

Abelian Group

A group is said to be abelian if it satisfies the following additional condition

A5 Commutative

Group

Property			Explanation
	Group	Closure	
		Associative	
		Identity element	
		Inverse element	

Group

Property			Explanation
Abelian Group	Group	Closure	$a, b \in G$, then $(a \bullet b) \in G$.
		Associative	$a \bullet (b \bullet c) = (a \bullet b) \bullet c$ for all $a, b, c \in G$.
		Identity element	$(a \bullet e) = (e \bullet a) = a$ for all $a, e \in G$.
		Inverse element	$(a \bullet a') = (a' \bullet a) = e$ for all $a, a' \in G$.
	Commutative		

Abelian Group

Property			Explanation
Abelian Group	Group	Closure	$a, b \in G$, then $(a \bullet b) \in G$.
		Associative	$a \bullet (b \bullet c) = (a \bullet b) \bullet c$ for all $a, b, c \in G$.
		Identity element	$(a \bullet e) = (e \bullet a) = a$ for all $a, e \in G$.
		Inverse element	$(a \bullet a') = (a' \bullet a) = e$ for all $a, a' \in G$.
	Commutative		$(a \bullet b) = (b \bullet a)$ for all $a, b \in G$.

Notations

$\mathbb{N} \rightarrow$ Set of all natural numbers.

$\mathbb{W} \rightarrow$ Set of all whole numbers.

$\mathbb{Z} \rightarrow$ Set of all integers.

$\mathbb{C} \rightarrow$ Set of all complex numbers.

$\mathbb{Q} \rightarrow$ Set of all rational numbers.

$\mathbb{R} \rightarrow$ Set of all real numbers.

$\mathbb{Z}^+ \rightarrow$ Set of all positive integers.

$\mathbb{Z}^- \rightarrow$ Set of all negative integers.

Group

Question: Is $(\mathbb{Z}, +)$ a group?

Solution:

$\mathbb{Z} = \{ \dots, -3, -2, -1, 0, 1, 2, 3, \dots \}$

CAIN Property	Explanation	Satisfied?
Closure	If $a, b \in G$, then $(a \bullet b) \in G$.	
Associative	$a \bullet (b \bullet c) = (a \bullet b) \bullet c$ for all $a, b, c \in G$.	
Identity element	$(a \bullet e) = (e \bullet a) = a$ for all $a \in G$.	
Inverse element	$(a \bullet a') = (a' \bullet a) = e$ for all $a, a' \in G$.	
Commutative	$(a \bullet b) = (b \bullet a)$ for all $a, b \in G$.	

Group

Question: Is $(\mathbb{Z}, +)$ a group?

Solution:

$\mathbb{Z} = \{ \dots, -3, -2, -1, 0, 1, 2, 3, \dots \}$

CAIN Property	Explanation	Satisfied?
Closure	If $a, b \in G$, then $(a \bullet b) \in G$. If $a = 5, b = -2 \in \mathbb{Z}$ then $(a + b) = -3 \in \mathbb{Z}$	✓
Associative	$a \bullet (b \bullet c) = (a \bullet b) \bullet c$ for all $a, b, c \in G$. $5 + (3 + 7) = (5 + 3) + 7 \in \mathbb{Z}$	✓
Identity element	$(a \bullet e) = (e \bullet a) = a$ for all $a \in G$. $(5 + 0) = (0 + 5) = 5$ for all $a \in G$.	✓
Inverse element	$(a \bullet a') = (a' \bullet a) = e$ for all $a, a' \in G$. $(5 + -5) = (-5 + 5) = 0$ for all $5, -5 \in \mathbb{Z}$	✓
Commutative	$(a \bullet b) = (b \bullet a)$ for all $a, b \in G$.	

Abelian Group

Question: Is $(\mathbb{Z}, +)$ a group?

Solution:

$\mathbb{Z} = \{ \dots, -3, -2, -1, 0, 1, 2, 3, \dots \}$ is an abelian group.

CAIN Property	Explanation	Satisfied?
Closure	If $a, b \in G$, then $(a \bullet b) \in G$. If $a = 5, b = -2 \in \mathbb{Z}$ then $(a + b) = -3 \in \mathbb{Z}$	✓
Associative	$a \bullet (b \bullet c) = (a \bullet b) \bullet c$ for all $a, b, c \in G$. $5 + (3 + 7) = (5 + 3) + 7 \in \mathbb{Z}$	✓
Identity element	$(a \bullet e) = (e \bullet a) = a$ for all $a \in G$. $(5 + 0) = (0 + 5) = 5$ for all $a \in G$.	✓
Inverse element	$(a \bullet a') = (a' \bullet a) = e$ for all $a, a' \in G$. $(5 + -5) = (-5 + 5) = 0$ for all $5, -5 \in \mathbb{Z}$	✓
Commutative	$(a \bullet b) = (b \bullet a)$ for all $a, b \in G$. $(5 + 9) = (9 + 5)$ for all $9, 5 \in \mathbb{Z}$.	✓

Rings

A ring is an algebraic structure consisting of a set equipped with two binary operations, typically addition and multiplication, satisfying:

A1 - A4 Group

A5 Abelian Group

M1 Closure under multiplication

M2 Associativity of multiplication

M3 Distributive laws

Closure under multiplication

If $a, b \in R$, then $ab \in R$

Multiplication is associative.

$a(bc) = (ab)c$ for all a, b, c in R

Multiplication distributes over addition

$a(b + c) = ab + ac$ for all a, b, c in R .

$(a + b)c = ac + bc$ for all a, b, c in R .

Commutative Ring

A ring is said to be commutative if it satisfies the following conditions:

A1 - A4 Group

A5 Abelian Group

M1- M3 Ring

M4 Commutativity of multiplication

Commutativity of multiplication

$ab = ba$ for all a, b in R

Integral Domain

we define an integral domain, which is a commutative ring that obeys the following axioms

A1 - A4 Group

A5 Abelian Group

M1- M3 Ring

M4 Commutativity Ring

M5 Multiplicative identity

M6 No zero divisors:

Multiplicative identity

There is an element 1 in R such that $a1 = 1a = a$ for all a in R

No zero divisors

If a, b in R and $ab = 0$, then either $a = 0$ or $b = 0$.

Rings

Applications in Cryptography

Lattice-based Cryptography

Homomorphic Encryption

RSA and Modular Arithmetic

Fields

A field is a ring with the additional property that every nonzero element has a multiplicative inverse.

A1 - M6 Group & Rings

M7 Multiplicative inverse

Multiplicative inverse

For each a in F , except 0 , there is an element a^{-1} in F such that $aa^{-1} = (a^{-1})a = 1$.

Fields

Applications in Cryptography

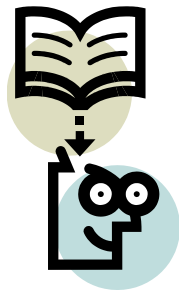
Finite Fields, Most cryptographic algorithms operate over finite fields

AES Encryption

Elliptic Curve Cryptography

Error-Correcting Codes

Polynomial-based Schemes



Lecture 15

Checking of primeness



Checking of Primeness

Fermat's Theorem

Miller-Rabin Algorithm

Fermat's Theorem

Is 'p' prime?

Test:

$a^p - a \rightarrow$ 'p' is prime if this is a multiple of 'p' for all $1 \leq a < p$.

Question 1: Is 5 prime?

Solution:

$a^p - a \rightarrow$ 'p' is prime if this is a multiple of 'p' for all $1 \leq a < p$.

$$1^5 - 1 = 1 - 1 = 0$$

$$2^5 - 2 = 32 - 2 = 30$$

$$3^5 - 3 = 243 - 3 = 240$$

$$4^5 - 4 = 1024 - 4 = 1020$$

$\therefore 5$ is prime

Fermat's Theorem

Is 'p' prime?

Test:

$a^p - a \rightarrow$ 'p' is prime if this is a multiple of 'p' for all $1 \leq a < p$.

Question 2: Is 3753 prime?

Solution: $1^{3753} - 1$

$$2^{3753} - 2$$

$$3^{3753} - 3$$

$$4^{3753} - 4$$

...

$$3752^{3753} - 3752$$

Fermat's Theorem

Is 'p' prime?

Test:

$a^p - a \rightarrow$ 'p' is prime if this is a multiple of 'p' for all $1 \leq a < p$.

Question: Is 561 prime?

Answer :

Miller-Rabin Algorithm

Algorithm

Step 1: Find $n-1 = 2^k \times m$

Step 2: Choose 'a' such that $1 < a < n-1$

Step 3: Compute $b_0 = a^m \pmod{n}$, ... , $b_i = b_{i-1}^2 \pmod{n}$

+1 $\rightarrow$ Composite

-1 $\rightarrow$ Probably Prime

Miller-Rabin Algorithm

Question: Is 561 prime?

Answer :

Step 1:

$$n-1 = 2^k \times m$$

$$560 = 2^4 \times 35$$

So $k = 4$, and $m = 35$

Step 2:

Choosing $a = 2$; $1 < 2 < 560$

Step 3:

Compute $b_0 = a^m \pmod{n}$

$$b_0 = a^m \pmod{n}$$

$$b_0 = 2^{35} \pmod{561} = 263$$

Is $b_0 = \pm 1 \pmod{561}$? **NO**

So calculate b_1

$$b_1 = b_0^2 \pmod{n}$$

$$b_1 = 263^2 \pmod{561}$$

$$b_1 = 166$$

Is $b_1 = \pm 1 \pmod{561}$? **NO**

$$b_2 = b_1^2 \pmod{n}$$

$$b_2 = 166^2 \pmod{561}$$

$$b_2 = 67$$

Is $b_2 = \pm 1 \pmod{561}$? **NO**

$$b_3 = b_2^2 \pmod{n}$$

$$b_3 = 67^2 \pmod{561}$$

$$b_3 = 1 \rightarrow \text{Composite}$$

$\therefore$ 561 is composite.

Solve $23^3 \bmod 30$.

$$\begin{aligned} 23^3 \bmod 30 &= -7^3 \bmod 30 \quad || \quad 23 \bmod 30 \text{ can be } 23 \text{ or } -7. \\ &= -7^3 \bmod 30 \\ &= -7^2 \times -7 \bmod 30 \\ &= 49 \times -7 \bmod 30 \\ &= -133 \bmod 30 \\ &= -13 \bmod 30 \\ &= 17 \bmod 30 \end{aligned}$$

$$23^3 \bmod 30 = 17$$

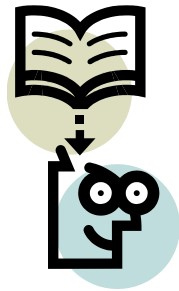
UNIT- 4

Modern symmetric key ciphers



Unit-4

- Modern block ciphers
- Modern stream ciphers
- Data Encryption standard
- Advanced encryption standard
- Electronic Code Book Mode
- Cipher Block Chaining Mode
- Cipher Feedback Mode
- Output Feedback Mode



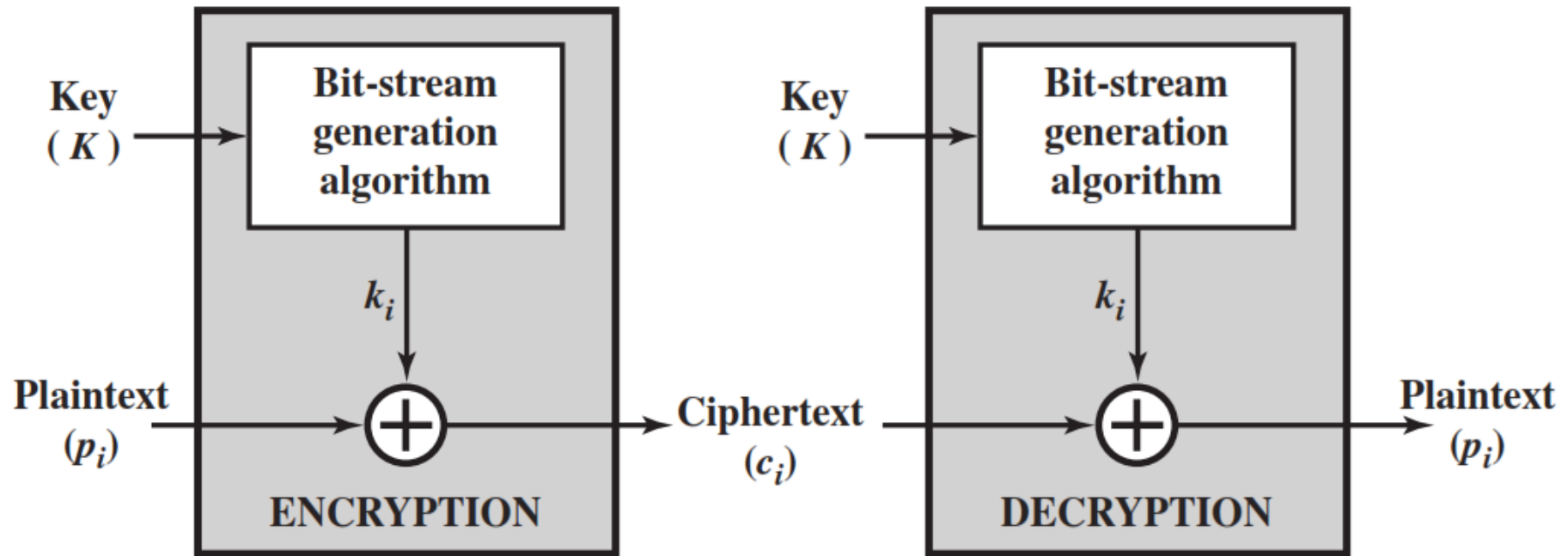
Lecture 16

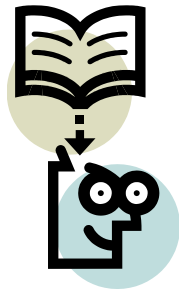
Stream Cipher



Stream Cipher

- A **stream cipher** is one that encrypts a digital data stream one bit or one byte at a time.
- Examples of classical stream ciphers are Autokeyed Vigenère cipher, A5/1, RC4 and Vernam cipher.





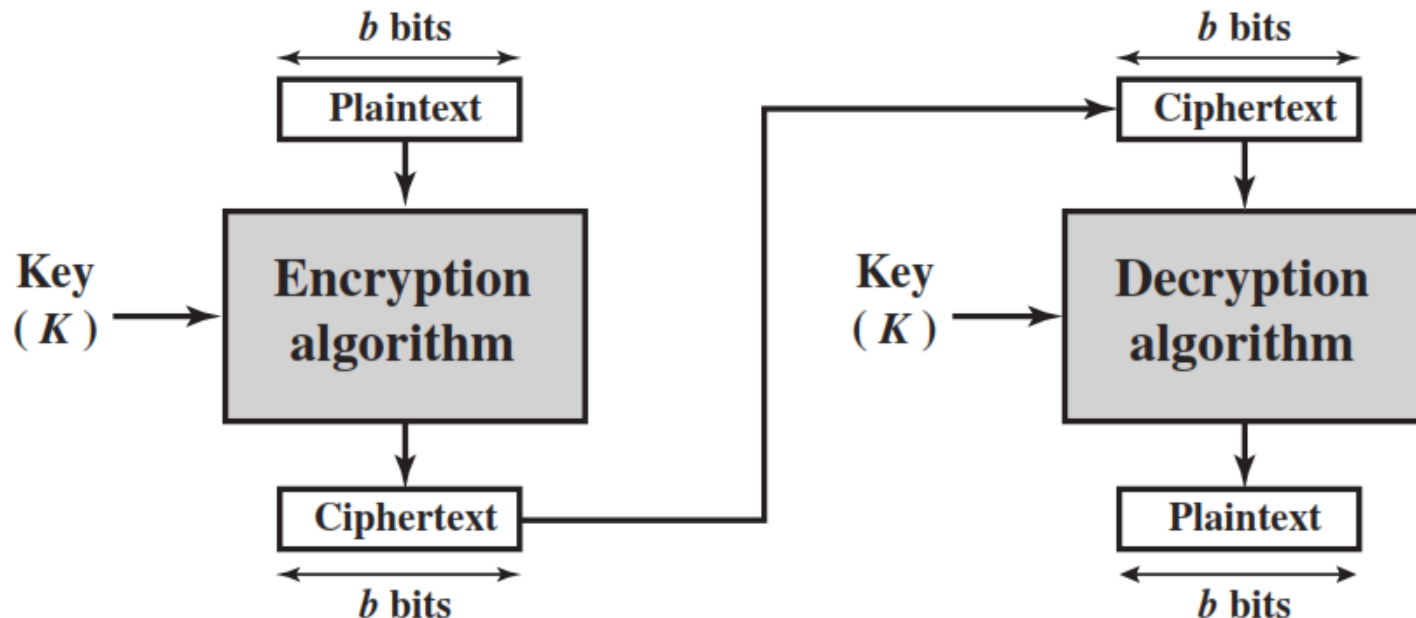
Lecture 17

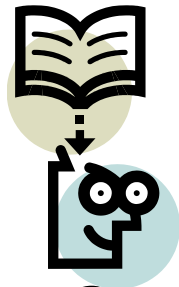
Block Cipher



Block Cipher

- A **block cipher** is one in which a block of plaintext is treated as a whole and used to produce a ciphertext block of equal length.
- Typically, a block size of **64 or 128** bits is used.
- Examples are Feistel Cipher, DES, Triple DES and AES





Lecture 18



Confusion and Diffusion

Confusion and Diffusion

- **Confusion** hides the relationship between the ciphertext and the key.
- Making relationship between EK and CT as complex as possible.
- This is achieved by the use of a complex substitution algorithm.
- Example: Substitution
- **Diffusion** hides the relationship between the ciphertext and the plaintext.
- This is achieved by having each plaintext digit affect the value of many ciphertext digits.
- Examples: Transpositions or Permutation

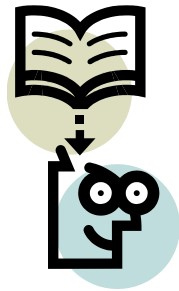
UNIT- 4

Modern symmetric key ciphers



Unit-4

- Modern block ciphers
- Modern stream ciphers
- Data Encryption standard
- Advanced encryption standard
- Electronic Code Book Mode
- Cipher Block Chaining Mode
- Cipher Feedback Mode
- Output Feedback Mode

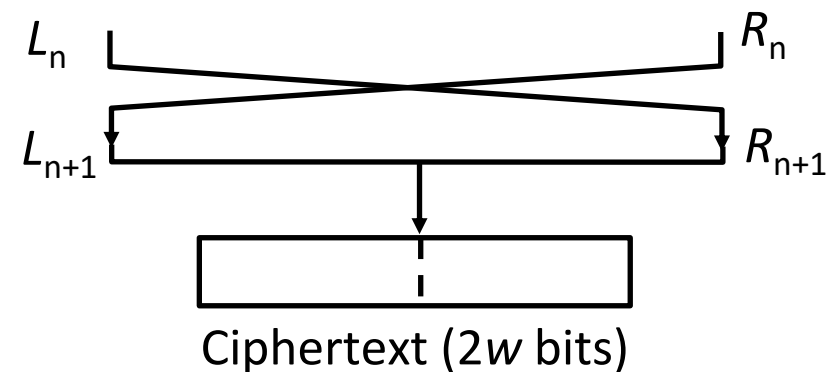
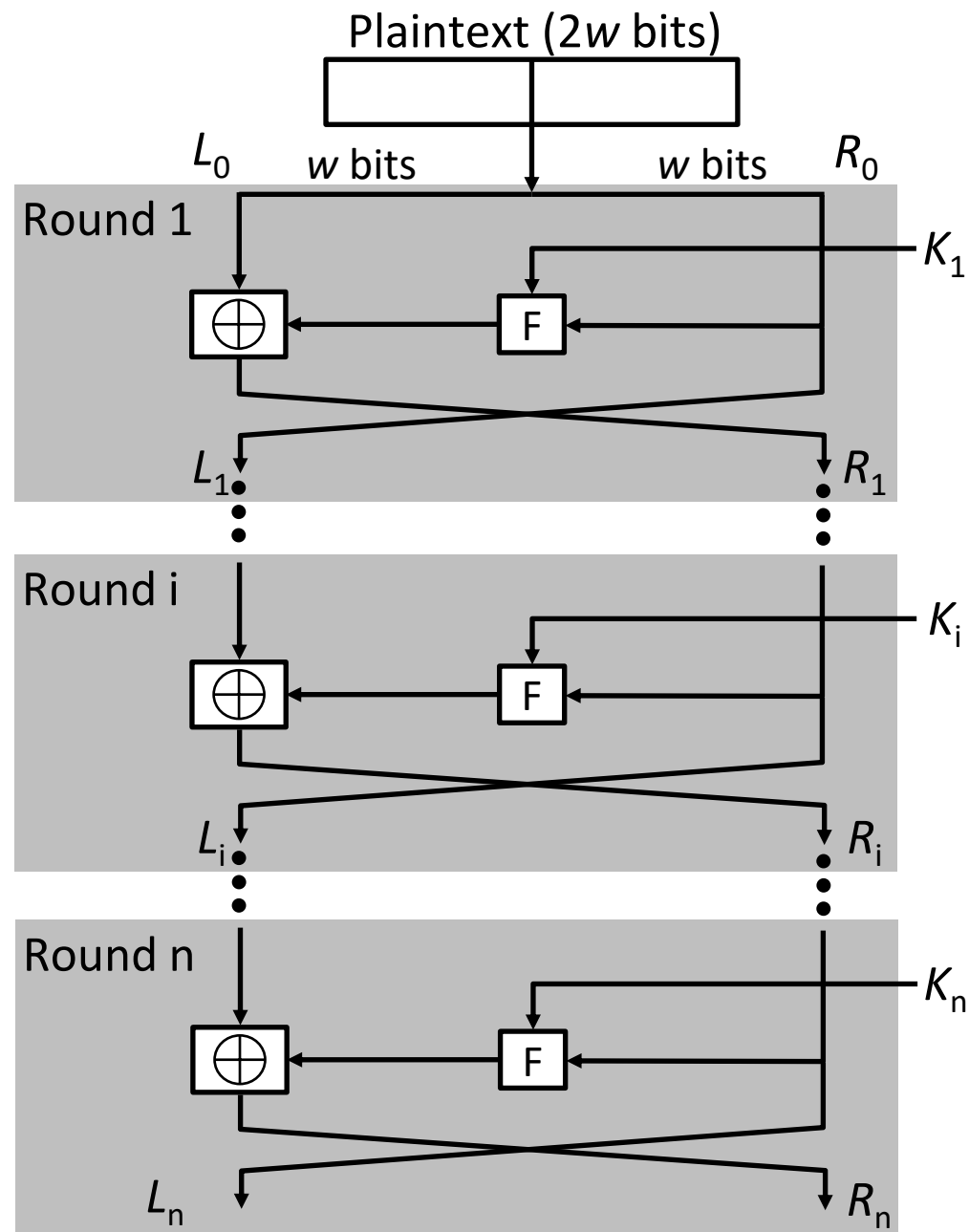


Lecture 19

Feistel Cipher



Feistel Cipher Structure Or Block Cipher Structure



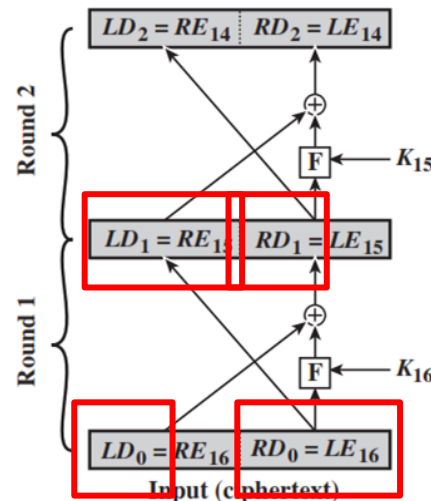
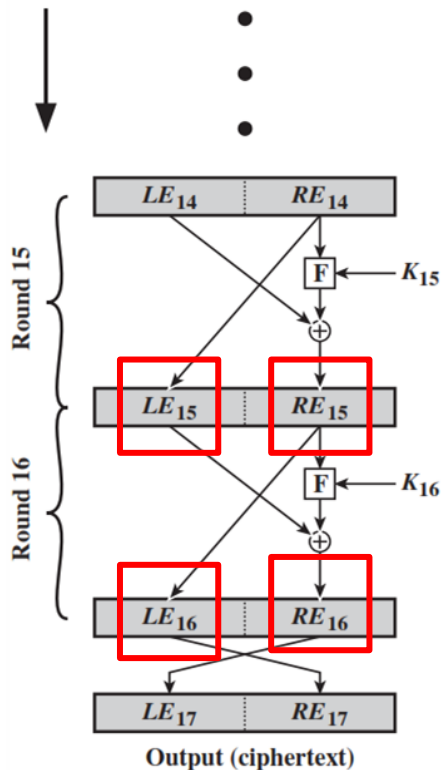
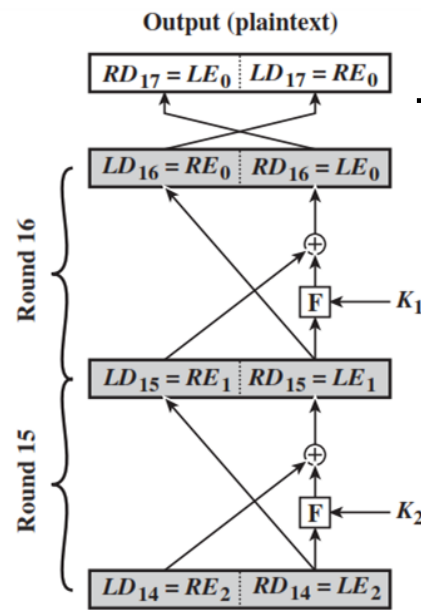
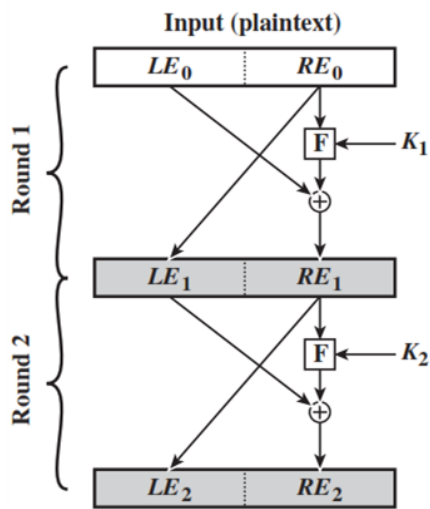
Feistel Cipher Structure

- Input plaintext block of length $2w$ bits
- key $K = n$ bits , Sub-keys: $K_1, K_2, \dots, K_n$ (Derived from K)
- All rounds have the same structure.
- A **substitution** is performed by taking exclusive-OR on left half(L_i) of the data and the output of round function F which has inputs right half(R_i) and sub key k_i .
- A **permutation** is performed that consists of interchange of two halves of data.
- This structure is called **Substitution-Permutation Network** (SPN)

Feistel Network Factors

- **Block size:** Common block size of 64-bit. However, the new algorithms use a 128-bit, 256-bit block size.
- **Key size:** Key sizes of 64 bits or less are now widely considered to be insufficient. These days at least 128 bit, more better, e.g. 192 or 256 bit
- **Number of rounds:** A typical size is 16 rounds.
- **Round function F:** Again, greater complexity generally means greater resistance to cryptanalysis.
- **Subkey generation algorithm:** Greater complexity in this algorithm should lead to greater difficulty of cryptanalysis.

Feistel Encryption & Decryption



- Prove that o/p of first round of Decryption is equal to 32-bit swap of i/p of 16th round of Encryption

- $LD_1 = RE_{15}$ & $RD_1 = LE_{15}$

- On Encryption Side:

$$LE_{16} = RE_{15}$$

$$RE_{16} = LE_{15} \oplus F(RE_{15}, K_{16})$$

- On Decryption Side:

$$LD_1 = RD_0 = LE_{16} = RE_{15}$$

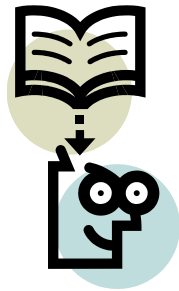
$$RD_1 = LD_0 \oplus F(RD_0, K_{16})$$

$$= RE_{16} \oplus F(RE_{15}, K_{16})$$

$$= [LE_{15} \oplus F(RE_{15}, K_{16})] \oplus F(RE_{15}, K_{16})$$

Thus,

$$LD_1 = RE_{15} \text{ \& \; } RD_1 = LE_{15} \oplus C$$



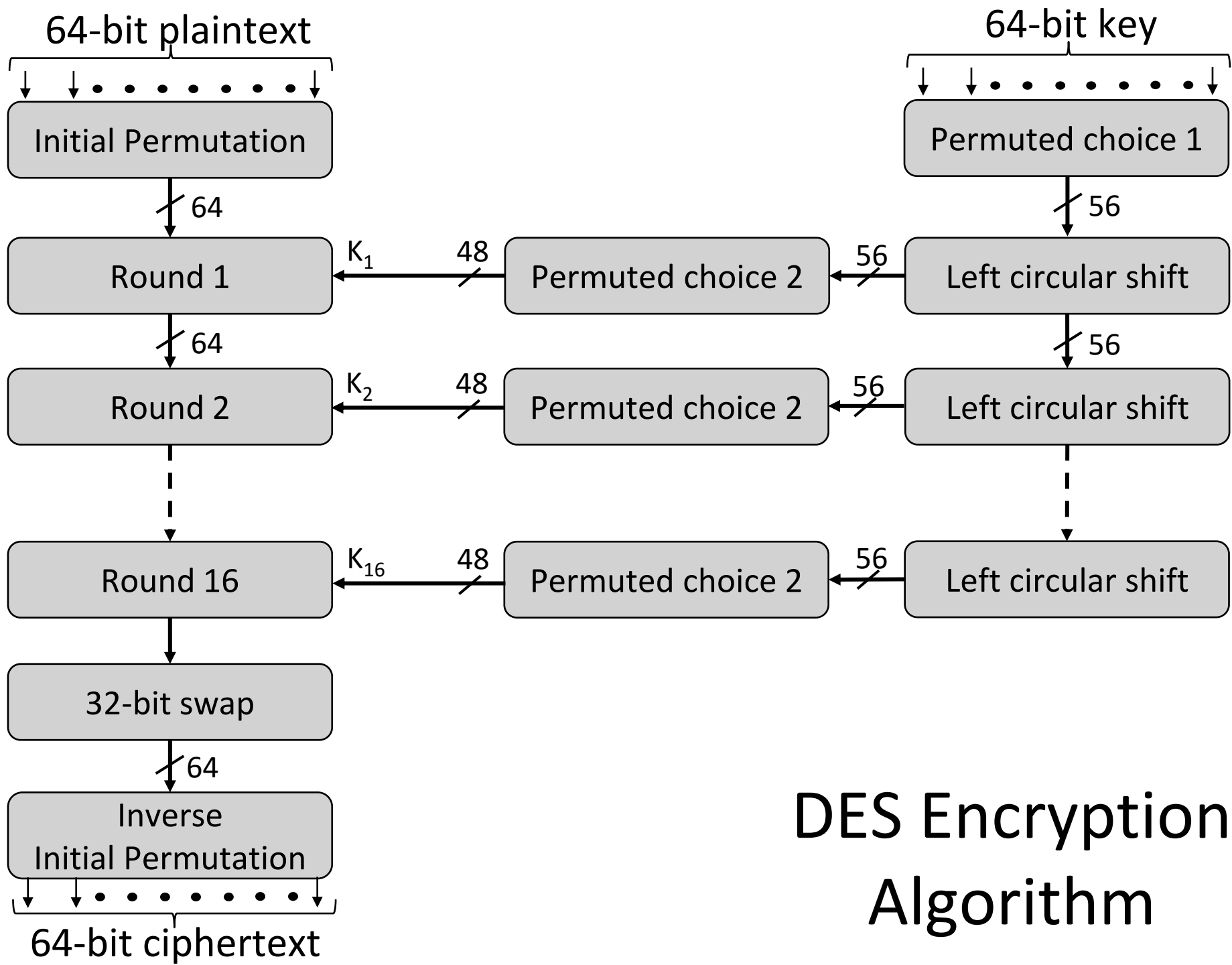
Lecture 20



Data Encryption Standard

Data Encryption Standard (DES)

- Type: Block Cipher
- Block Size : 64-bit
- Key Size: 64-bit, with only 56-bit effective
- Number of Rounds: 16



1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

Initial Permutation

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

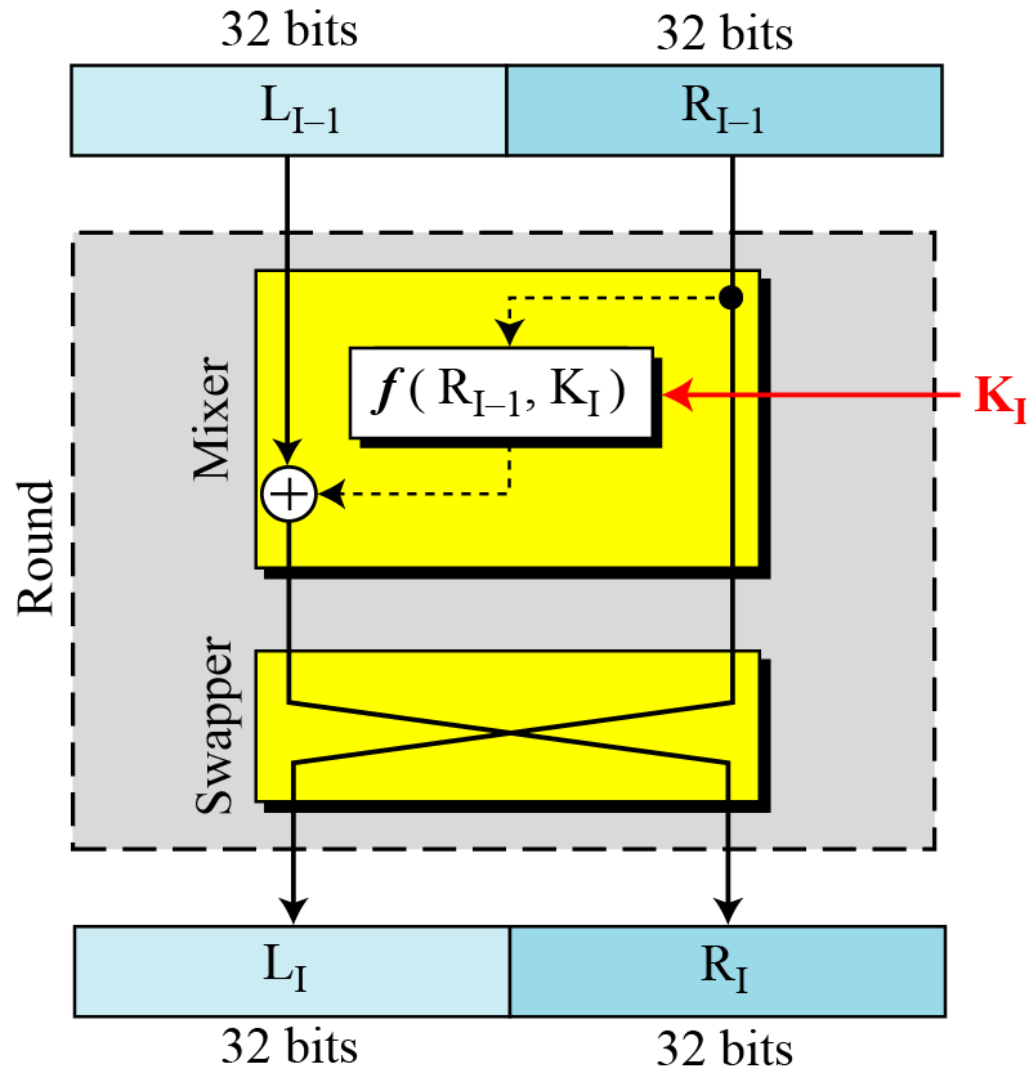
Inverse Initial Permutation

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

DES Encryption Algorithm (Cont...)

- First, the 64-bit plaintext passes through an **initial permutation** (IP) that rearranges the bits to produce the permuted input.
- This is followed by a phase consisting of sixteen rounds of the same function, which involves both **permutation** and **substitution** functions.
- Finally, the preoutput is passed through a permutation that is the **inverse of the initial permutation** function, to produce the 64-bit ciphertext.
- The 56-bit key is passed through a **permutation function**.
- For each of the sixteen rounds, a subkey (K_i) is produced by the combination of a **left circular shift** and a **permutation**.

DES Single Round



DES Single Round (Cont...)

1. Key Transformation

- Permutation of selection of sub-key from original key

2. Expansion Permutation (E-table)

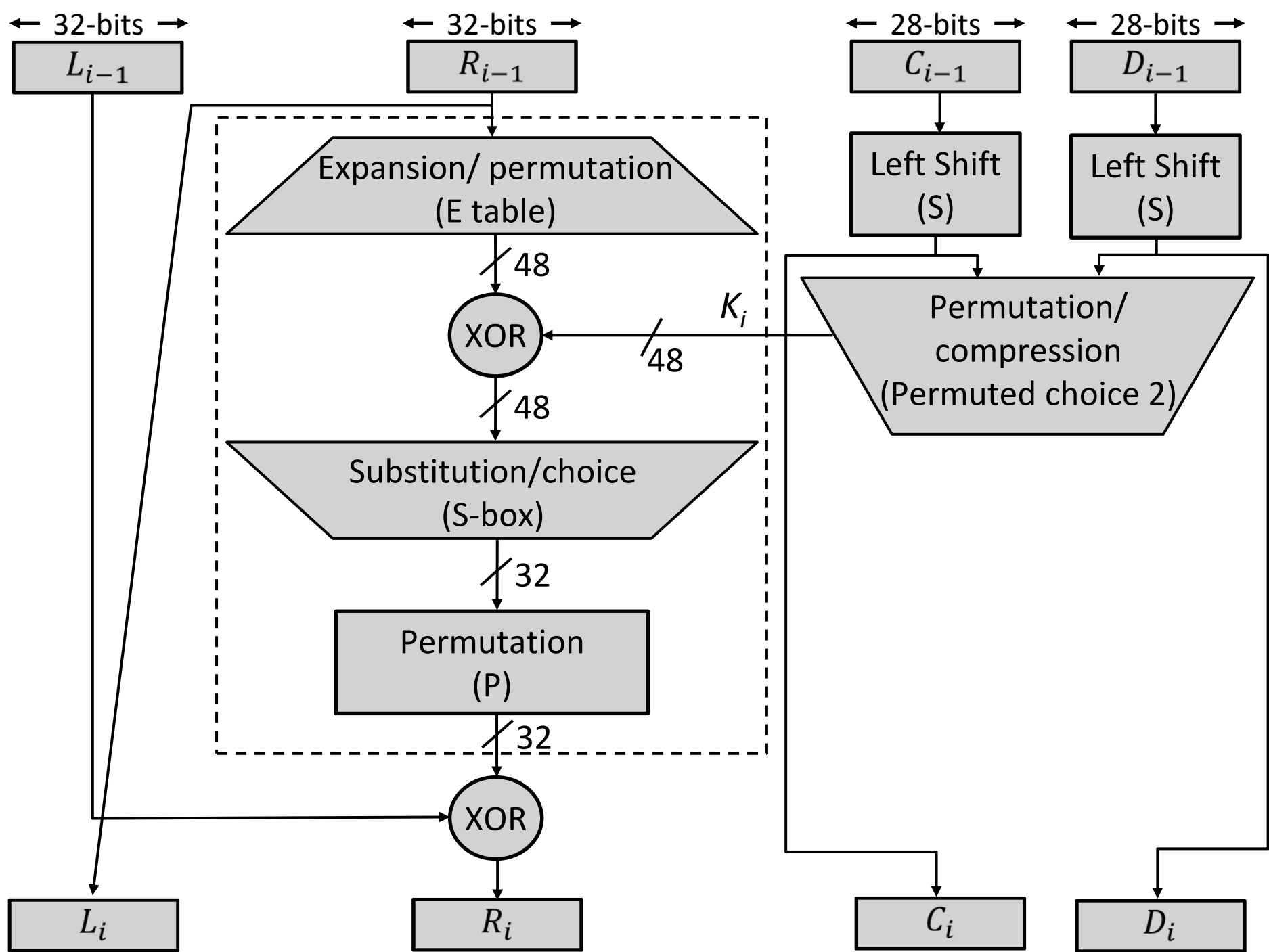
- Right half is expanded from 32-bits to 48-bits

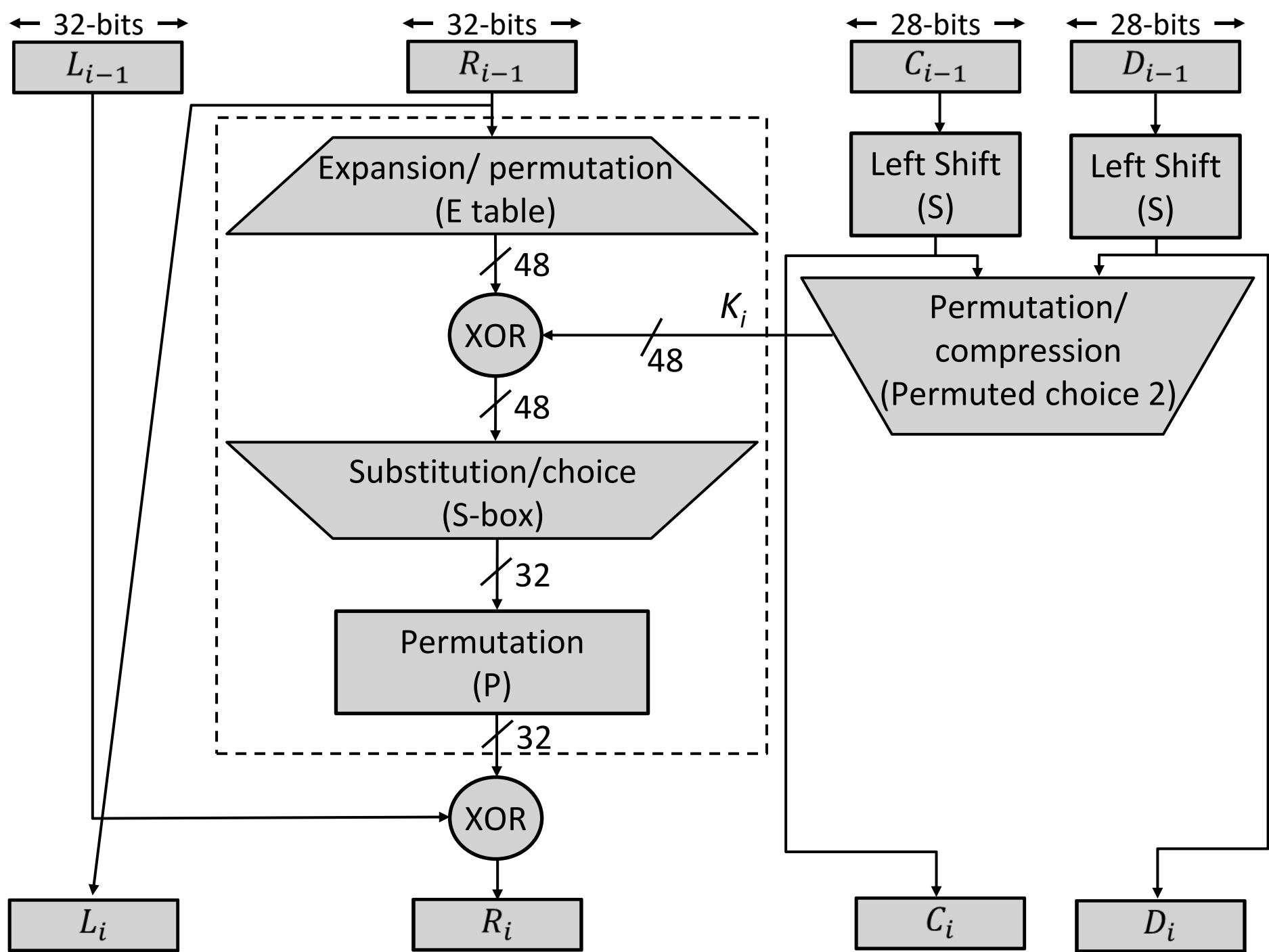
3. S-box Substitution

- Accepts 48-bits from XOR operation and produce 32-bits using 8 substitution boxes (each S-boxes has a 6-bit i/p and 4-bit o/p).

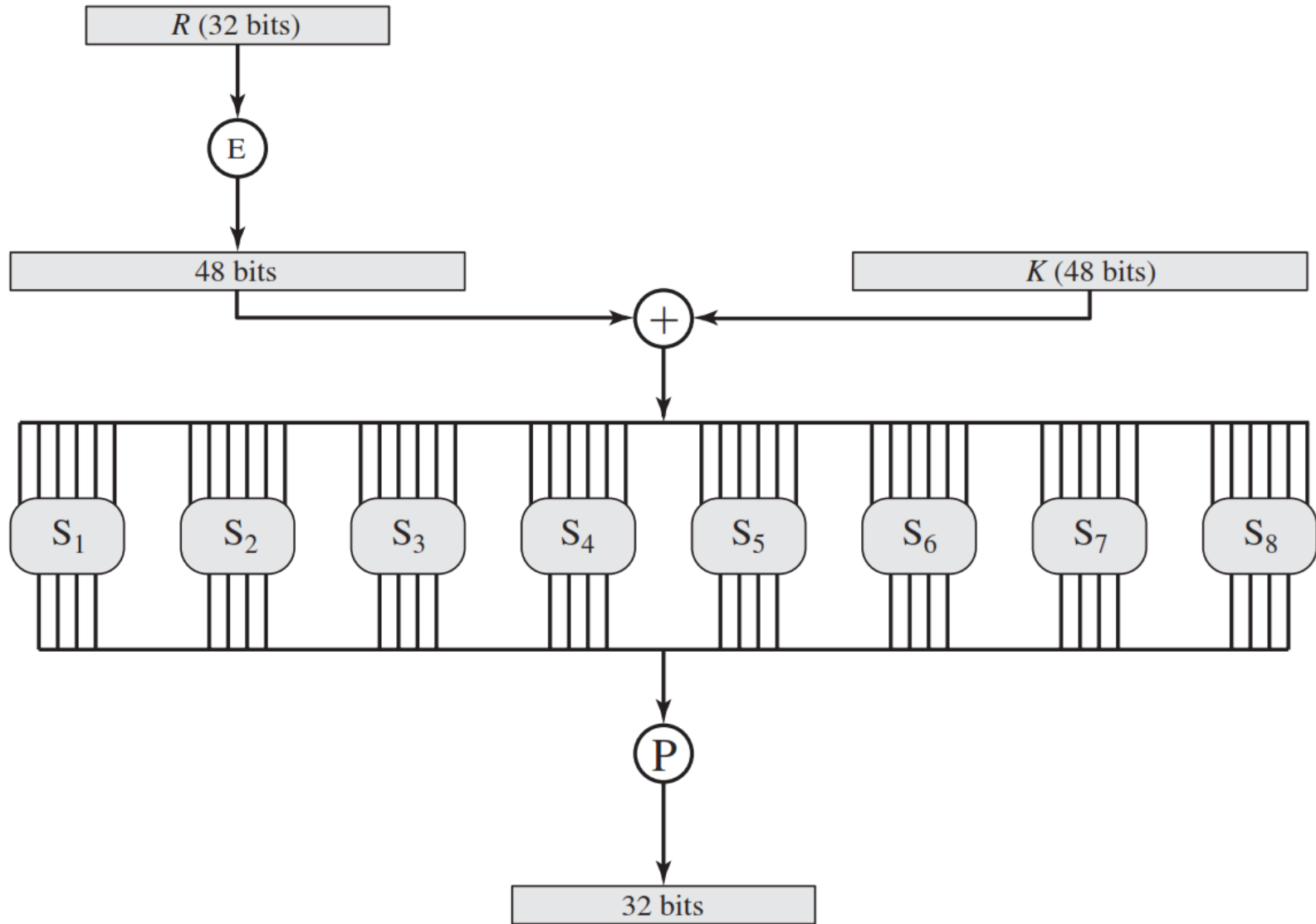
4. P-Box Permutation

5. XOR and Swap





Role of S-box

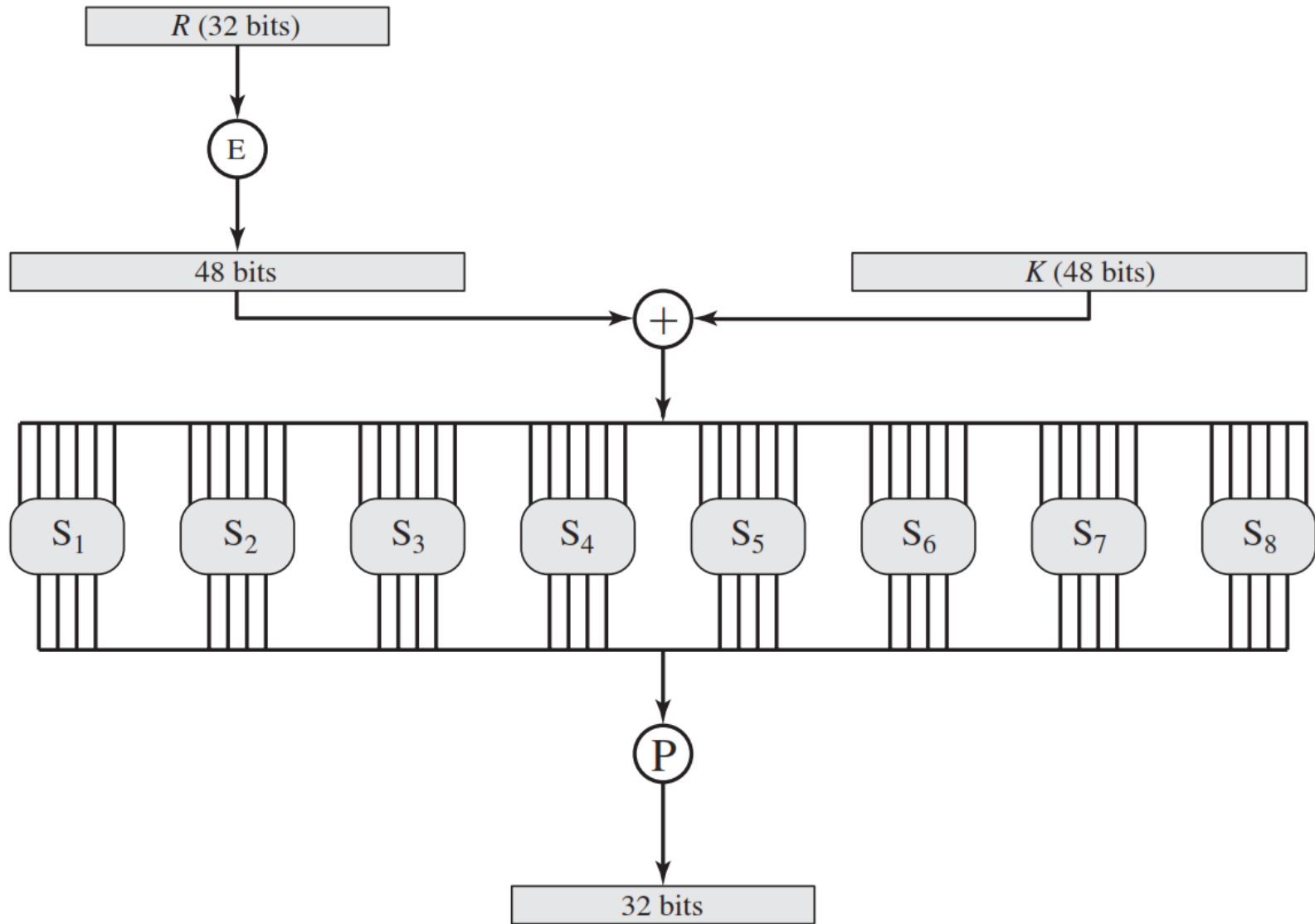


1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16
17	18	19	20
21	22	23	24
25	26	27	28
29	30	31	32

Expansion Permutation

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Role of S-box



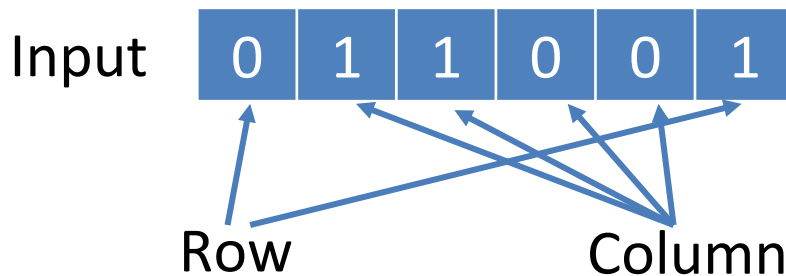
Role of S-box (Cont...)

- The outer two bits of each group select one row of an S-box.
- Inner four bits selects one column of an S-box.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	10	03	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

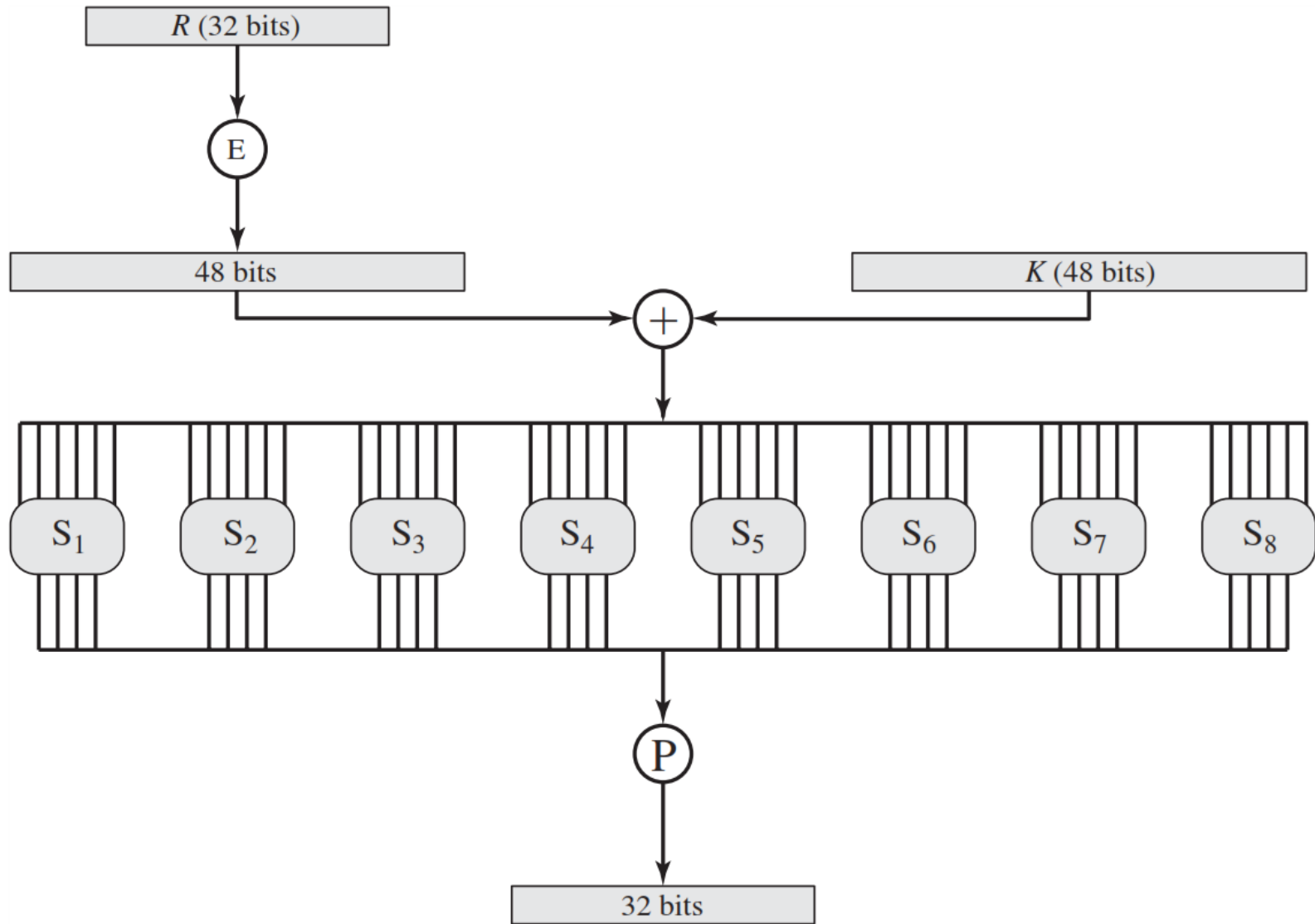
S-box 1

- Example:



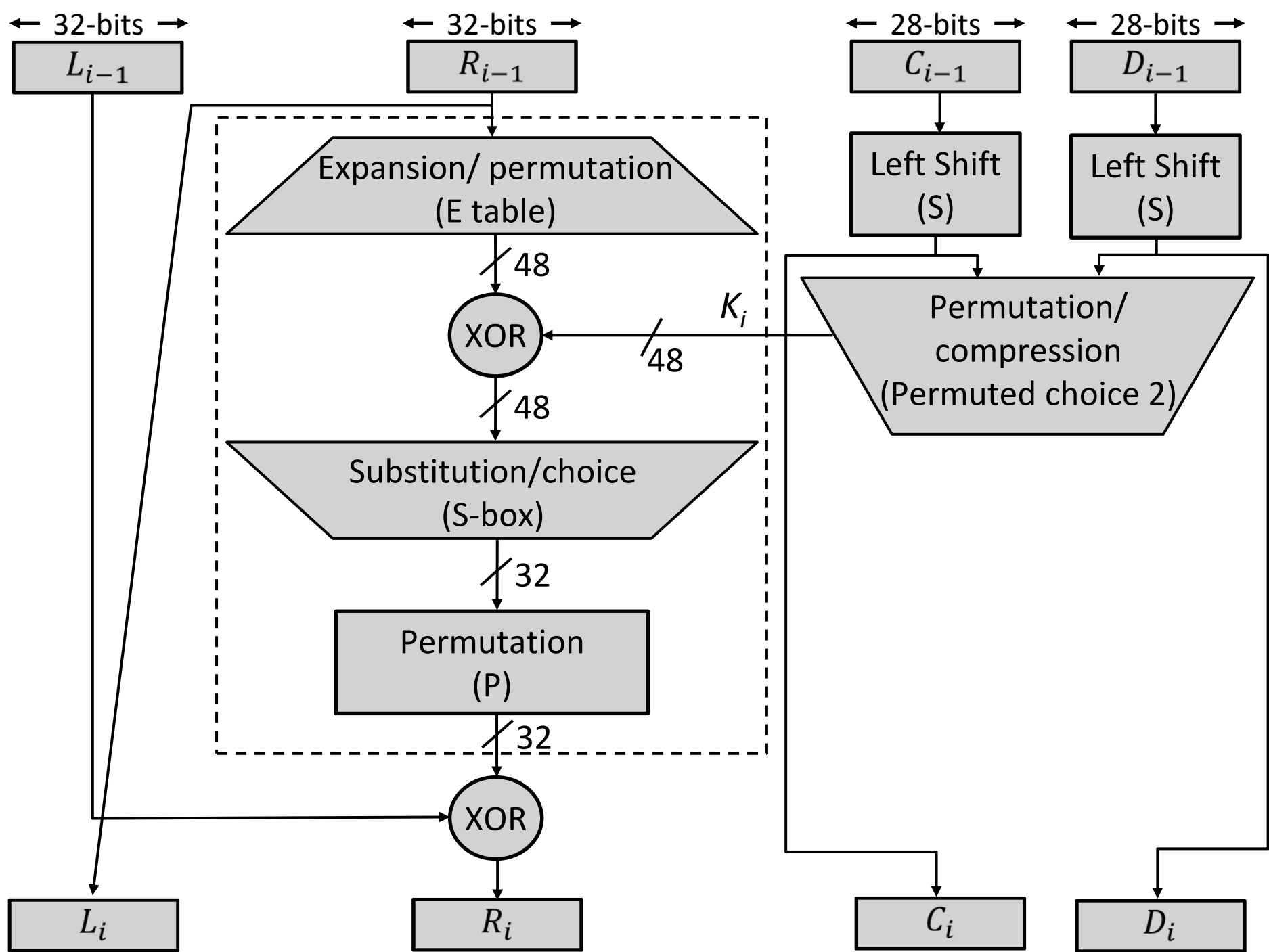
Output **1 0 0 1**

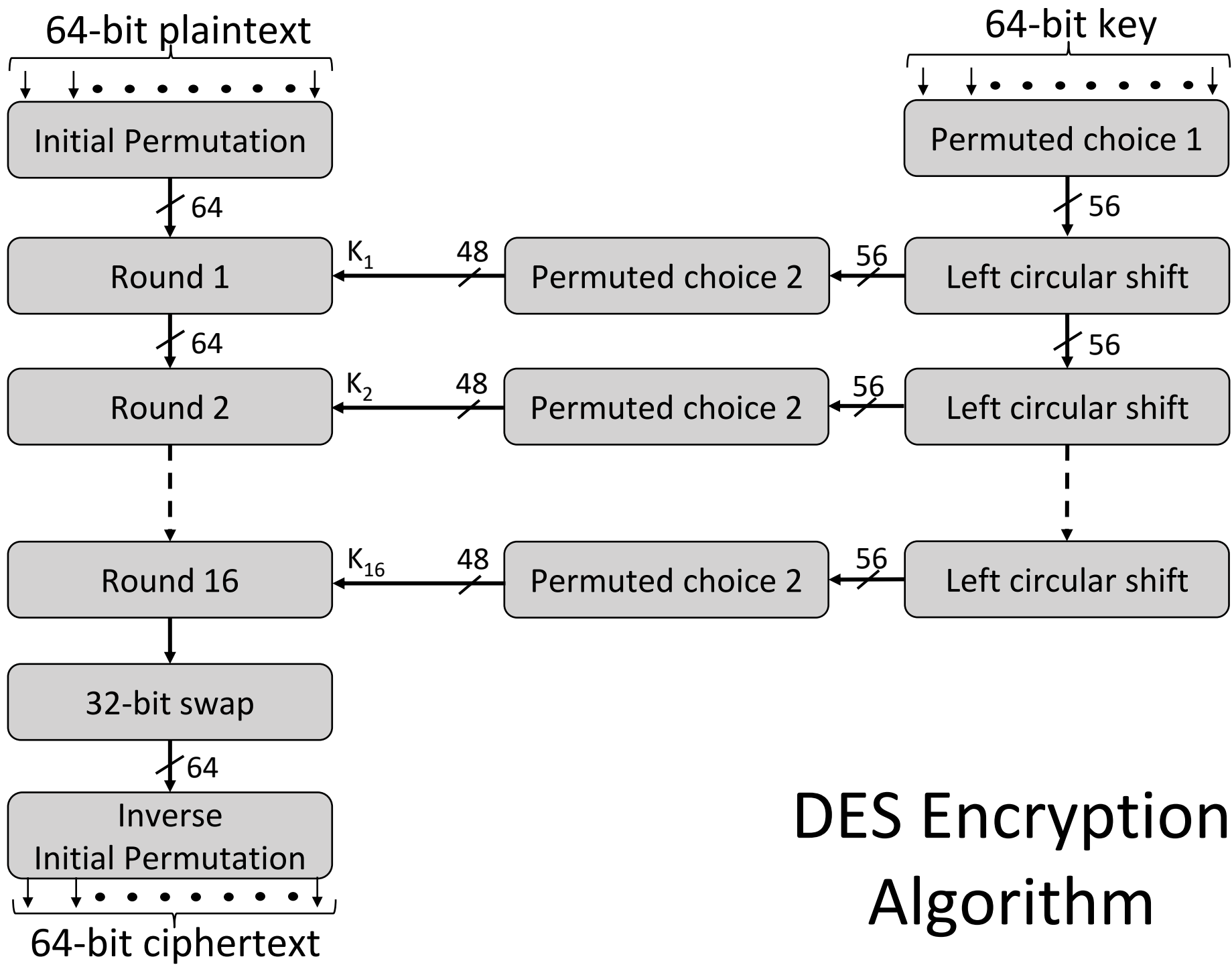
Role of S-box

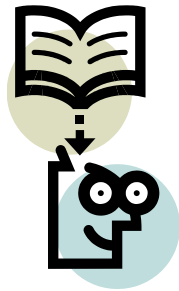


Permutation Function

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25





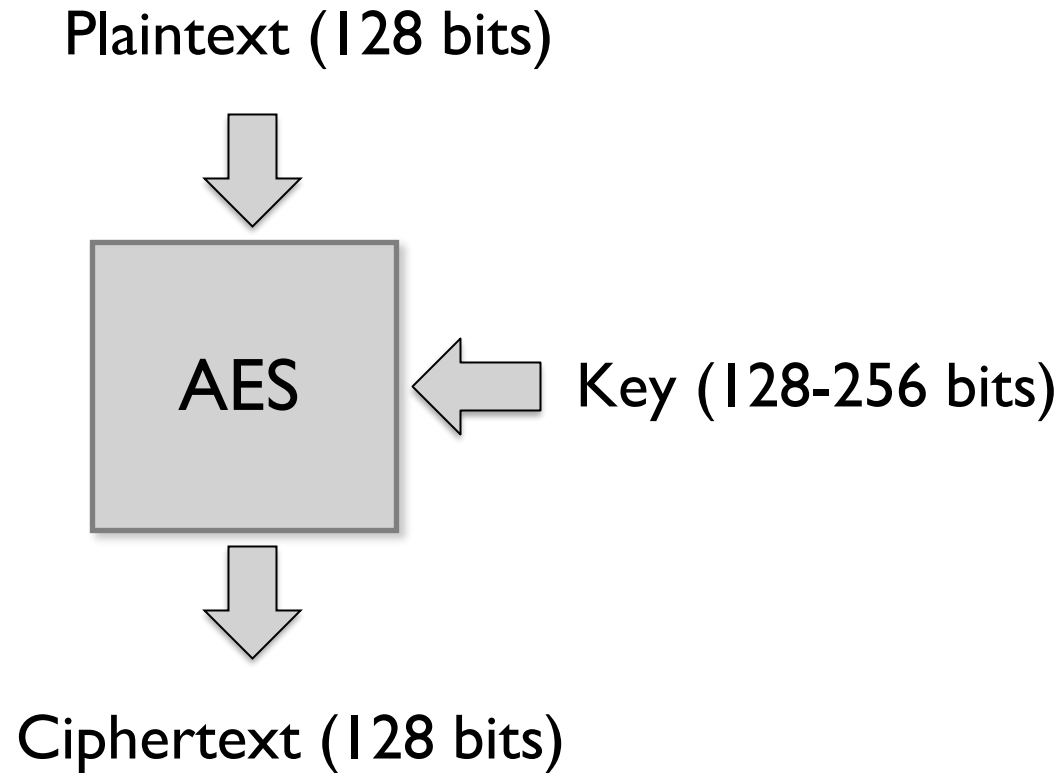


Lecture 22



AES (Advance Encryption Standard)

AES (Advanced Encryption Standard)



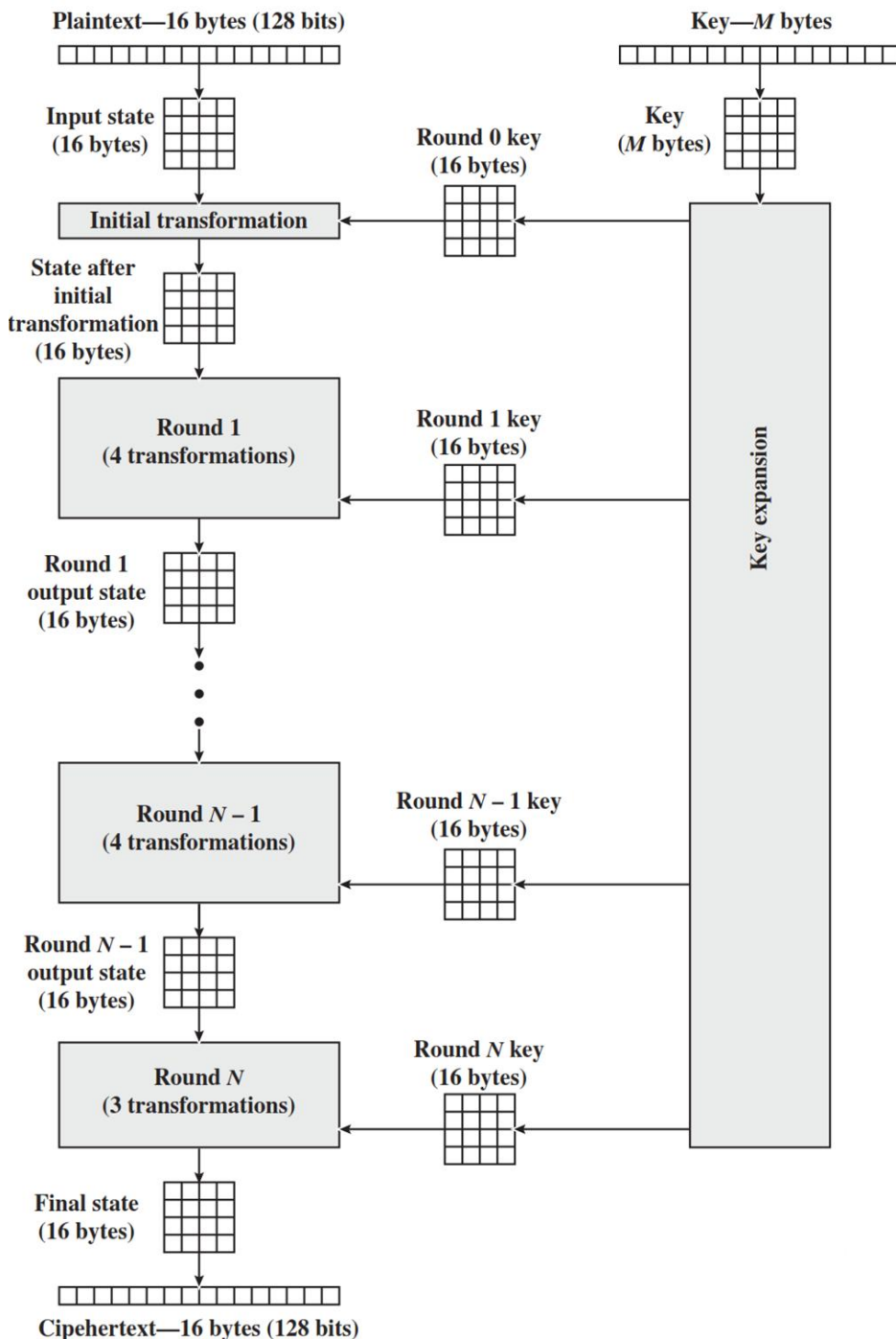
AES Structure

Initialization

1. Expand 16-byte key to get the actual **key block** to be used.
2. Initialize 16-byte plaintext block called as **state**.
3. XOR the **state** with the **key block**.

For each round

1. Apply S-box
2. Rotate rows of state
3. Mix columns
4. Add Round key: XOR the state with key block.



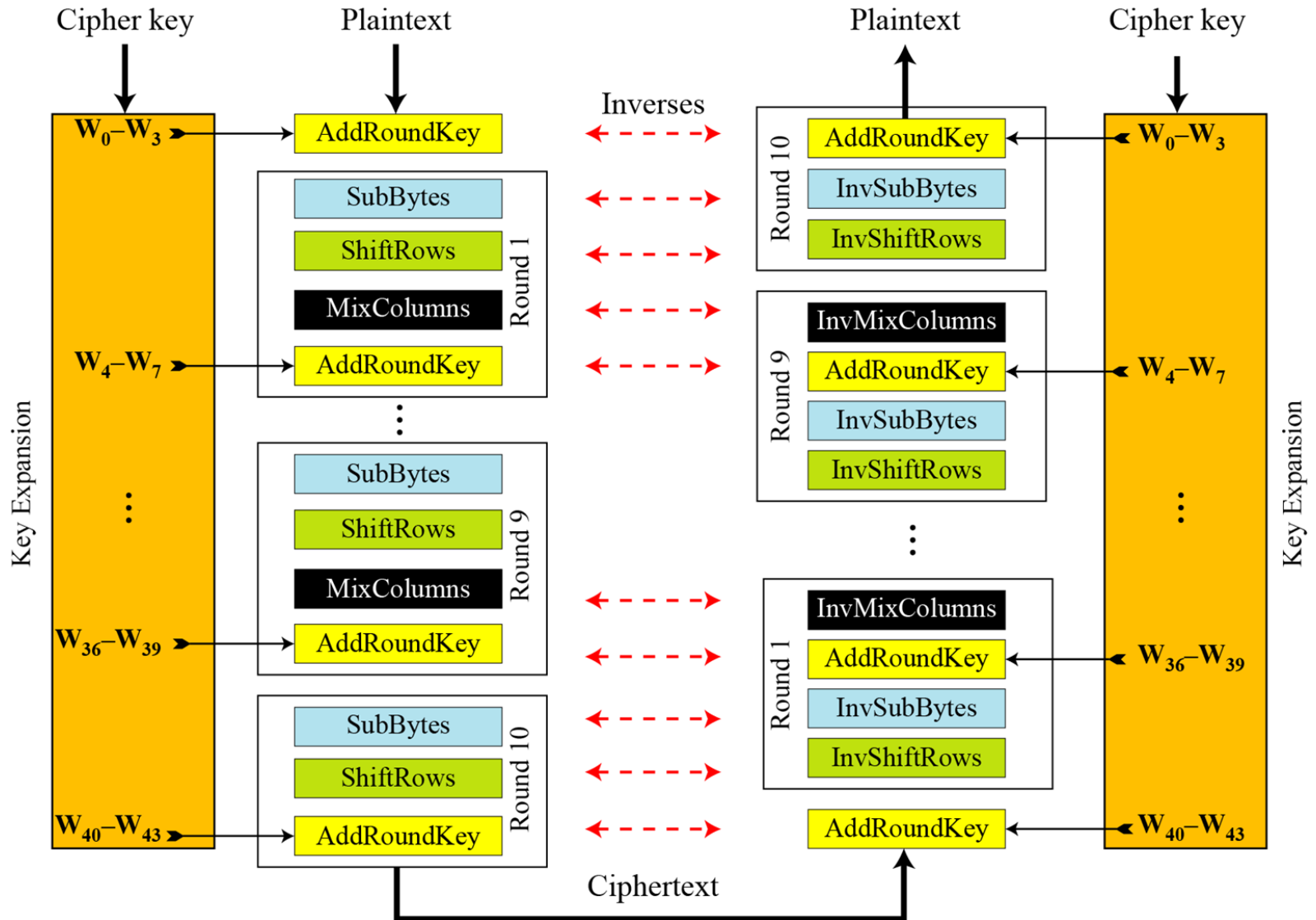
AES (Advanced Encryption Standard)

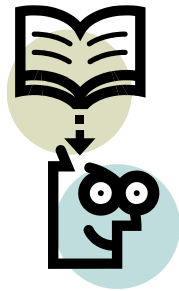
- The Rijndael proposal for AES defined a cipher in which the block length and the key length can be independently specified to be 128, 192, or 256 bits.

Key size	128	192	256
Plaintext Size	128	128	128
Round key	128	128	128
Number of Rounds	10	12	14

- AES designed to have characteristics
 1. Resistance against all known attacks
 2. Speed and code compactness on a wide range of platforms
 3. Design simplicity

AES Overall Structure



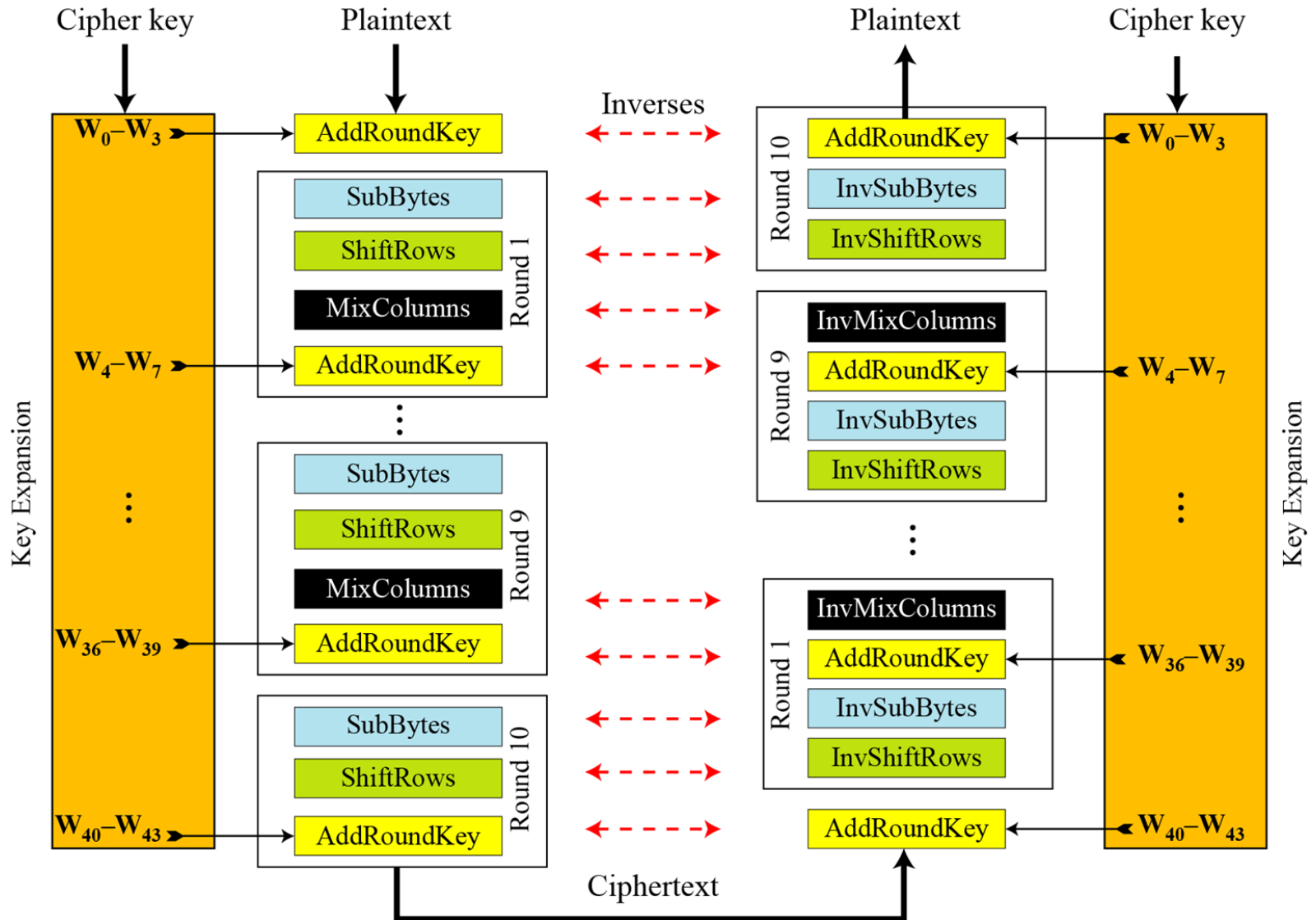


Lecture 23

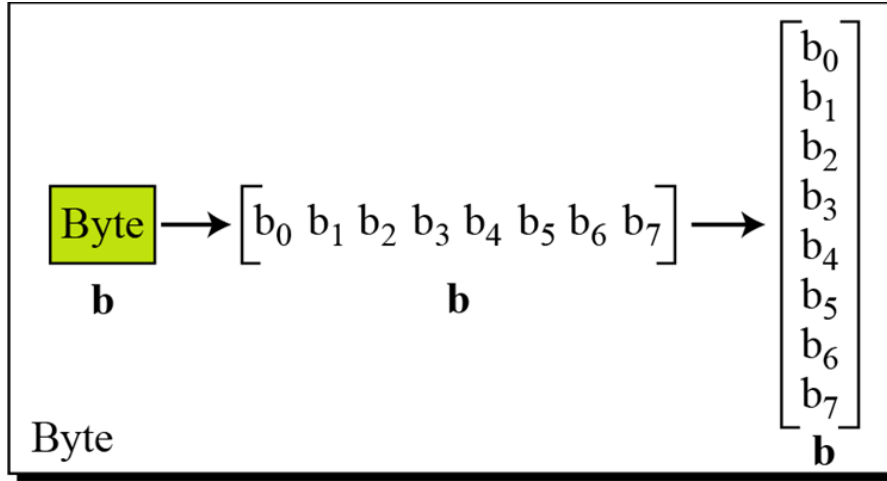


Single Round AES & Key Scheduling

AES Overall Structure



Data Units in AES



Block to State & State to Block

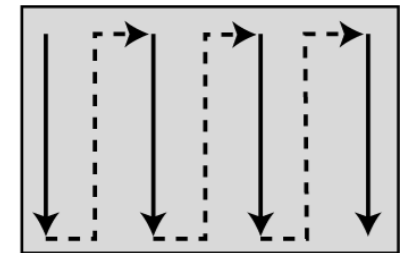


Block

$$s_{i \bmod 4, i/4} \leftarrow \text{block}_i$$

State

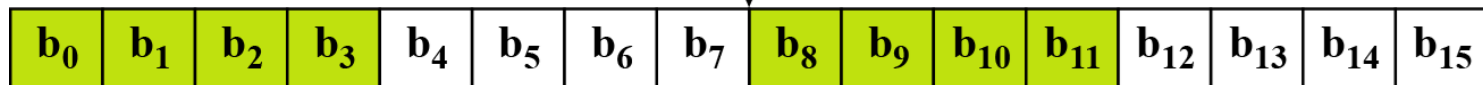
$$\begin{bmatrix} s_{0,0} = b_0 & s_{0,1} = b_4 & s_{0,2} = b_8 & s_{0,3} = b_{12} \\ s_{1,0} = b_1 & s_{1,1} = b_5 & s_{1,2} = b_9 & s_{1,3} = b_{13} \\ s_{2,0} = b_2 & s_{2,1} = b_6 & s_{2,2} = b_{10} & s_{2,3} = b_{14} \\ s_{3,0} = b_3 & s_{3,1} = b_7 & s_{3,2} = b_{11} & s_{3,3} = b_{15} \end{bmatrix}$$



Insertion and
extraction flow

$$\text{block}_{i+4j} \leftarrow s_{i,j}$$

Block



Plain Text to State

Text

A	E	S	U	S	E	S	A	M	A	T	R	I	X	Z	Z
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

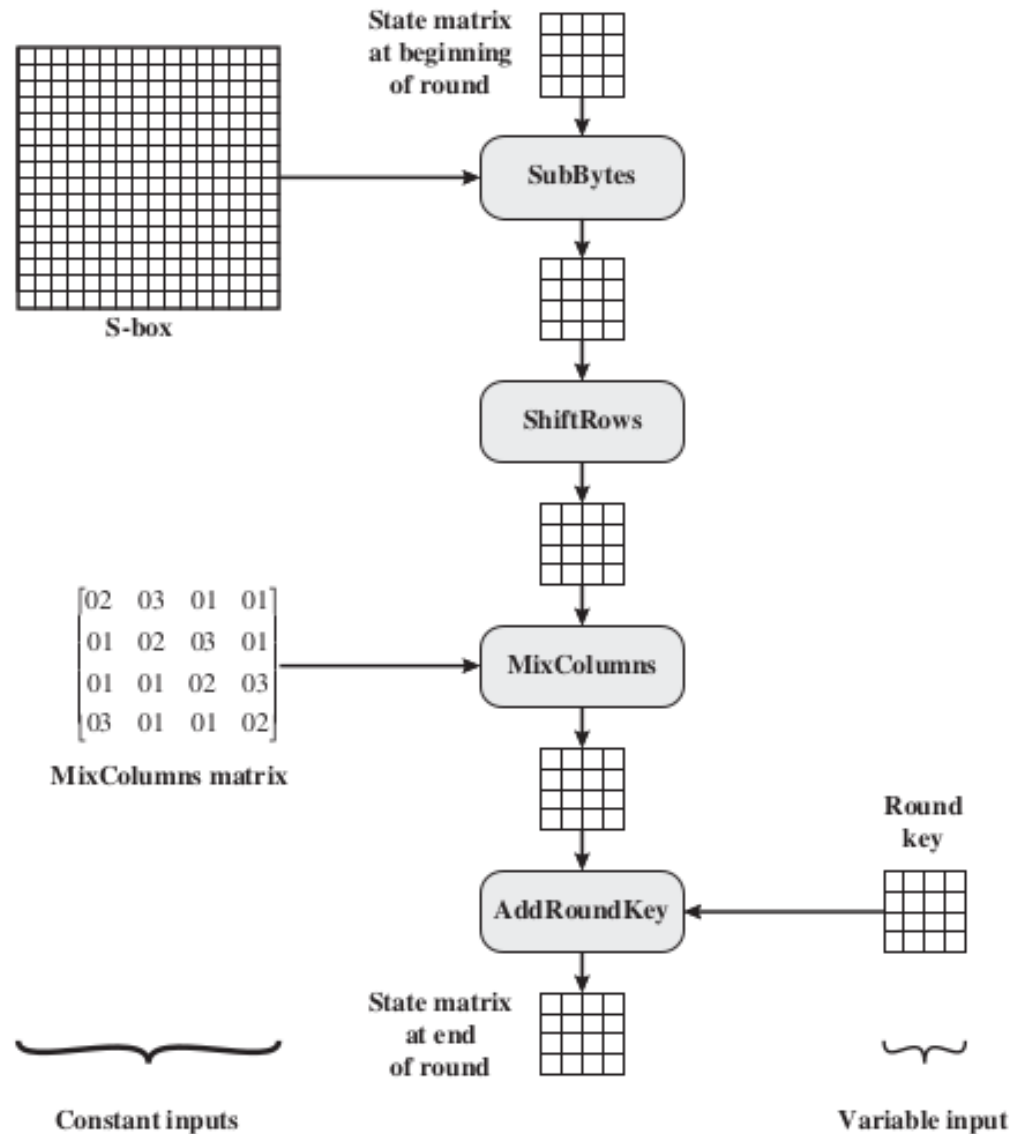
Hexadecimal

00	04	12	14	12	04	12	00	0C	00	13	11	08	23	19	19
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

00	12	0C	08
04	04	00	23
12	12	13	19
14	00	11	19

State

Single AES Round



AES Structure

- The first N-1 rounds consist of four distinct transformation functions.

SubBytes

- The 16 input bytes are substituted using an **S-box**

ShiftRows

- Each of the four rows of the matrix is shifted to the left

MixColumns

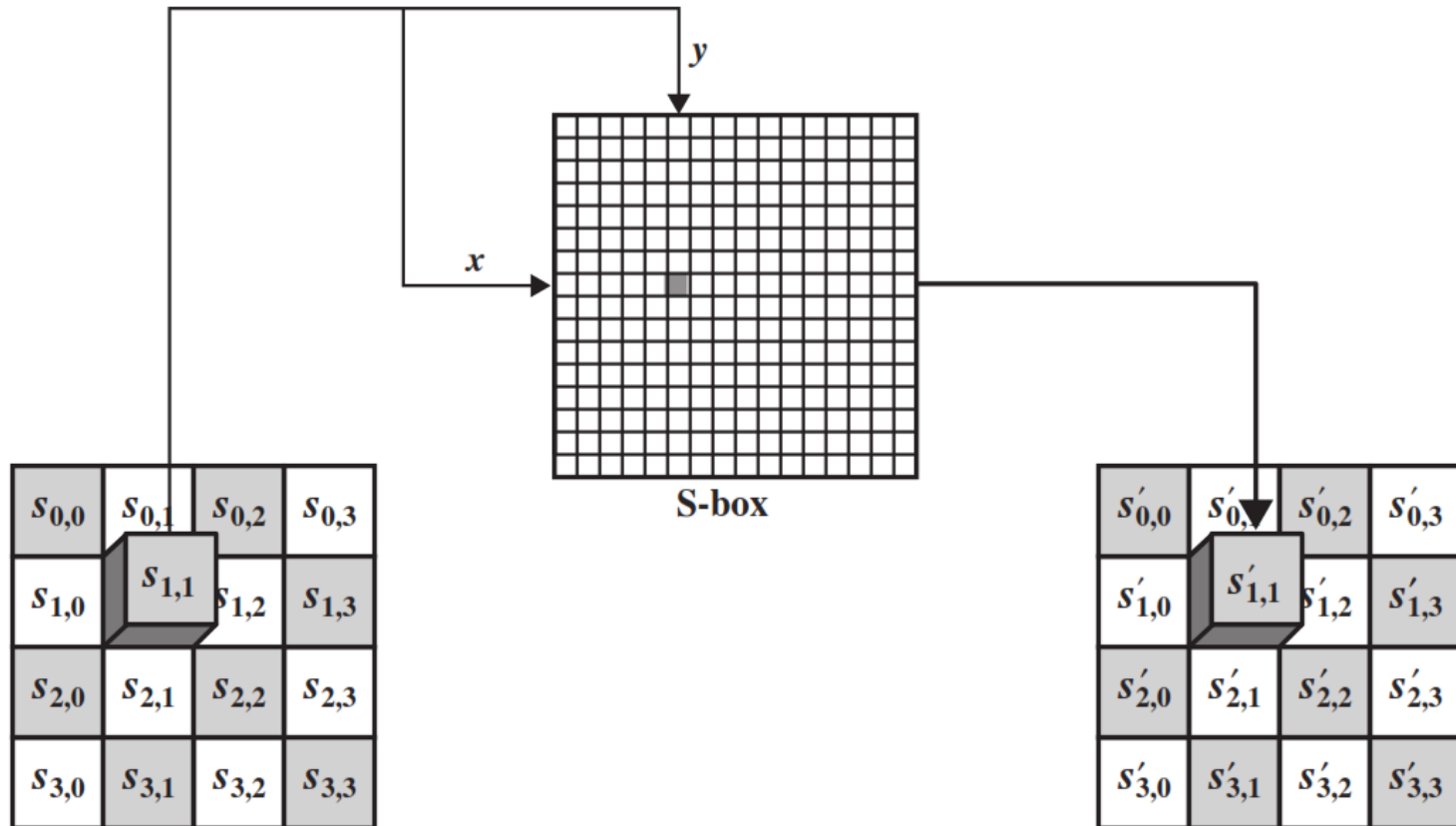
- Each column of four bytes is now transformed using a special mathematical function.

AddRoundKey y

- The 16 bytes of the matrix are now considered as 128 bits and are XORed to the 128 bits of the round key.

SubByte Transformation

- The forward substitute byte transformation, called **SubBytes**, is a simple table lookup



State before
substitution operation

34	35	89	10
A3	D5	6B	8F
89	78	67	56
45	68	6D	13

State after substitution
operation

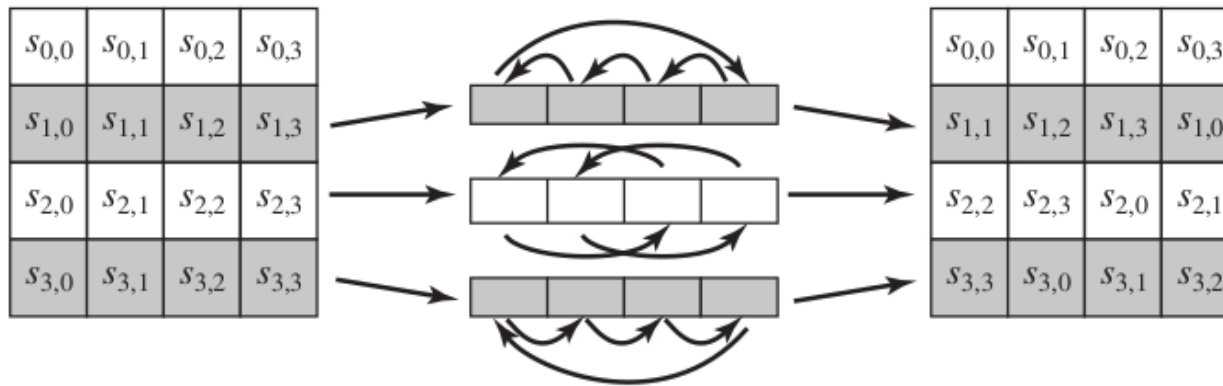
18	96	A7	CA
0A	03	7F	73
A7	BC	85	B1
6E	FB	3C	7D

S-Box

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	E0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3E	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	C6	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

ShiftRows

- The first row of **State is not altered**.
- For the second row, a 1-byte circular left shift is performed.
- For the third row, a 2-byte circular left shift is performed.
- For the fourth row, a 3-byte circular left shift is performed.



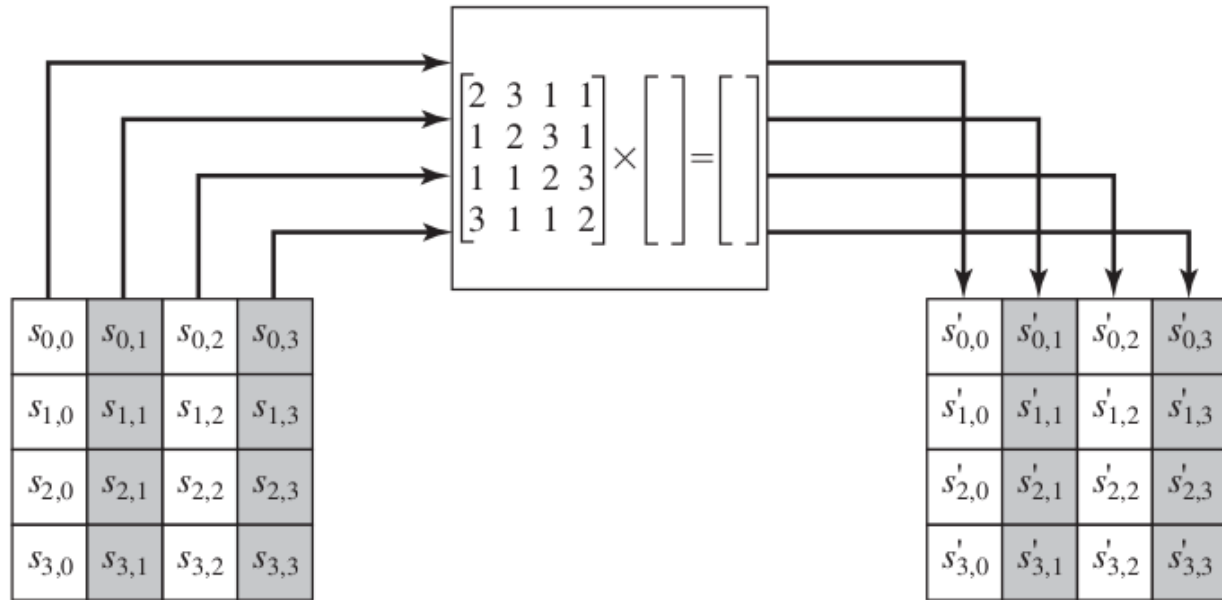
87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

→

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

MixColumns

- Each byte of a column is mapped into a new value that is a function of all four bytes in that column.



87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

→

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

AddRoundKey

- In the forward add round key transformation, the 128 bits of State are bitwise XORed with the 128 bits of the round key.

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

State

 $\oplus$

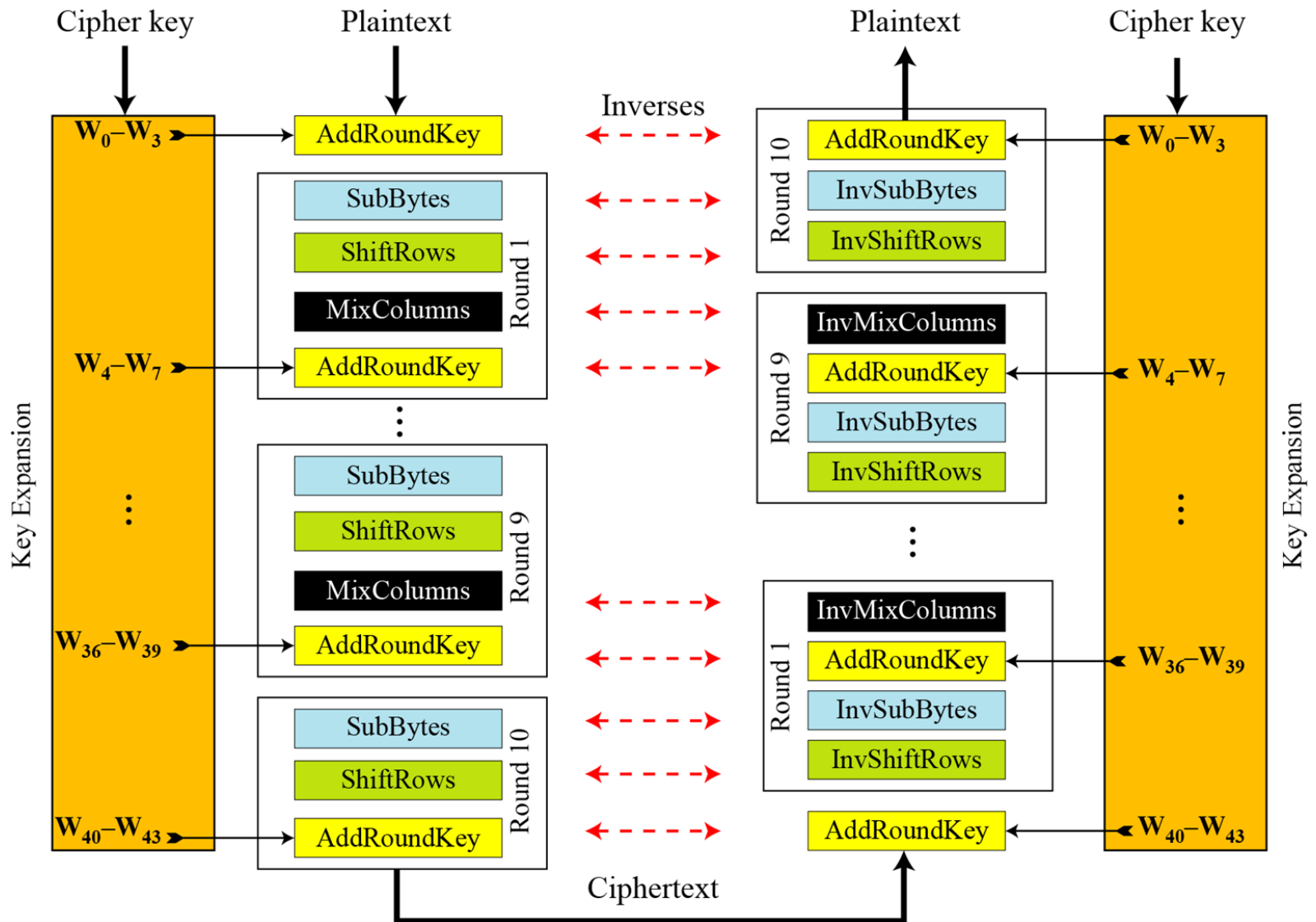
AC	19	28	57
77	FA	D1	5C
66	DC	29	00
F3	21	41	6A

Round Key

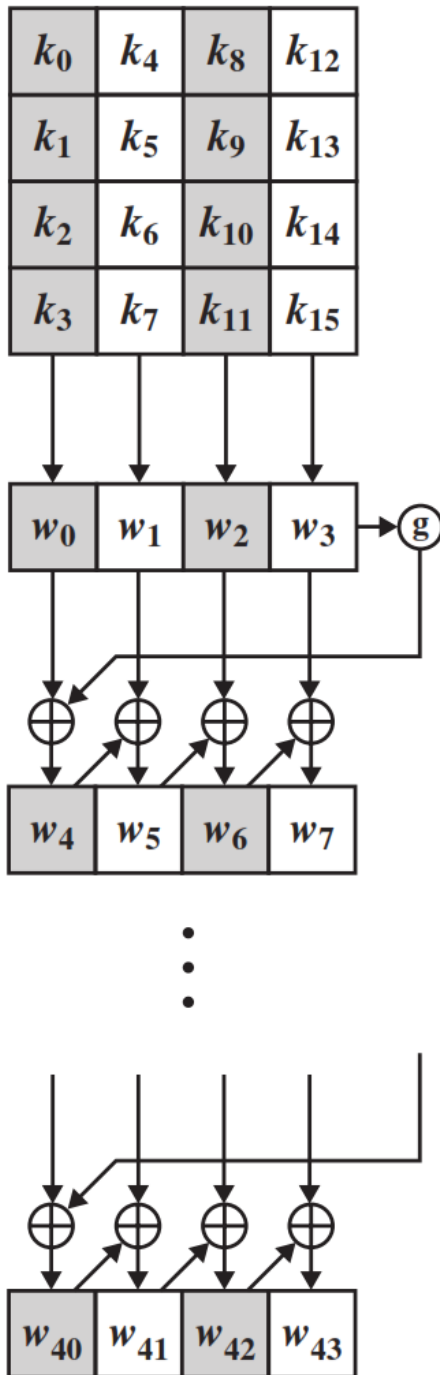
 $=$

EB	59	8B	1B
40	2E	A1	C3
F2	38	13	42
1E	84	E7	D6

AES Overall Structure

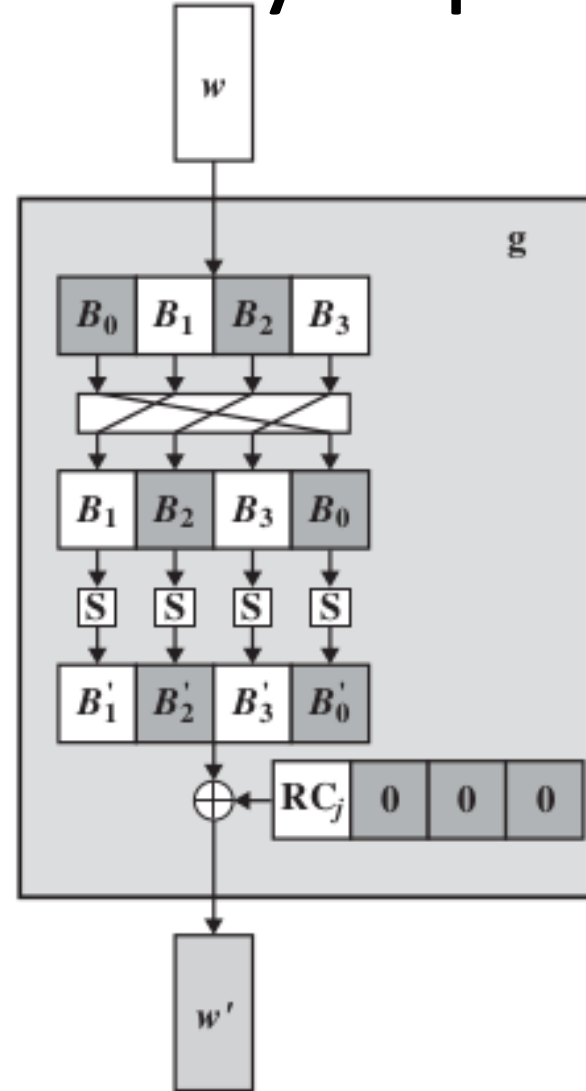
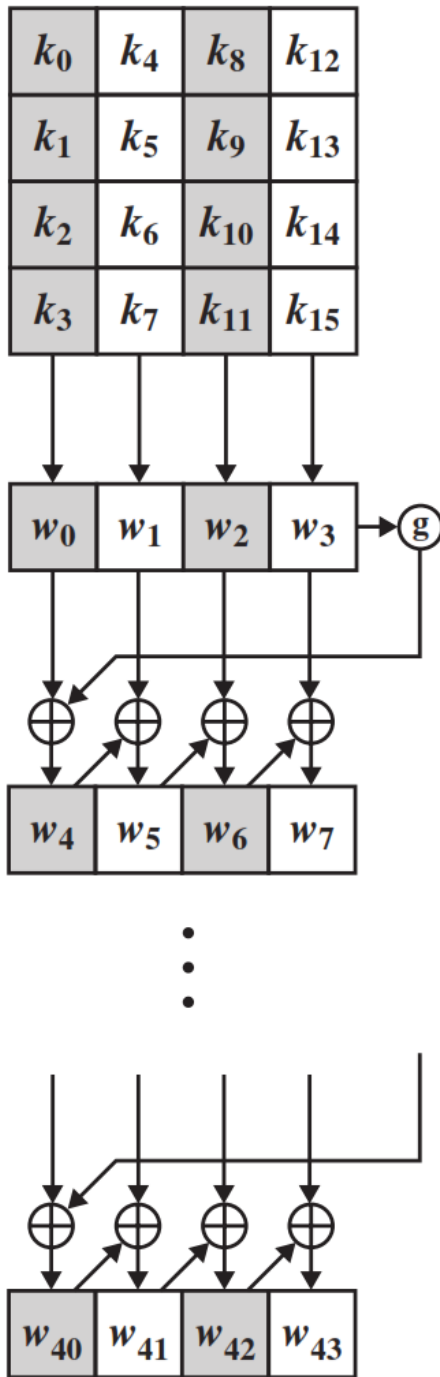


AES Key Expansion

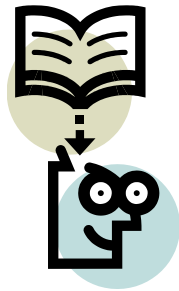


- The AES key expansion algorithm takes as input a four-word (16-byte) key and produces a linear array of **44 words** (176 bytes).
- Each added word **$w[i]$** depends on the immediately preceding word, $w[i - 1]$.
- In three out of four cases, a simple XOR is used.

AES Key Expansion



j	1	2	3	4	5	6	7	8	9	10
RC[j]	01	02	04	08	10	20	40	80	1B	36

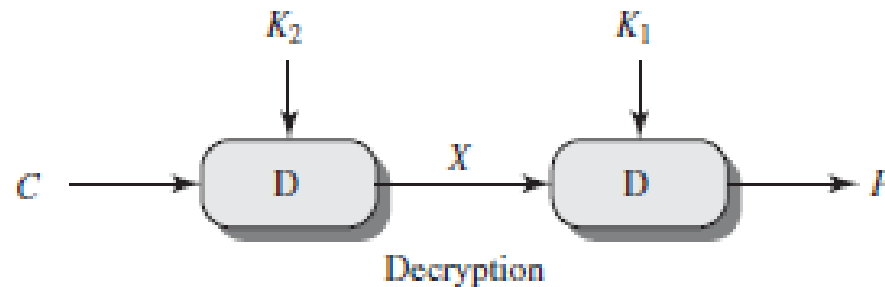
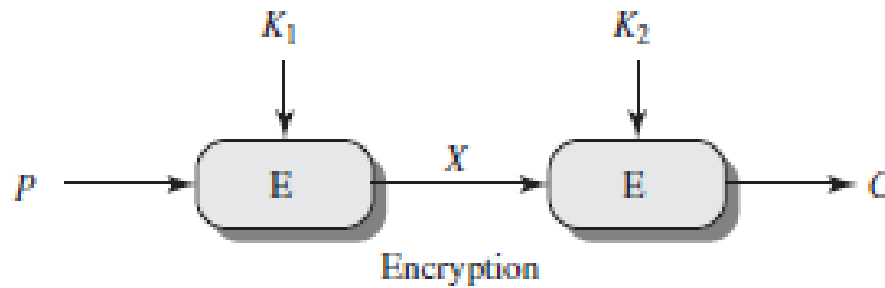


Lecture 24



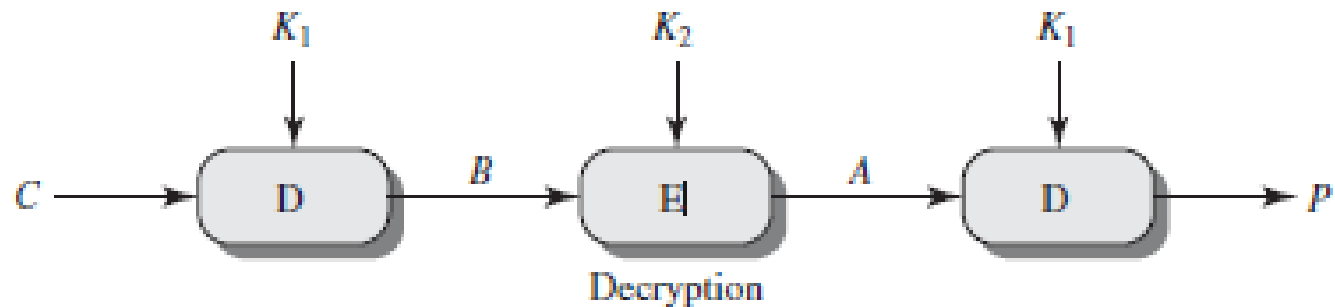
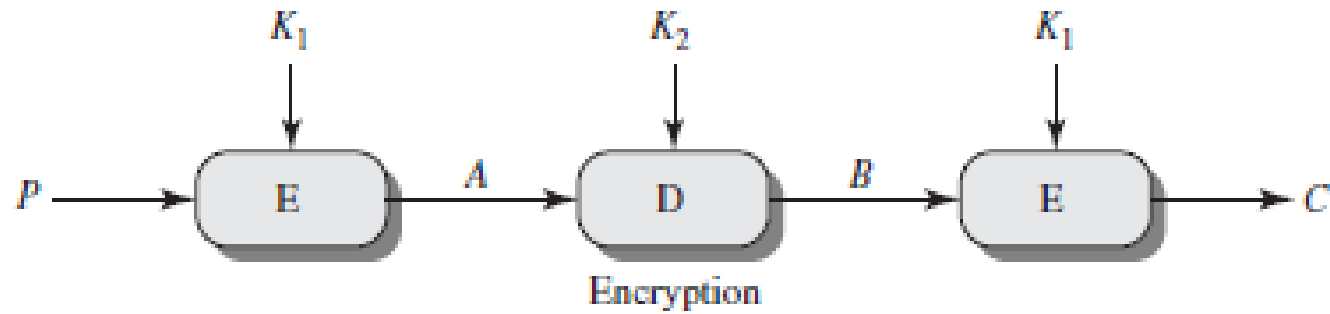
Multiple Encryptions and Triple DES

Double encryption



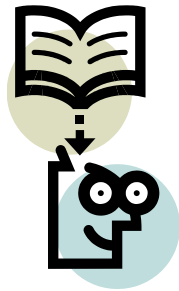
meet-in-the-middle attack

Triple encryption



Block Cipher Modes of Operations

- To apply a block cipher in a **variety of applications**, five "modes of operation" have been defined.
- The **five modes** are intended to cover a wide variety of applications of encryption for which a block cipher could be used.
- These modes are intended for use with any symmetric block cipher, including triple DES and AES.
 1. Electronic Code Book (ECB)
 2. Cipher Block Chaining (CBC)
 3. Cipher Feedback (CFB)
 4. Output Feedback (OFB)
 5. Counter (CTR)



Lecture 25



Block Cipher Modes

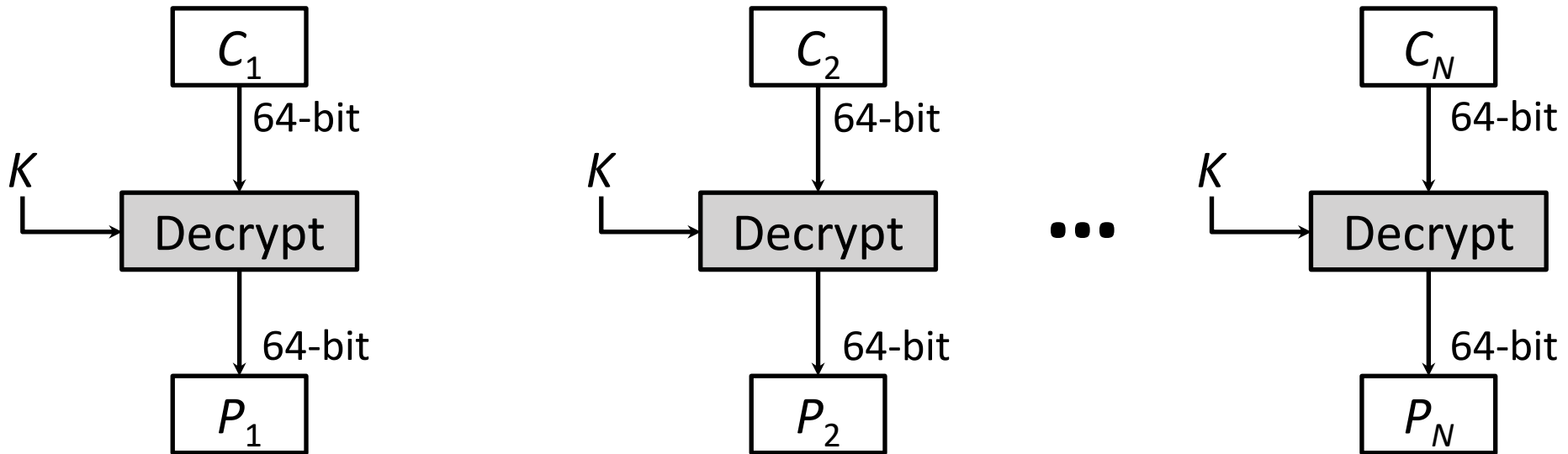
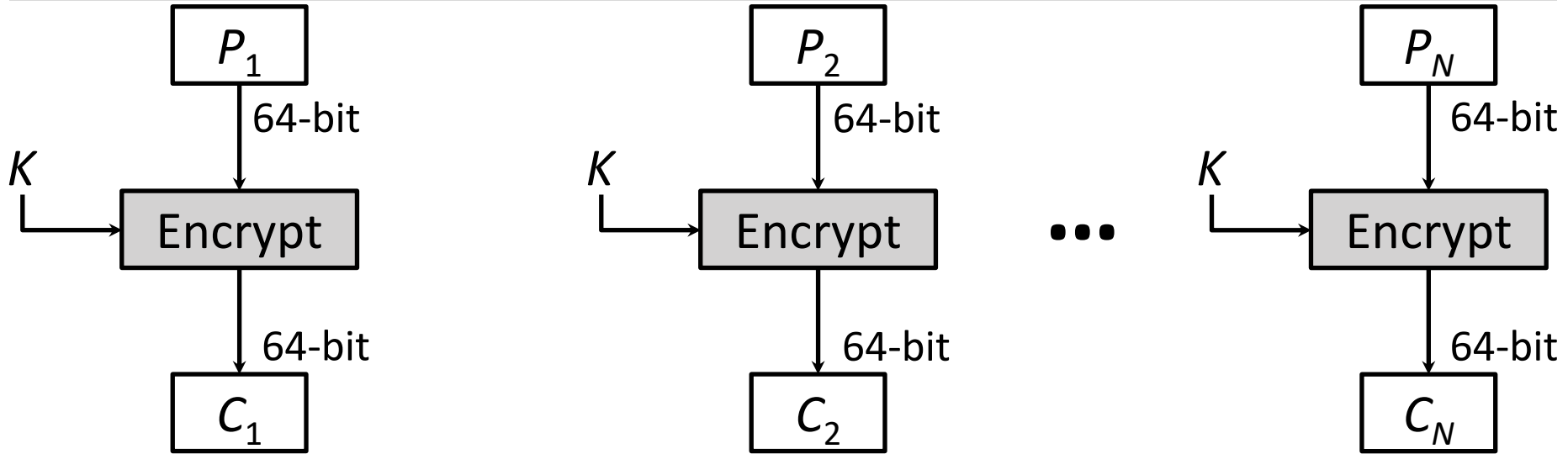
Block Cipher Modes of Operations

- To apply a block cipher in a **variety of applications**, five "modes of operation" have been defined.
- The **five modes** are intended to cover a wide variety of applications of encryption for which a block cipher could be used.
- These modes are intended for use with any symmetric block cipher, including triple DES and AES.
 1. Electronic Code Book (ECB)
 2. Cipher Block Chaining (CBC)
 3. Cipher Feedback (CFB)
 4. Output Feedback (OFB)
 5. Counter (CTR)

1. Electronic Code Book (ECB)

- In **ECB** Mode Plaintext handled one block at a time and each block of plaintext is encrypted using the same key.
- The term **codebook** is used because, for a given key, there is a unique ciphertext for every b-bit block of plaintext.

1. ECB Encryption & Decryption



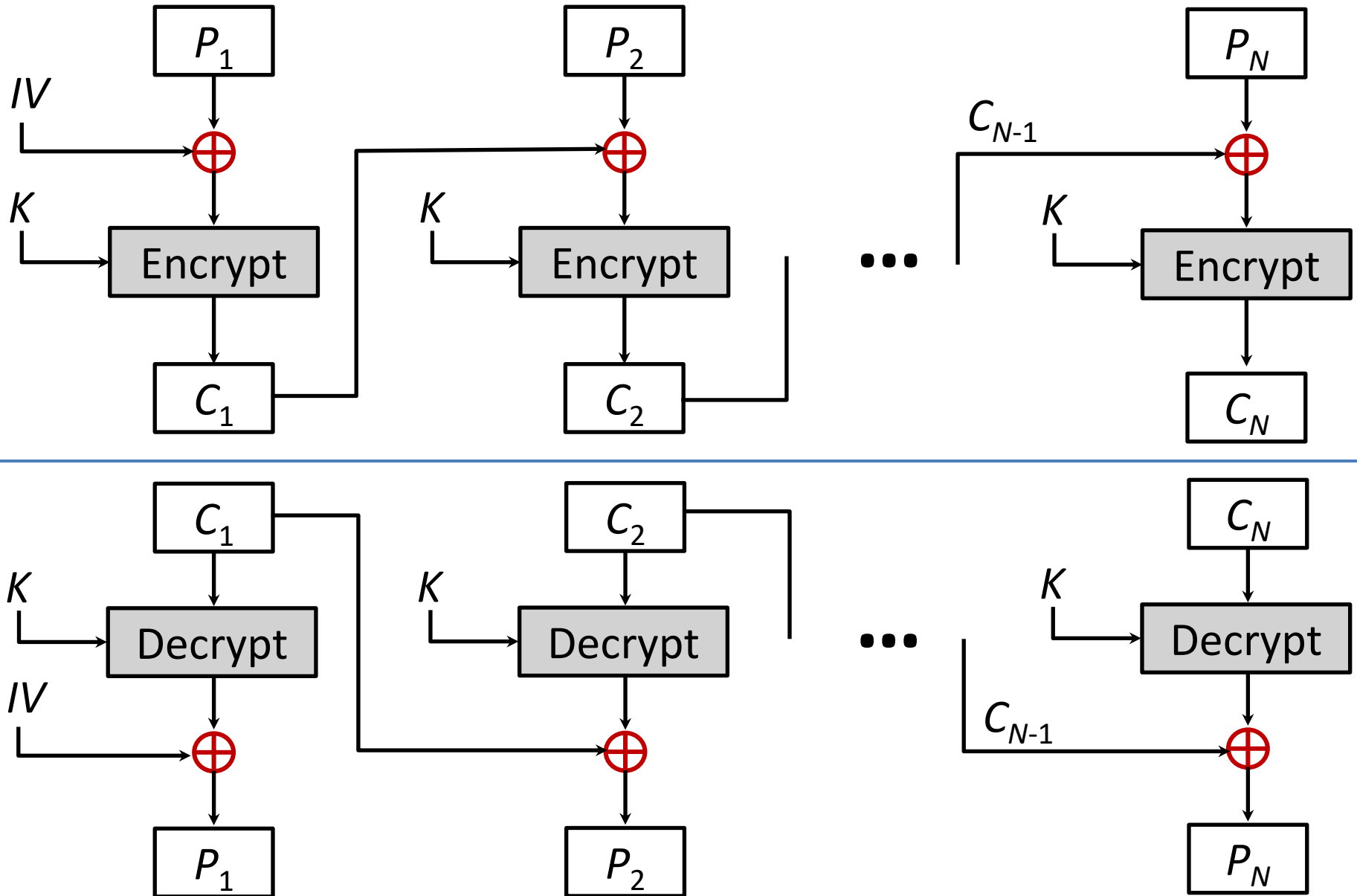
Electronic Code Book - Cont...

- **Strength:** it's simple.
- **Weakness:**
 - Repetitive information contained in the plaintext may show in the ciphertext, if aligned with blocks.
 - If the message has repetitive elements with a period of repetition a multiple of b bits, then these elements can be identified by the analyst.
- **Typical application:**
 - Secure transmission of short pieces of information (e.g. a temporary encryption key)

2. Cipher Block Chaining (CBC)

- **CBC** is a technique in which the same plaintext block, if repeated, produces different ciphertext blocks.
- In this scheme, the input to the encryption algorithm is the **XOR** of the current plaintext block and the preceding ciphertext block; the same key is used for each block.
- To produce the first block of ciphertext, an **initialization vector (IV)** is XORed with the first block of plaintext.
- On decryption, the **IV** is XORed with the output of the decryption algorithm to recover the first block of plaintext.

2. CBC - Encryption & Decryption



2. Cipher Block Chaining (CBC) – Cont...

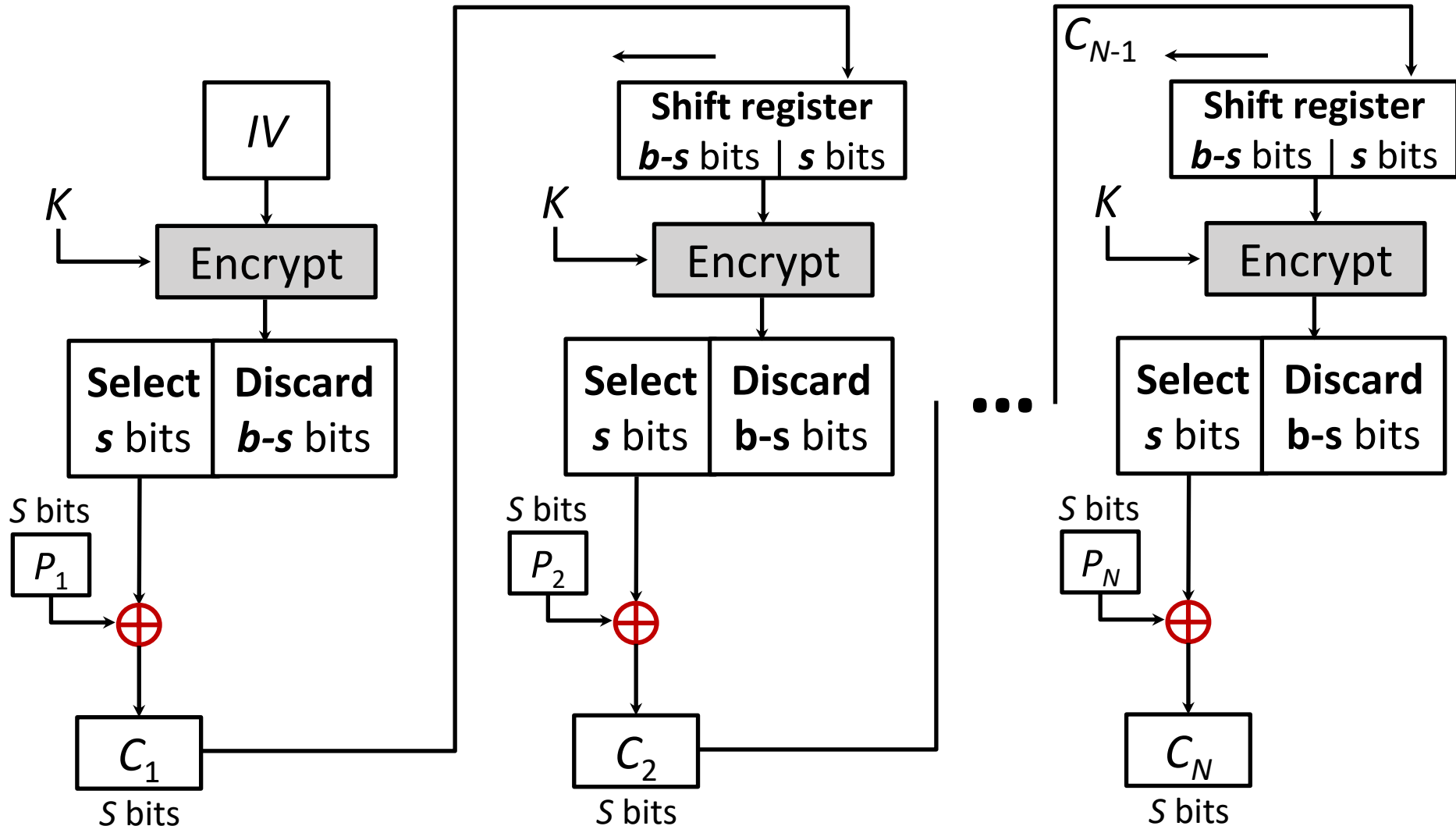
- **Strength:** because of the chaining mechanism of CBC, it is an appropriate mode for encrypting messages of length greater than b bits
- **Typical application:**
 - General-purpose block oriented transmission
 - Authentication

CBC	$C_1 = E(K, [P_1 \oplus IV])$	$P_1 = D(K, C_1) \oplus IV$
	$C_j = E(K, [P_j \oplus C_{j-1}]) \quad j = 2, \dots, N$	$P_j = D(K, C_j) \oplus C_{j-1} \quad j = 2, \dots, N$

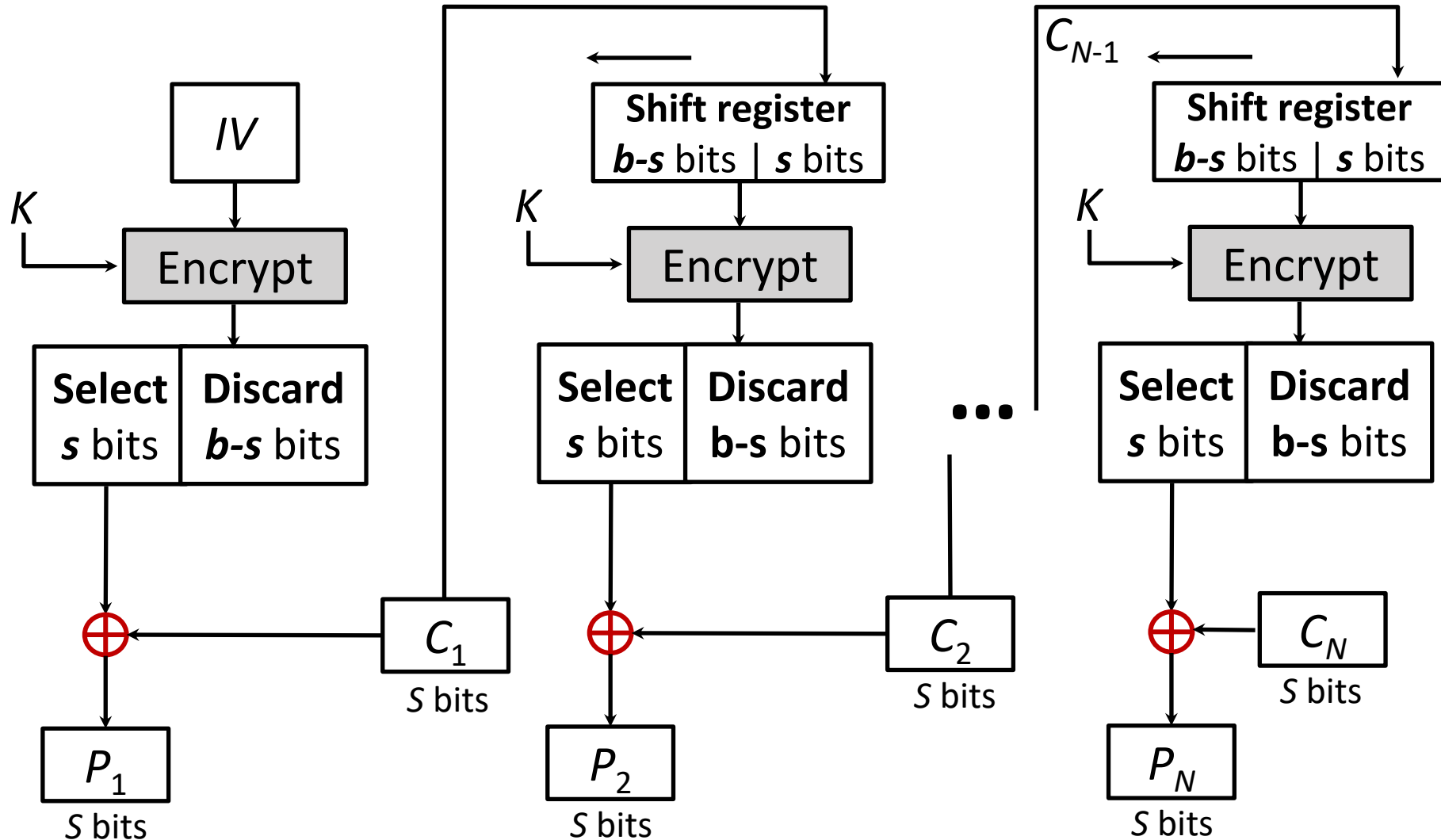
3. Cipher Feedback Mode (CFB)

- For AES, DES, or any block cipher, encryption is performed on a block of b bits. In DES, $b = 64$ and in AES, $b = 128$.
- However, it is possible to convert a block cipher into a stream cipher, using cipher feedback (CFB) mode, output feedback (OFB) mode, and counter (CTR) mode.
- A stream cipher eliminates the need to pad a message to be an integral number of blocks.

3. CFB Encryption



3. CFB Decryption



CFB Mode

- The input to the encryption function is a **b-bit shift register** that is initially set to some initialization vector (IV).
- The leftmost (most significant) **s bits** of the output of the encryption function are XORed with the first segment of plaintext **P1** to produce the first unit of ciphertext **C1** , which is then transmitted.
- In addition, the contents of the shift register are shifted left by **s bits**, and C1 is placed in the rightmost (least significant) s bits of the shift register.
- For decryption, the same scheme is used, except that the received ciphertext unit is XORed with the output of the encryption function to produce the plaintext unit.

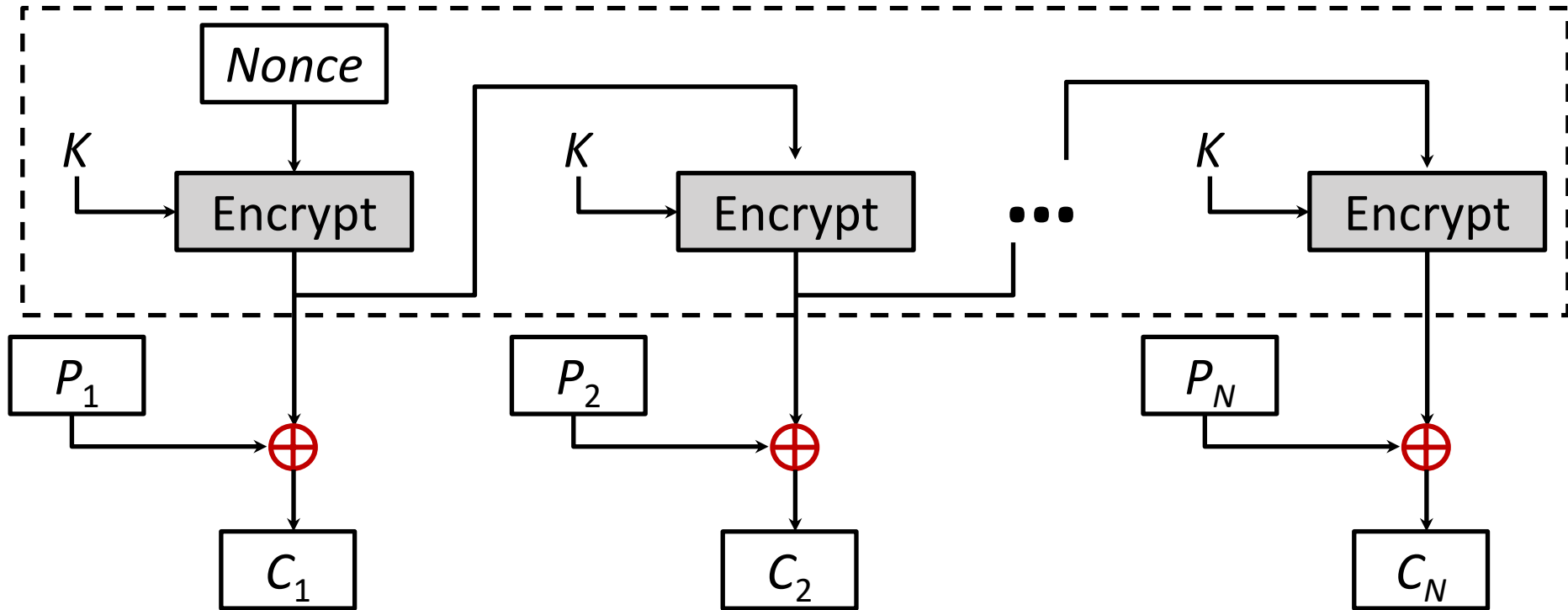
CFB Mode – Cont...

CFB	$I_1 = IV$	$I_1 = IV$
	$I_j = \text{LSB}_{b-s}(I_{j-1}) \parallel C_{j-1} \quad j = 2, \dots, N$	$I_j = \text{LSB}_{b-s}(I_{j-1}) \parallel C_{j-1} \quad j = 2, \dots, N$
	$O_j = E(K, I_j) \quad j = 1, \dots, N$	$O_j = E(K, I_j) \quad j = 1, \dots, N$
	$C_j = P_j \oplus \text{MSB}_s(O_j) \quad j = 1, \dots, N$	$P_j = C_j \oplus \text{MSB}_s(O_j) \quad j = 1, \dots, N$

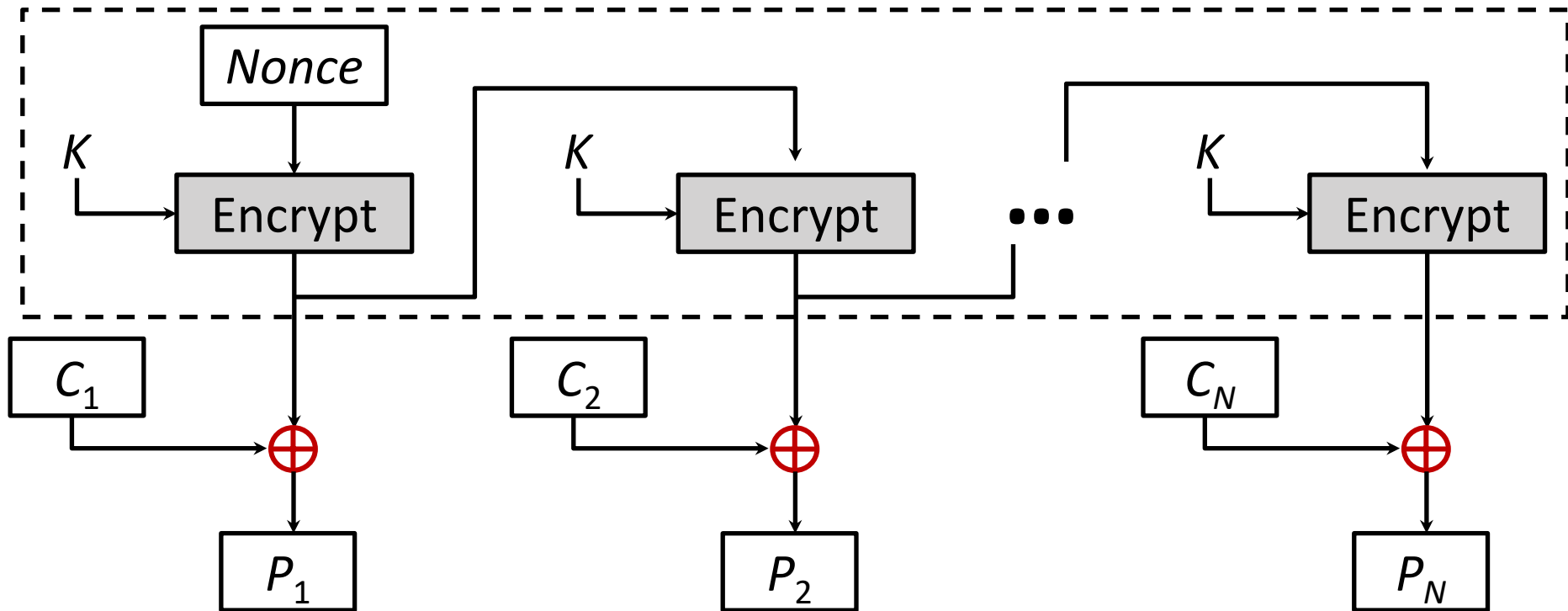
4. Output Feedback Mode (OFB)

- The output feedback (**OFB**) mode is similar in structure to that of **CFB**.
- For **OFB**, the output of the encryption function is fed back to become the input for encrypting the next block of plaintext.
- In **CFB**, the output of the XOR unit is fed back to become input for encrypting the next block.
- The other difference is that the **OFB** mode operates on full blocks of plaintext and ciphertext, whereas **CFB** operates on an **s-bit** subset.
- **Nonce:** A time-varying value that has at most a negligible chance of repeating, for example, a random value that is generated anew for each use, a timestamp, a sequence number, or some combination of these.

4. OFB Encryption



4. OFB Decryption

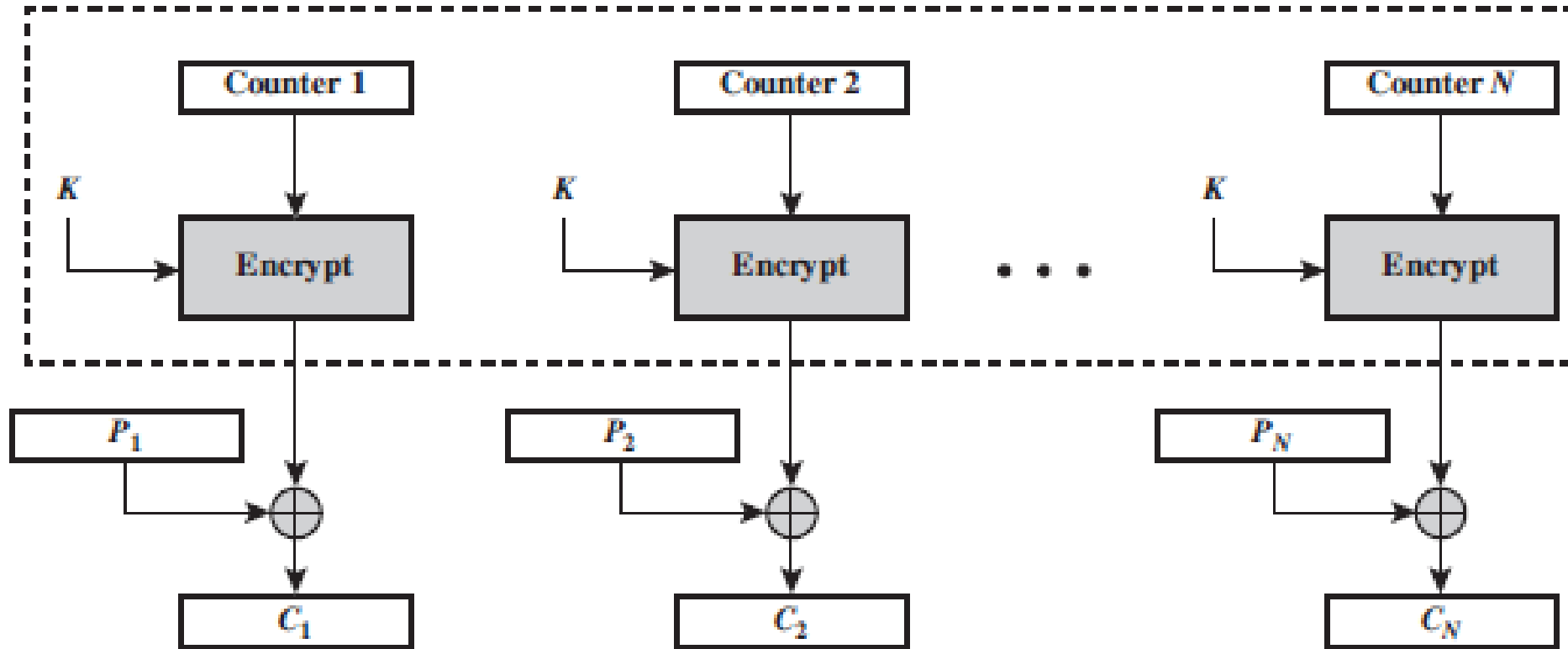


OFB Mode

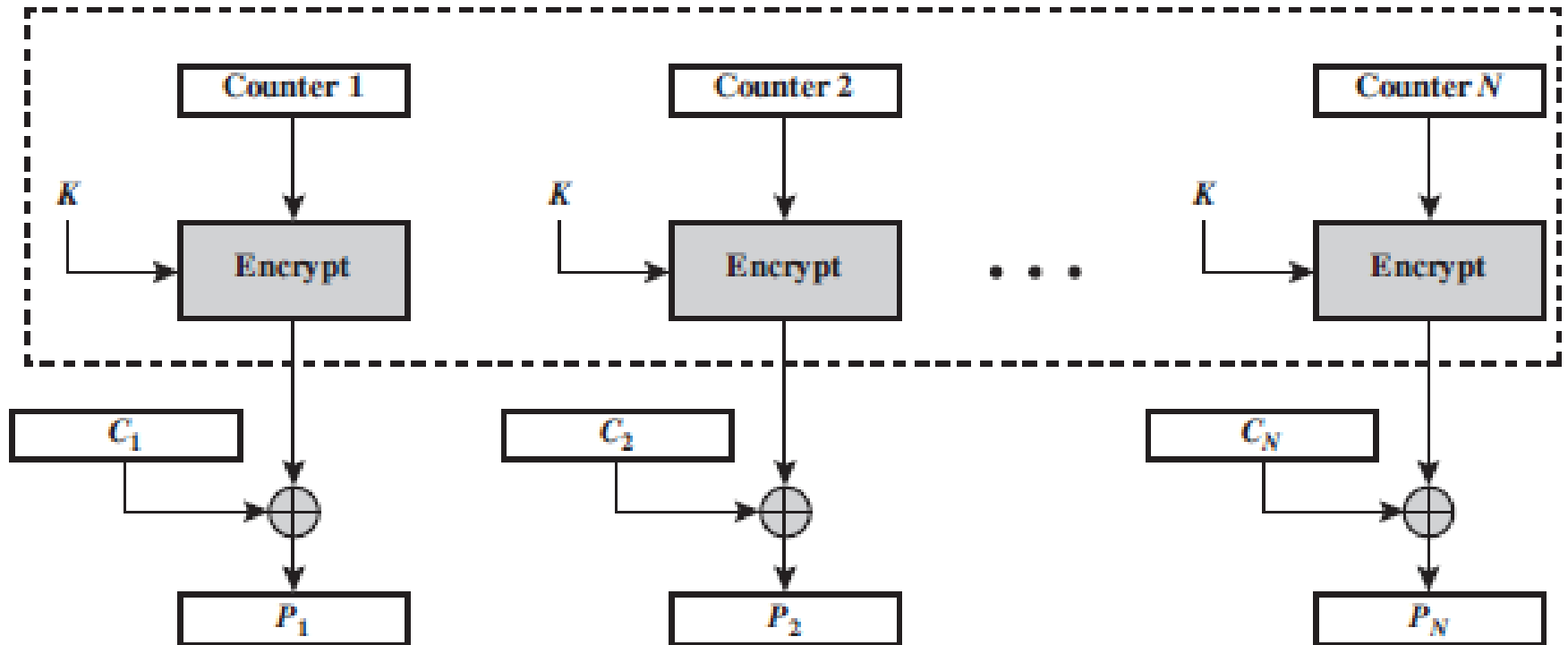
- Each bit in the ciphertext is independent of the previous bit or bits.
- This avoids error propagation
- Pre-compute of forward cipher is possible

OFB	$I_1 = \text{Nonce}$	$I_1 = \text{Nonce}$
	$I_j = O_{j-1} \quad j = 2, \dots, N$	$I_j = O_{j-1} \quad j = 2, \dots, N$
	$O_j = E(K, I_j) \quad j = 1, \dots, N$	$O_j = E(K, I_j) \quad j = 1, \dots, N$
	$C_j = P_j \oplus O_j \quad j = 1, \dots, N - 1$	$P_j = C_j \oplus O_j \quad j = 1, \dots, N - 1$
	$C_N^* = P_N^* \oplus \text{MSB}_u(O_N)$	$P_N^* = C_N^* \oplus \text{MSB}_u(O_N)$

Counter (CTR) Mode



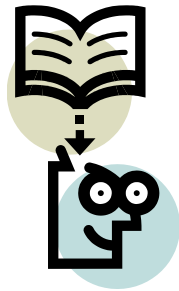
Counter (CTR) Mode



UNIT-5

Public key cryptography





Lecture 26

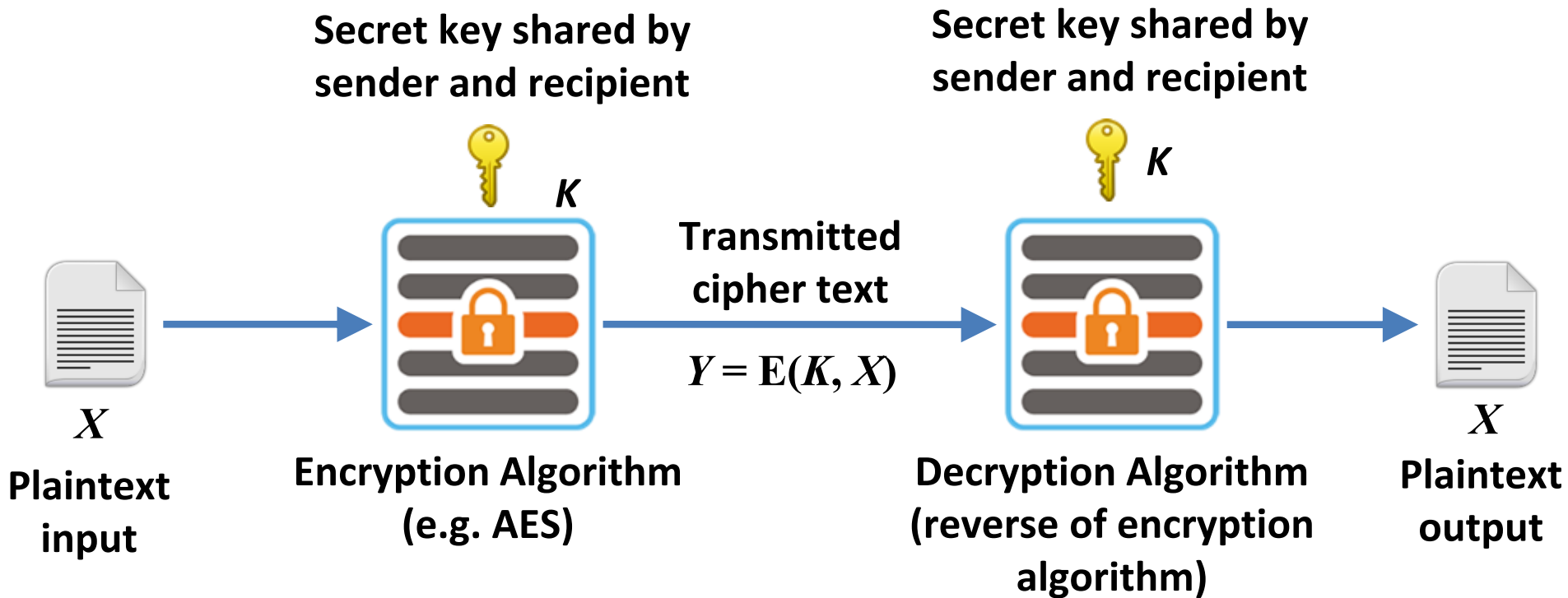


Asymmetric Encryption and RSA

Outline

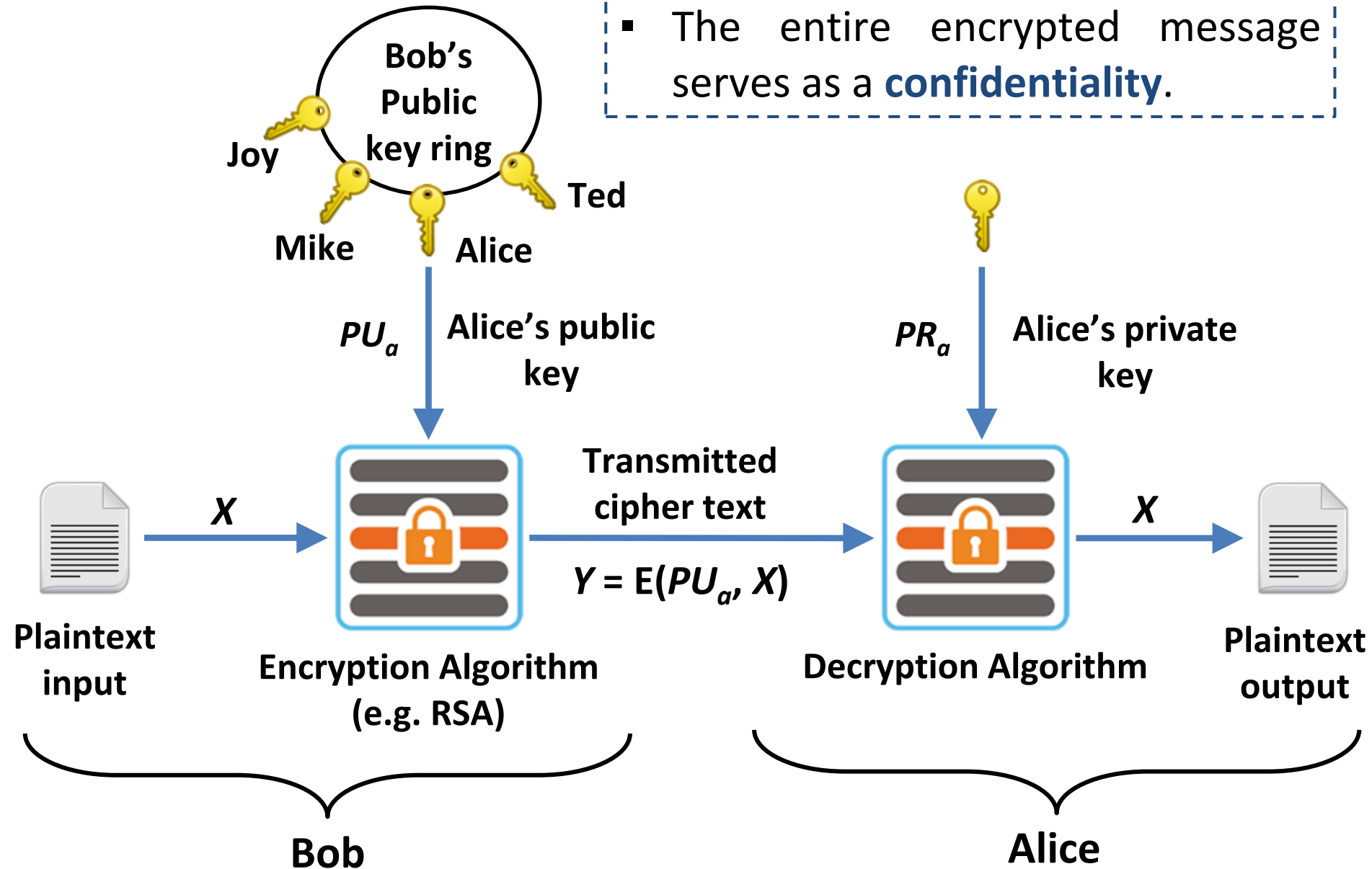
- RSA
- RSA proof with example
- RSA attacks
- Rabin cryptosystem
- Key Management
- Diffie-Hillman Key Exchange algorithm

Symmetric key Encryption



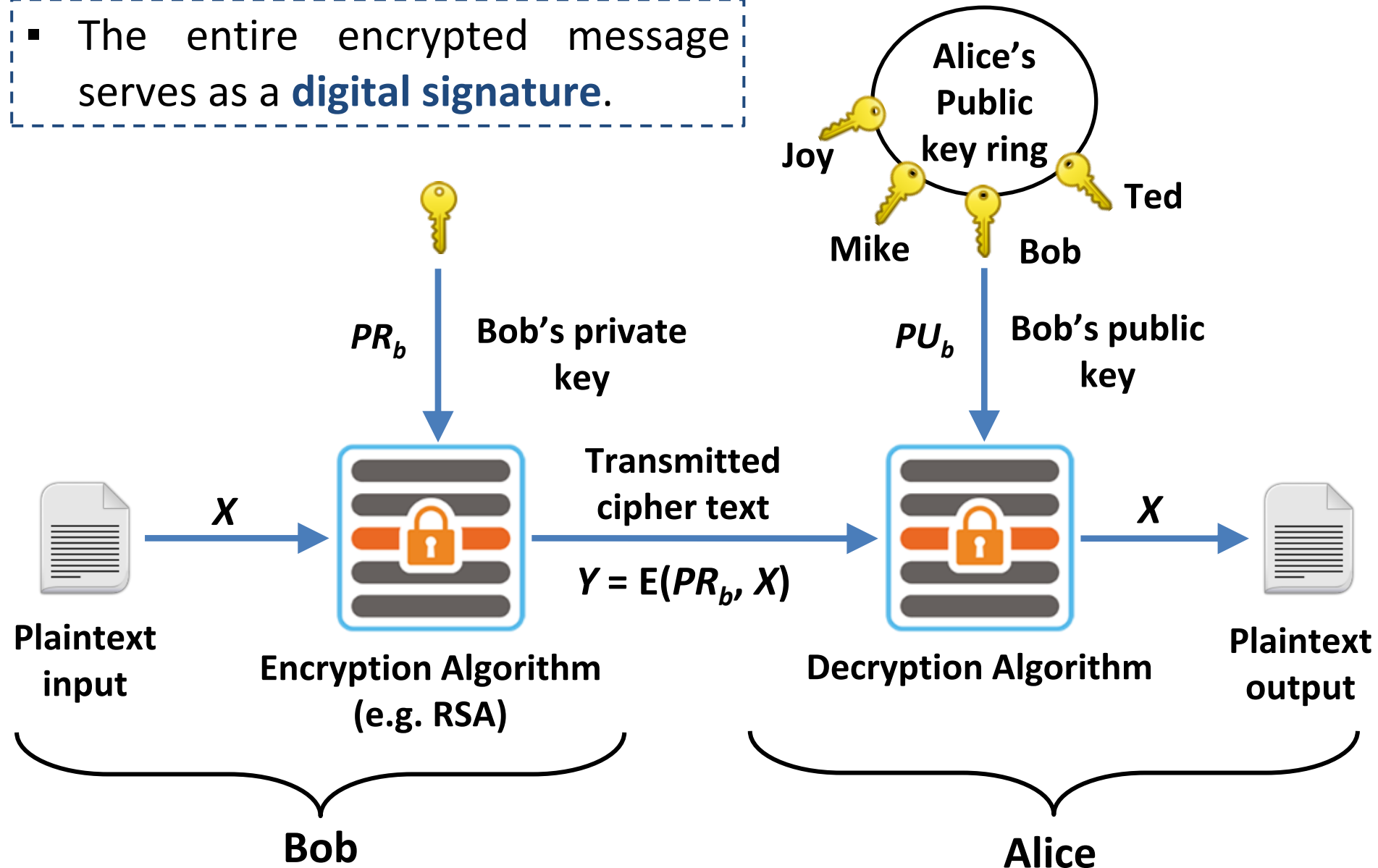
Asymmetric key Encryption with Public Key

- The entire encrypted message serves as a **confidentiality**.

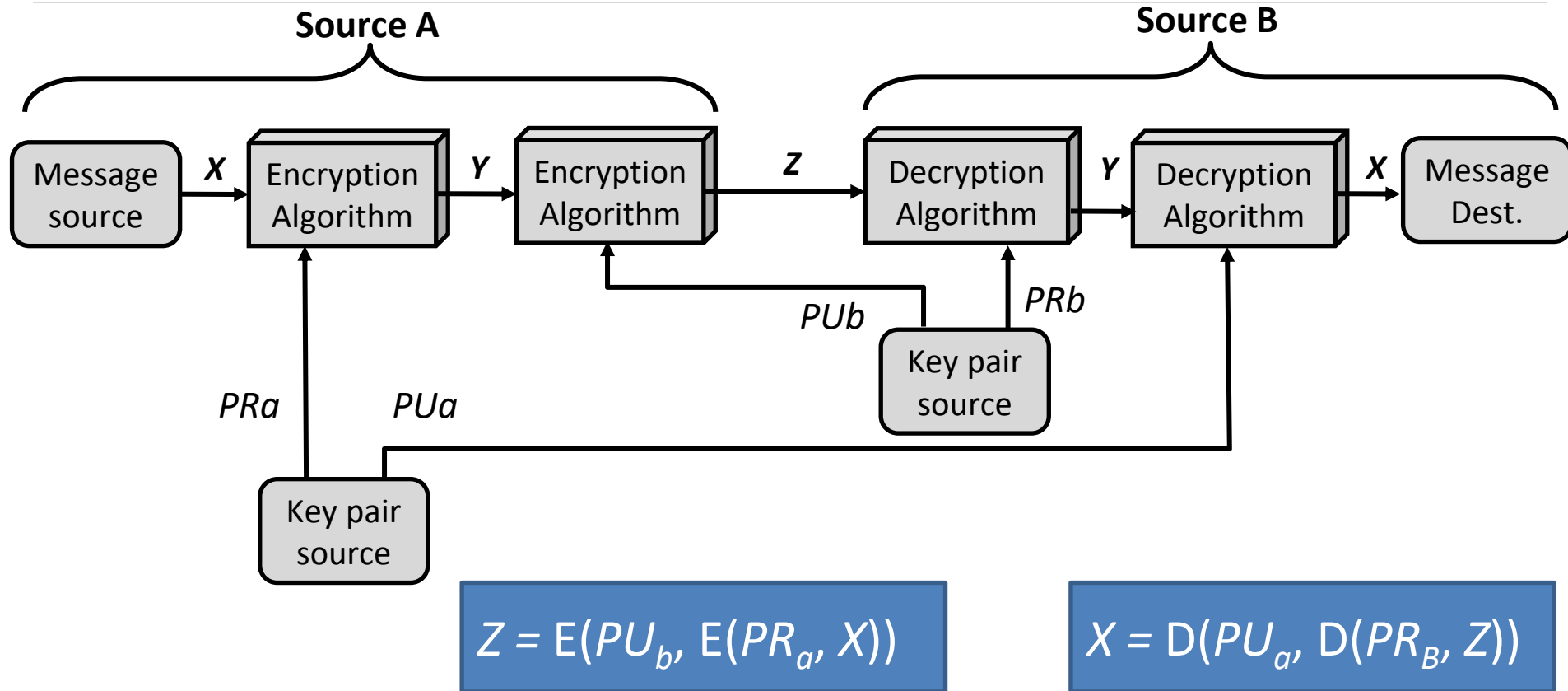


Asymmetric key Encryption with Private Key

- The entire encrypted message serves as a **digital signature**.



Authentication and Confidentiality



Applications for Public-Key Cryptosystems

- **Encryption/decryption:** The sender encrypts a message with the recipient's public key.
- **Digital signature:** The sender “signs” a message with its private key. Signing is achieved by a cryptographic algorithm applied to the message or to a small block of data that is a function of the message.
- **Key exchange:** Two sides cooperate to exchange a session key. Several different approaches are possible, involving the private key(s) of one or both parties.

RSA Algorithm

- **RSA** is a block cipher in which the Plaintext and Ciphertext are represented as integers between **0** and **$n-1$** for some n .
- Large messages can be broken up into a number of blocks.
- Each block would then be represented by an integer.

Step-1: Generate Public key and Private key

Step-2: Encrypt message using Public key

Step-3: Decrypt message using Private key

Step-1: Generate Public key and Private key

- Select two large prime numbers: **p** and **q**
- Calculate modulus : **$n = p * q$**
- Calculate Euler's totient function : **$\phi(n) = (p-1) * (q-1)$**
- Select **e** such that **e** is **relatively prime** to **$\phi(n)$** and **$1 < e < \phi(n)$**

Two numbers are relatively prime if they have no common factors other than 1.

- Determine **d** such that **$d * e \equiv 1 \pmod{\phi(n)}$**
- Publickey : **$PU = \{ e, n \}$**
- Privatekey : **$PR = \{ d, n \}$**

Step-2 : Encrypt Message

$$PU = \{ e, n \}$$

- Encryption Using Public key:

$$C = M^e \bmod n$$

The diagram illustrates the encryption formula $C = M^e \bmod n$. Three labels are positioned below the formula, each with an arrow pointing to a specific part of the equation: 'Ciphertext' points to 'C', 'Input Message' points to 'M', and 'Publickey' points to 'e'.

Step-3 : Decrypt Message

$$PR = \{ d, n \}$$

- Encryption Using Public key:

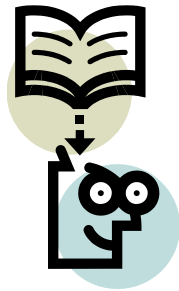
$$M = C^d \bmod n$$

Plaintext Message

Cipher Message

Privatekey

The diagram illustrates the decryption process. A blue box contains the formula $M = C^d \bmod n$. Three arrows point from labels below to components of the formula: 'Plaintext Message' points to 'M', 'Cipher Message' points to 'C', and 'Privatekey' points to 'd'.



Lecture 27



RSA proof with example

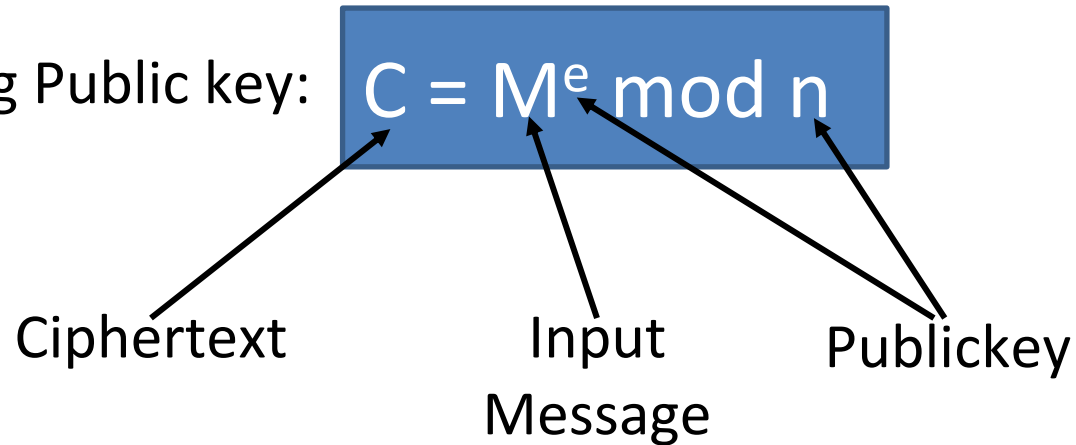
Step-1: Generate Public key and Private key

- Select two large prime numbers: $p = 3$ and $q = 11$
- Calculate modulus : $n = p * q, n = 33$
- Calculate Euler's totient function : $\phi(n) = (p-1) * (q-1)$
 $\phi(n) = (3 - 1) * (11 - 1) = 20$
- Select e such that e is **relatively prime** to $\phi(n)$ and $1 < e < \phi(n)$
- We have several choices for e : $7, 11, 13, 17, 19$ Let's take $e = 7$
- Determine d such that $d * e \equiv 1 \pmod{\phi(n)}$
- $? * 7 \equiv 1 \pmod{20}$
- $3 * 7 \equiv 1 \pmod{20}$
- Public key : $PU = \{ e, n \}, PU = \{ 7, 33 \}$
- Private key : $PR = \{ d, n \}, PR = \{ 3, 33 \}$

- This is equivalent to finding d which satisfies $de = 1 + j.\phi(n)$ where j is any integer.
- We can rewrite this as $d = (1 + j.\phi(n)) / e$

Step-2 : Encrypt Message

- Encryption Using Public key:



$PU = \{ e, n \}, PU = \{ 7, 33 \}$

For message $M = 14$

$$C = 14^7 \bmod 33$$

$$C = [(14^1 \bmod 33) \times (14^2 \bmod 33) \times (14^4 \bmod 33)] \bmod 33$$

$$C = (14 \times 31 \times 4) \bmod 33 = 1736 \bmod 33$$

$$C = 20$$

Step-3 : Decrypt Message

- Encryption Using Public key:

$$M = C^d \bmod n$$

Plaintext
Message

Cipher
Message

Privatekey

$$PR = \{ d, n \}, PR = \{ 3, 33 \}$$

For Ciphertext $C = 20$

$$M = 20^3 \bmod 33$$

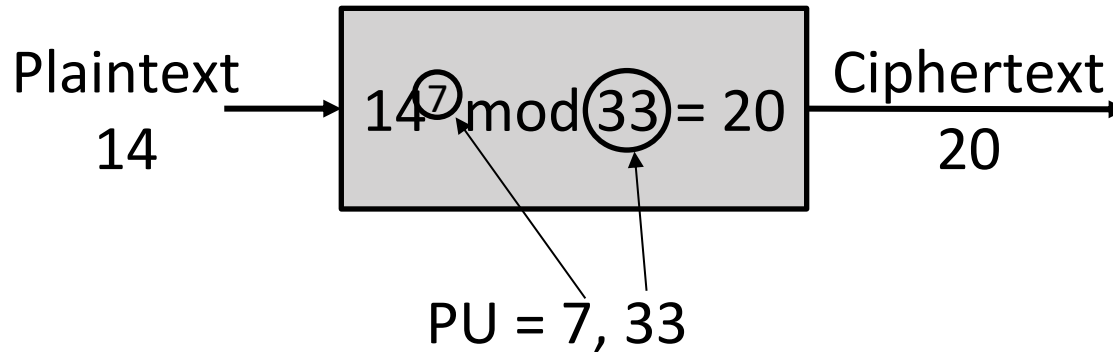
$$M = [(20^1 \bmod 33) \times (20^2 \bmod 33)] \bmod 33$$

$$M = (20 \times 4) \bmod 33 = 80 \bmod 33$$

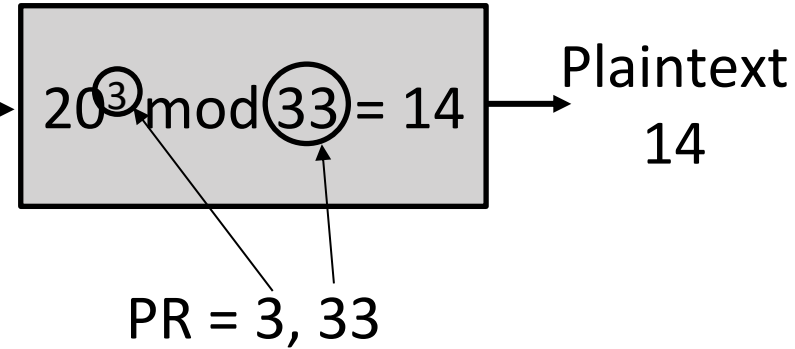
$$M = 14$$

Example RSA Algorithm

Encryption



Decryption



RSA Example

- Find n , $\phi(n)$, e , d for $p=7$ and $q=19$ then demonstrate encryption and decryption for $M=6$

$$n = p * q = 7 * 19 = 133$$

$$\phi(n) = (p - 1) * (q - 1) = 108$$

Finding e relatively prime to 108

$$e = 2 \Rightarrow \text{GCD}(2, 108) = 2 \text{ (no)}$$

$$e = 3 \Rightarrow \text{GCD}(3, 108) = 3 \text{ (no)}$$

$$e = 5 \Rightarrow \text{GCD}(5, 108) = 1 \text{ (Yes)}$$

- Finding d such that $(d * e) \bmod \phi(n) = 1$
- We can rewrite this as $d = (1 + j * \phi(n)) / e$

$$j = 0 \Rightarrow d = 1 / 5 = 0.2 \text{ ? integer ? (no)}$$

$$j = 1 \Rightarrow d = 109 / 5 = 21.8 \text{ ? integer ? (no)}$$

$$j = 2 \Rightarrow d = 217 / 5 = 43.4 \text{ ? integer ? (no)}$$

$$j = 3 \Rightarrow d = 325 / 5 = 65 \text{ integer ? (yes)}$$

Public key :

$$\text{PU} = \{ e, n \} = \{ 5, 133 \}$$

Private key :

$$\text{PR} = \{ d, n \} = \{ 65, 133 \}$$

RSA Example – cont...

- Encryption:

$$C = M^e \bmod n$$

$$PU = \{ e, n \}, PU = \{ 5, 133 \}$$

For message $M = 6$

$$C = 6^5 \bmod 133$$

$$C = 7776 \bmod 133$$

$$C = 62$$

- Decryption:

$$M = C^d \bmod n$$

$$PR = \{ d, n \}, PR = \{ 65, 133 \}$$

For $C = 62$

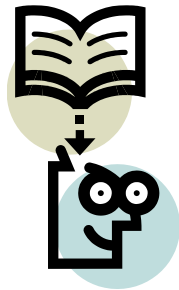
$$M = 62^{65} \bmod 133$$

$$M = 2666 \bmod 133$$

$$M = 6$$

RSA Example

- P and Q are two prime numbers. $P=7$, and $Q=17$. Take public key $E=5$. If plain text value is 10, then what will be cipher text value according to RSA algorithm?
- $n = 119$
- $\phi(n) = 96$
- $e = 5$
- $d = 77$
- $PU = \{ 5, 119 \}$
- $PR = \{77, 119\}$
- $C = 10^5 \bmod 119 \Rightarrow C = 40$



Lecture 28



Diffie-Hellman key Exchange

Diffie-Hellman key Exchange

- The purpose of the **Diffie-Hellman** algorithm is to enable two users to securely exchange a key that can be used for subsequent encryption of message.
- This algorithm depends for its effectiveness on the difficulty of computing **discrete logarithms**.

Primitive root

- Let p be a prime number
- Then a is a primitive root for p , if the powers of a modulo p generates all integers from 1 to $p - 1$ in some permutation.

$$a \bmod p, a^2 \bmod p, \dots, a^{p-1} \bmod p$$

- Example: $p = 7$ then primitive root is 3 because powers of 3 mod 7 generates all the integers from 1 to 6

$$3^1 = 3 \equiv 3 \pmod{7}$$

$$3^2 = 9 \equiv 2 \pmod{7}$$

$$3^3 = 27 \equiv 6 \pmod{7}$$

$$3^4 = 81 \equiv 4 \pmod{7}$$

$$3^5 = 243 \equiv 5 \pmod{7}$$

$$3^6 = 729 \equiv 1 \pmod{7}$$

Discrete Logarithm

- For any integer b and a primitive root a of prime number p , we can find a unique exponent i such that

$$b = a^i \pmod{p} \text{ where } 0 \leq i \leq (p - 1)$$

- The exponent i is referred as the discrete logarithm of b for the base a , mod p . It expressed as below.

$$i = \text{dlog}_{a,p}(b)$$

Diffie-Hellman Key Exchange – Cont...

- User A and User B agree on two large prime numbers q and α .
User A and User B can use insecure channel to agree on them.
- User A selects a random integer $X_A < q$ and calculates Y_A

Diffie-Hellman Key Exchange – Cont...

Global Public Elements

q prime number

α $\alpha < q$ and α is primitive root of q

User A Key Generation

Select private X_A $X_A < q$

Calculate public Y_A $Y_A = \alpha^{X_A} \bmod q$

User B Key Generation

Select private X_B $X_B < q$

Calculate public Y_B $Y_B = \alpha^{X_B} \bmod q$

Diffie-Hellman Key Exchange – Cont...

User A Key Generation

Select private X_A $X_A < q$

Calculate public Y_A $Y_A = \alpha^{X_A} \bmod q$

User B Key Generation

Select private X_B $X_B < q$

Calculate public Y_B $Y_B = \alpha^{X_B} \bmod q$

Calculation of Secret Key by User A

$K =$

Calculation of Secret Key by User b

$K =$

Diffie-Hellman Key Exchange – Cont...

User A Key Generation

Private X_A $X_A < q$,

Public Y_A

$$Y_A = \alpha^{X_A} \bmod q$$

User B Key Generation

Private X_B $X_B < q$,

Public Y_B

$$Y_B = \alpha^{X_B} \bmod q$$

Secret Key by User A : $K = (Y_B)^{X_A} \bmod q$

Secret Key by User B : $K = (Y_A)^{X_B} \bmod q$

$$K = (Y_B)^{X_A} \bmod q$$

$$K = (\alpha^{X_B} \bmod q)^{X_A} \bmod q$$

$$K = (\alpha^{X_B})^{X_A} \bmod q$$

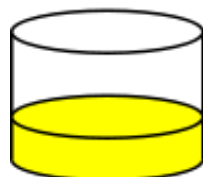
$$K = \alpha^{X_B X_A} \bmod q$$

$$K = (\alpha^{X_A})^{X_B} \bmod q$$

$$K = (\alpha^{X_A} \bmod q)^{X_B} \bmod q$$

$$K = (Y_A)^{X_B} \bmod q$$

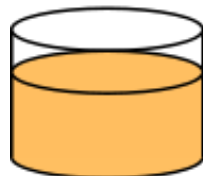
Alice



+



=



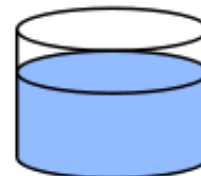
Bob



+



=

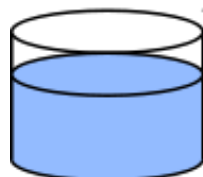


Common paint

Secret colours

Public transport

(assume
that mixture separation
is expensive)



+



=



+



=



Secret colours

Common secret

Diffie-Hellman Key Exchange Example

- Alice and Bob agree on a prime number $q = 23$
- $\alpha = 5$ as primitive root of q
- Alice selects a private integer $X_A = 6$
- Alice computes $Y_A = \alpha^{X_A} \bmod q \Rightarrow Y_A = 5^6 \bmod 23 = 8$
- Bob selects a private integer $X_B = 15$
- Bob computes $Y_B = \alpha^{X_B} \bmod q \Rightarrow Y_B = 5^{15} \bmod 23 = 19$
- Alice sends Y_A to Bob and Bob sends Y_B to Alice
- Alice computes key $K = (Y_B)^{X_A} \bmod q \Rightarrow K = (19)^6 \bmod 23$
- $K = 2$
- Bob computes key $K = (Y_A)^{X_B} \bmod q \Rightarrow K = (8)^{15} \bmod 23$
- $K = 2$

Diffie-Hellman Key Exchange Illustration

Alice

Alice and Bob share a prime number q and an integer α , such that $\alpha < q$ and α is a primitive root of q

Alice generates a private key X_A such that $X_A < q$

Alice calculates a public key $Y_A = \alpha^{X_A} \bmod q$

Alice receives Bob's public key Y_B in plaintext

Alice calculates shared secret key $K = (Y_B)^{X_A} \bmod q$

Bob

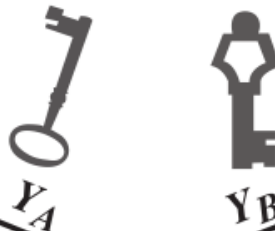
Alice and Bob share a prime number q and an integer α , such that $\alpha < q$ and α is a primitive root of q

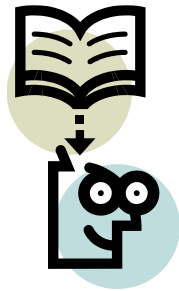
Bob generates a private key X_B such that $X_B < q$

Bob calculates a public key $Y_B = \alpha^{X_B} \bmod q$

Bob receives Alice's public key Y_A in plaintext

Bob calculates shared secret key $K = (Y_A)^{X_B} \bmod q$





Lecture 29



Rabin cryptosystem

Step-1: Generate Public key and Private key

- Select two large prime numbers: $p = 7$ and $q = 11$
- Calculate modulus : $n = p * q, n = 77$
- Public key : $PU = \{ n \}, PU = \{ 77 \}$
- Private key : $PR = \{ p, q \}, PR = \{ 7, 11 \}$

Step-2 : Encrypt Message

$$C = M^2 \bmod n$$

$$PU = \{ n \}, PU = \{ 77 \}$$

For message $M = 5$ ($0 < M < n$)

$$C = 5^2 \bmod 77$$

$$C = 25 \bmod 77$$

$$C = 25$$

Step-3 : Decrypt Message

Calculate Roots Modulo p and q

$$M_p = C^{(p+1)/4} \bmod p$$

$$M_q = C^{(q+1)/4} \bmod q$$

$$M_p = 25^{(7+1)/4} \bmod 7$$

$$M_p = 25^2 \bmod 7$$

$$M_p = 625 \bmod 7$$

$$M_p = 2$$

$$M_q = 25^{(11+1)/4} \bmod 7$$

$$M_q = 25^3 \bmod 7$$

$$M_q \equiv 3^3 \bmod 7$$

$$M_q = 5$$

Step-3 : Decrypt Message

Determine the Four Plaintext Candidates

- ☐ $M \equiv +M_p(\text{mod } p)$ and $M \equiv +M_q(\text{mod } q)$
- ☐ $M \equiv +M_p(\text{mod } p)$ and $M \equiv -M_q(\text{mod } q)$
- ☐ $M \equiv -M_p(\text{mod } p)$ and $M \equiv +M_q(\text{mod } q)$
- ☐ $M \equiv -M_p(\text{mod } p)$ and $M \equiv -M_q(\text{mod } q)$

(Finding the Four Plaintexts M_1, M_2, M_3, M_4 for $C=25$):

We have $p=7$, $q=11$, $n=77$, $M_p=2$, $M_q=5$. The roots are:

- $\pm M_p \text{ mod } 7 \rightarrow 2$ and $7-2=5$
- $\pm M_q \text{ mod } 11 \rightarrow 5$ and $11-5=6$

Step-3 : Decrypt Message

Determine the Four Plaintext Candidates

Pair	Congruences	Chinese Remainder Theorem Solution	Result	
P1	$M \equiv 2 \pmod{7}$ and $M \equiv 5 \pmod{11}$	Solved for M	M1=16	
P2	$M \equiv 2 \pmod{7}$ and $M \equiv 6 \pmod{11}$	Solved for M	M2=72	
P3	$M \equiv 5 \pmod{7}$ and $M \equiv 5 \pmod{11}$	Solved for M	M3=5 (The original message)	
P4	$M \equiv 5 \pmod{7}$ and $M \equiv 6 \pmod{11}$	Solved for M	M4=61	

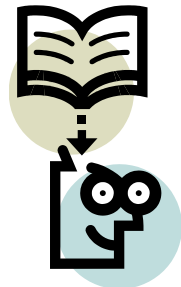
UNIT-6

Message Authentication & Hash Function



Outline

- Authentication requirements
- Functions
- Message authentication codes (MAC)
- Hash functions .
- Security of Hash functions



Lecture 30



Authentication Requirements

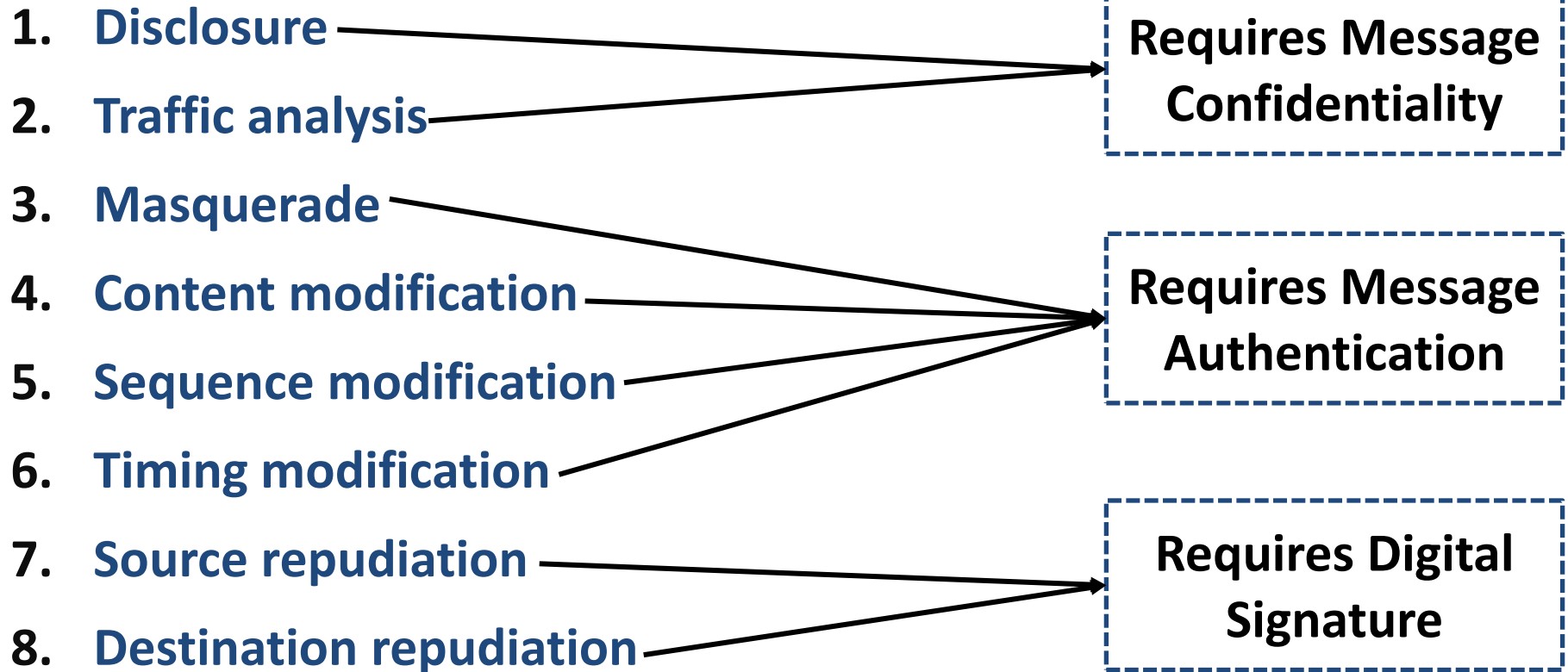
Message Authentication

- **Message authentication** is a procedure to verify that received messages come from the genuine source and have not been altered.
- Message authentication may also verify sequencing and timeliness.
- Message authentication is a mechanism or service used to verify the **integrity of a message**.
- Message authentication assures that data received are exactly as sent (i.e., contain no modification, insertion, deletion, or replay).

Message Authentication Requirements

1. **Disclosure:** Release of message contents
2. **Traffic analysis:** Discovery of the pattern of traffic between parties
3. **Masquerade:** Insertion of messages into the network from a fraudulent source
4. **Content modification:** Changes to the contents of a message
5. **Sequence modification:** Any modification to a sequence of messages between parties
6. **Timing modification:** Delay or replay of messages
7. **Source repudiation:** Denial of transmission of message by source
8. **Destination repudiation:** Denial of receipt of message by destination

Message Authentication Requirements





online hash calculator

AI Mode All Shopping Images Short videos Videos Forums More Tools

 Tools 4 noobs
https://www.tools4noobs.com/online_tools/hash

Online hash calculator - Online tools

This tool calculates the hash of a string using various algorithms, including md2, md4, md5, sha1, sha224, sha256, sha3-224, and many more.

 Md5file.com
<https://md5file.com/calculator>

File Hash Online Calculator WASM

Calculates MD5, SHA1, SHA2 (SHA256), and SHA512 hashes all at once · The browser performs all calculations without uploading data to the server · Supports ...

 Hash-File.Online
<https://hash-file.online>

Hash File Online | Hash Value Calculator - File Hash Checker ...

Hash file online with our free hash value calculator. Check file hash, calculate hash values, and verify checksums for MD5, SHA-1, SHA-256, SHA-512.

 PELock
<https://www.pelock.com/products/hash-calculator>

Hash Calculator Online — String & File Hash Generator

The Hash Calculator Online calculates cryptographic hash values of strings or files (max 32MB) using algorithms like MD5, SHA1, SHA2, and CRC32.

 github.io
<https://emn178.github.io/online-tools/sha256>

SHA256 - Online Tools

This SHA256 online tool helps you calculate hashes from strings. You can input UTF-8, UTF-16, Hex, Base64, or other encodings. It also supports HMAC.

Online hash calculator

[Home](#) / [Online tools](#) / Hash calculator

Calculates the hash of string using various algorithms.

Ganpat University

Algorithm:

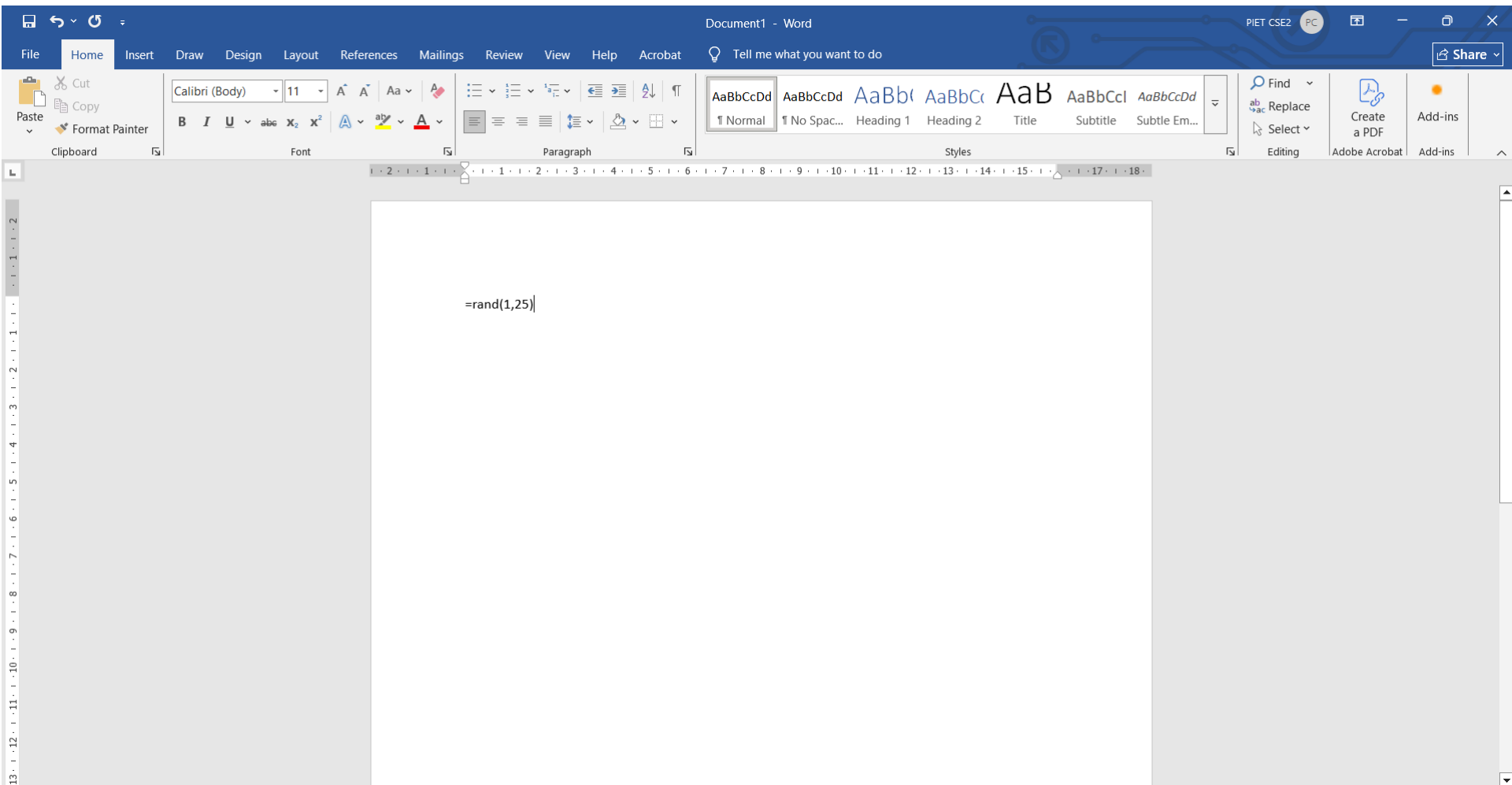
md2

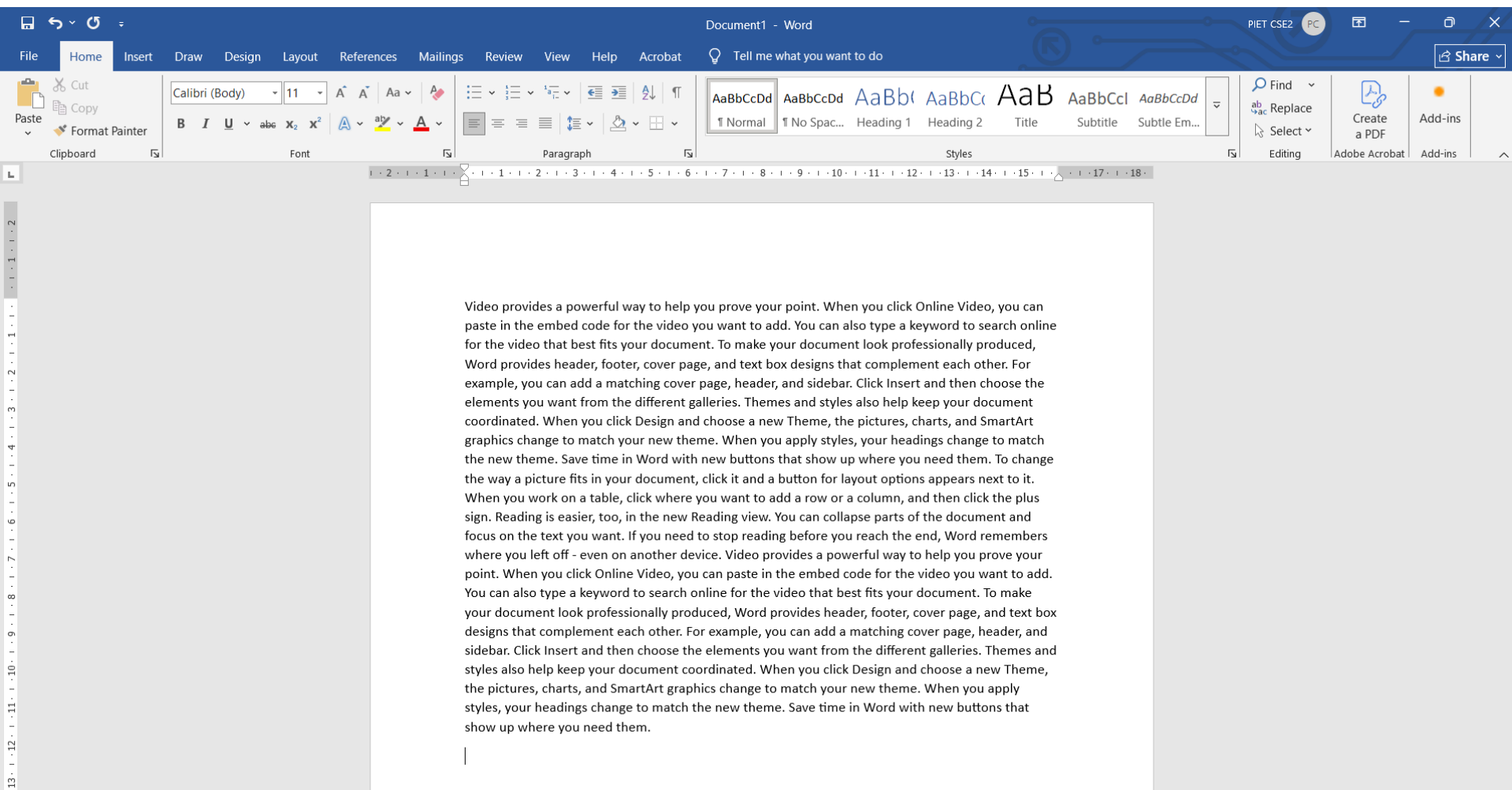
Hash this!

Result: c5ff629b3c61639e7c787337240976a9

Supported algorithms

Hashing engines supported: md2, md4, md5, sha1, sha224, sha256, sha384, sha512/224, sha512/256, sha512, sha3-224, sha3-256, sha3-384, sha3-512, ripemd128, ripemd160, ripemd256, ripemd320, whirlpool, tiger128.3, tiger160.3, tiger192.3, tiger128.4, tiger160.4, tiger192.4, snefru, snefru256, gost, gost-crypto, adler32, crc32, crc32b, crc32c, fnv132, fnv1a32, fnv164, fnv1a64, joaat, haval128.3, haval160.3, haval192.3, haval224.3, haval256.3, haval128.4, haval160.4, haval192.4, haval224.4, haval256.4, haval128.5, haval160.5, haval192.5, haval224.5, haval256.5.





Online hash calculator

[Home](#) / [Online tools](#) / Hash calculator

Calculates the hash of string using various algorithms.

Video provides a powerful way to help you prove your point. When you click Online Video, you can paste in the embed code for the video you want to add. You can also type a keyword to search online for the video that best fits your document. To make your document look professionally produced, Word provides header, footer, cover page, and text box designs that complement each other. For example, you can add a matching cover page, header, and sidebar. Click Insert and then choose the elements you want from the different galleries. Themes and styles also help keep your document coordinated. When you click Design and choose a new Theme, the pictures, charts, and SmartArt graphics change to match your new theme. When you apply styles, your headings change to match the new theme. Save time in Word with new buttons that show up where you need them. To change the way a picture fits in your document, click it and a button for layout options appears next to it. When you work on a table, click where you want to add a row or a column, and then click the plus sign. Reading is easier, too, in the new Reading view. You can collapse parts of the document and focus on the text you want. If you need to stop reading before you reach the end, Word remembers where you left off - even on another

Algorithm:

md2

Hash this!

Result: 820460828c166b36b797fd49a229999b

Supported algorithms

Hashing engines supported: md2, md4, md5, sha1, sha224, sha256, sha384, sha512/224, sha512/256, sha512, sha3-224, sha3-256, sha3-384, sha3-512, ripemd128, ripemd160, ripemd256, ripemd320, whirlpool, tiger128.3, tiger160.3, tiger192.3, tiger128.4, tiger160.4, tiger192.4, snefru, snefru256, gost, gost-crypto, adler32, crc32, crc32b, crc32c, fnv132, fnv1a32, fnv164, fnv1a64, joaat, haval128.3, haval160.3, haval192.3, haval224.3, haval256.3, haval128.4, haval160.4, haval192.4, haval224.4, haval256.4, haval128.5, haval160.5, haval192.5, haval224.5, haval256.5.

Online hash calculator

[Home](#) / [Online tools](#) / Hash calculator

Calculates the hash of string using various algorithms.

b

Algorithm:

md2

Hash this!

Result: 82ce940b1b4fd2ecd8236e81a6f8b5cb

Supported algorithms

Hashing engines supported: md2, md4, md5, sha1, sha224, sha256, sha384, sha512/224, sha512/256, sha512, sha3-224, sha3-256, sha3-384, sha3-512, ripemd128, ripemd160, ripemd256, ripemd320, whirlpool, tiger128,3, tiger160,3, tiger192,3, tiger128,4, tiger160,4, tiger192,4, snefru, snefru256, gost, gost-crypto, adler32, crc32, crc32b, crc32c, fnv132, fnv1a32, fnv164, fnv1a64, joaat, haval128,3, haval160,3, haval192,3, haval224,3, haval256,3, haval128,4, haval160,4, haval192,4, haval224,4, haval256,4, haval128,5, haval160,5, haval192,5, haval224,5, haval256,5.

Online hash calculator

[Home](#) / [Online tools](#) / Hash calculator

Calculates the hash of string using various algorithms.

b

Algorithm: sha512

Hash this!

Result:
5267768822ee624d48fce15ec5ca79cbd602cb7f4c2157a516556991f22ef8c7b5ef7b18d1ff41c59370efb0858651d44a936c11b7b144c48fe04df3c6a3e8da

Supported algorithms

Hashing engines supported: md2, md4, md5, sha1, sha224, sha256, sha384, sha512/224, sha512/256, sha512, sha3-224, sha3-256, sha3-384, sha3-512, ripemd128, ripemd160, ripemd256, ripemd320, whirlpool, tiger128,3, tiger160,3, tiger192,3, tiger128,4, tiger160,4, tiger192,4, snefru, snefru256, gost, gost-crypto, adler32, crc32, crc32b, crc32c, fnv132, fnv1a32, fnv164, fnv1a64, joaat, haval128,3, haval160,3, haval192,3, haval224,3, haval256,3, haval128,4, haval160,4, haval192,4, haval224,4, haval256,4, haval128,5, haval160,5, haval192,5, haval224,5, haval256,5.

Online hash calculator

[Home](#) / [Online tools](#) / Hash calculator

Calculates the hash of string using various algorithms.

the new theme. Save time in Word with new buttons that show up where you need them. To change the way a picture fits in your document, click it and a button for layout options appears next to it. When you work on a table, click where you want to add a row or a column, and then click the plus sign. Reading is easier, too, in the new Reading view. You can collapse parts of the document and focus on the text you want. If you need to stop reading before you reach the end, Word remembers where you left off - even on another device. Video provides a powerful way to help you prove your point. When you click Online Video, you can paste in the embed code for the video you want to add. You can also type a keyword to search online for the video that best fits your document. To make your document look professionally produced, Word provides header, footer, cover page, and text box designs that complement each other. For example, you can add a matching cover page, header, and sidebar. Click Insert and then choose the elements you want from the different galleries. Themes and styles also help keep your document coordinated. When you click Design and choose a new Theme, the pictures, charts, and SmartArt graphics change to match your new theme. When you apply styles, your headings change to match the new theme. Save time in Word with new buttons that show up where you need them.

Algorithm:

Hash this!

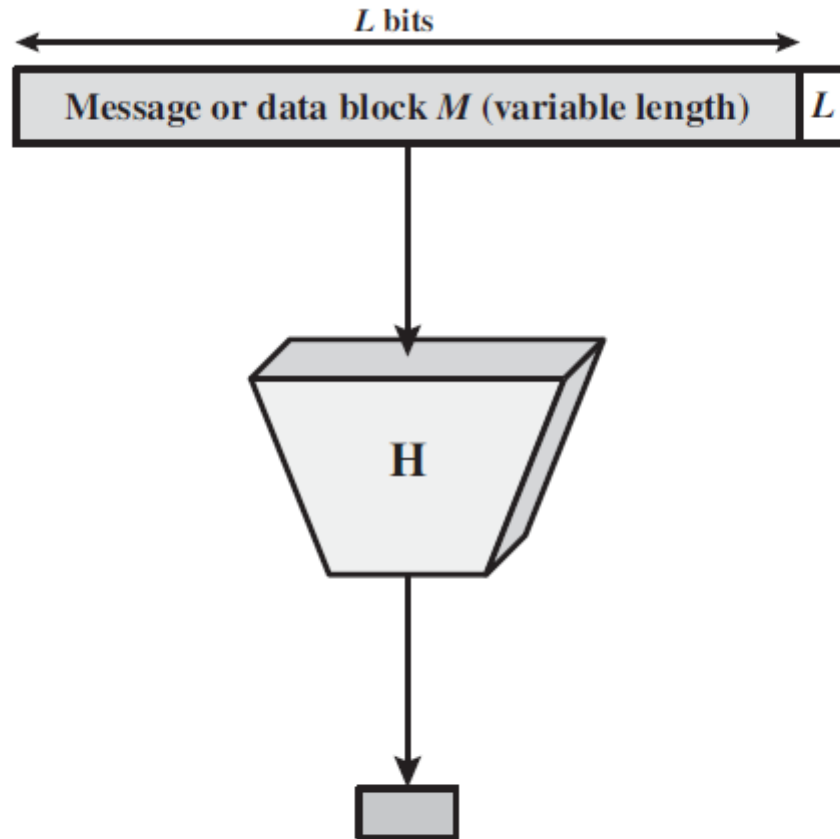
Result:

342e789f777e4e6f6251133bd67a42f2a230462e6090211c03e756a84813850f8c6ae0a4f562d4b542fb45c66d56953956b6712cb0a8592c7e85d7f1be80e812

Supported algorithms

Hashing engines supported: md2, md4, md5, sha1, sha224, sha256, sha384, sha512/224, sha512/256, sha512, sha3-224, sha3-256, sha3-384, sha3-512, ripemd128, ripemd160, ripemd256, ripemd320, whirlpool, tiger128,3, tiger160,3, tiger192,3, tiger128,4, tiger160,4, tiger192,4, snefru, snefru256, gost, gost-crypto, adler32, crc32, crc32c, crc32c, fnv132, fnv1a32, fnv164, fnv1a64, joaat, haval128,3, haval160,3, haval192,3, haval128,4, haval160,4, haval192,4, haval224,4, haval256,4, haval128,5, haval160,5, haval192,5, haval224,5, haval256,5.

Cryptographic Hash Function



Hash value h
(fixed length)

$$h = H(M)$$

Key Properties of cryptographic hash function

Deterministic

Computationally infeasible

Collision Resistance

Avalanche Effect

Application of cryptographic hash function

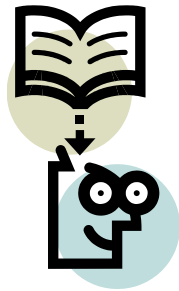
Message Authentication Codes (MACs)

Digital signatures

Data integrity verification

Password storage (hashed passwords)

Blockchain and cryptocurrency



Lecture 31



Message Authentication Code

Message Authentication Function

Hash Function

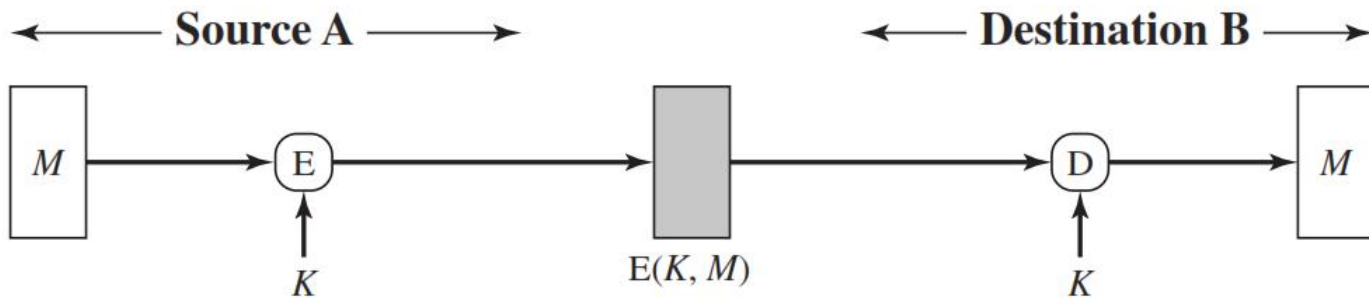
A function that maps a message of any length into a fixed length hash value, which serves as the authenticator

Message encryption

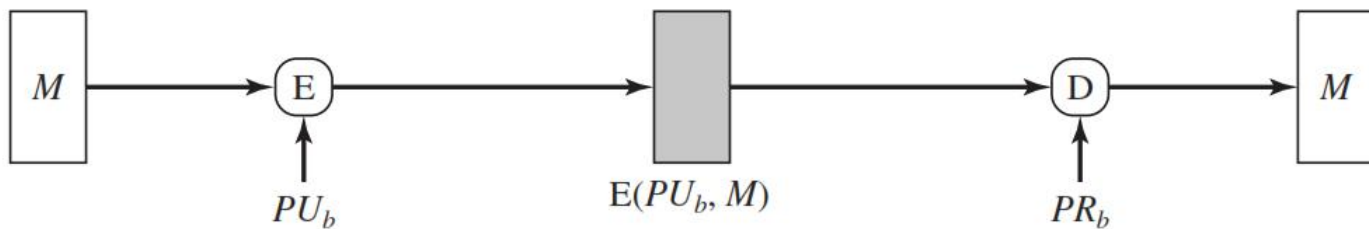
The ciphertext of the entire message serves as its authenticator

Message authentication code

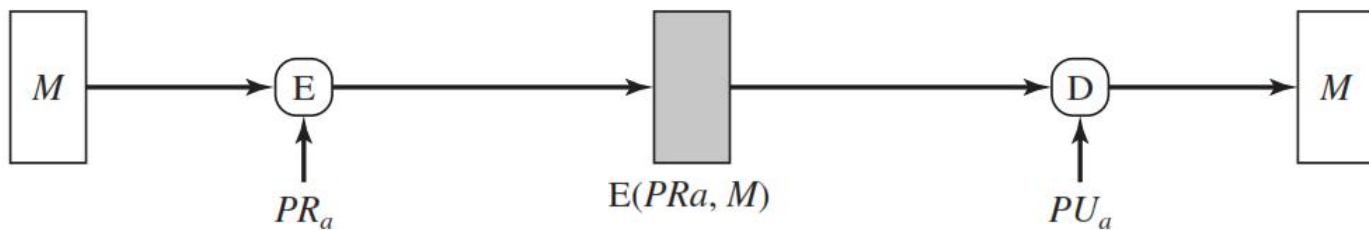
A function of the message and a secret key that produces a fixed-length value that serves as the authenticator



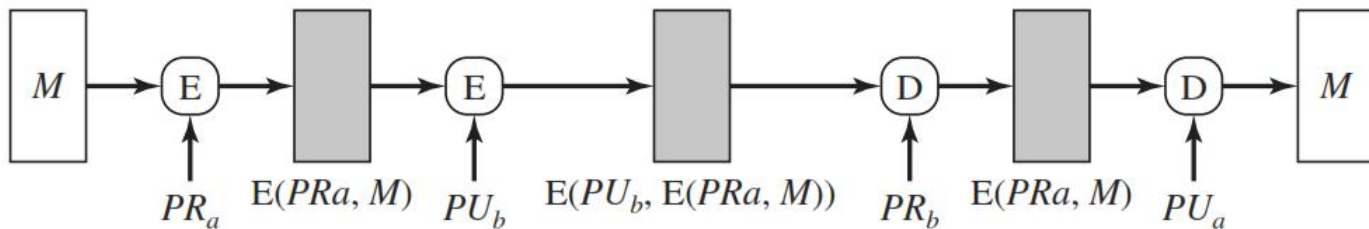
(a) Symmetric encryption: confidentiality and authentication



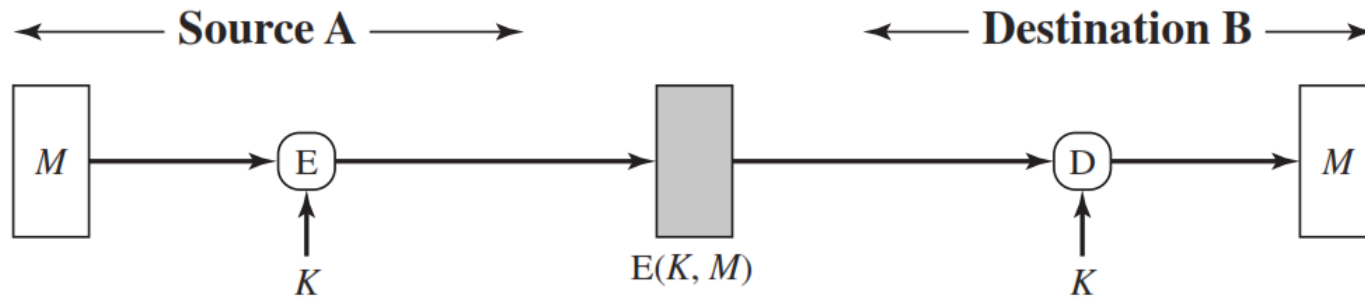
(b) Public-key encryption: confidentiality



(c) Public-key encryption: authentication and signature



(d) Public-key encryption: confidentiality, authentication, and signature



(a) Symmetric encryption: confidentiality and authentication

Process:

Both Source A and Destination B share the **same secret key (K)**.

A encrypts the message **M** using this key:

$$C = E(K, M)$$

B decrypts it using the **same key K**:

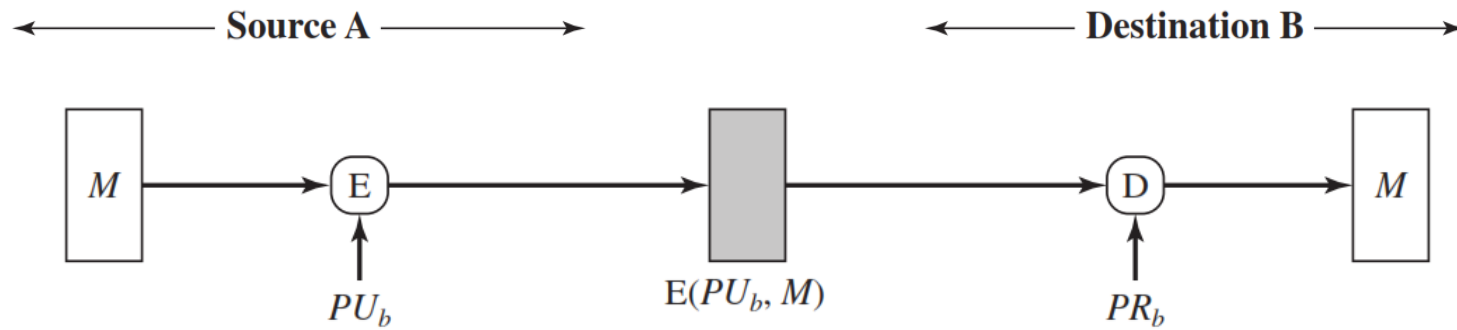
$$M = D(K, C)$$

Features:

Confidentiality: Only A and B know the secret key, so others cannot read the message.

Authentication: Since only A and B know K , B can be sure that the message came from A.

Drawback: Managing and securely sharing secret keys between multiple users is difficult.



(b) Public-key encryption: confidentiality

Process:

Destination B has a **public/private key pair** (PU_b , PR_b).

Source A encrypts the message with **B's public key**:

$$C = E(PU_b, M)$$

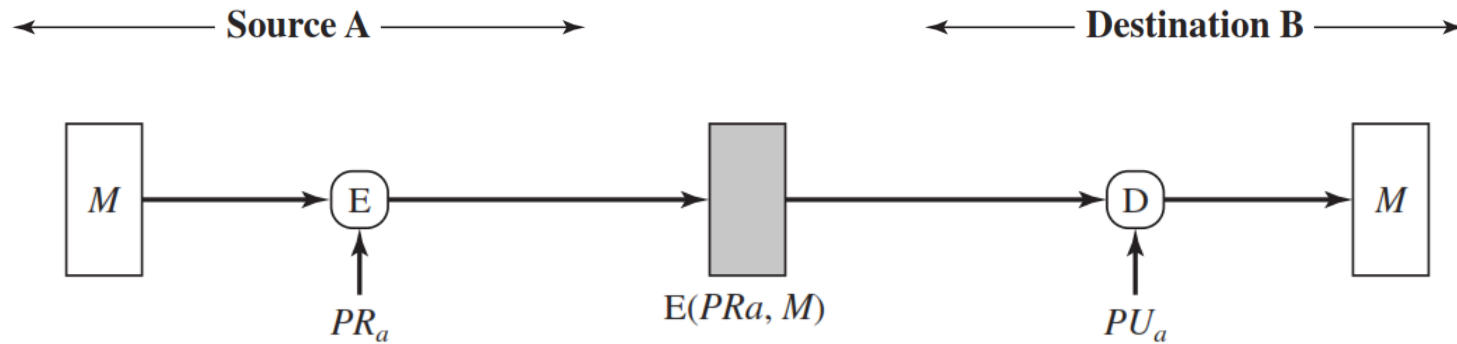
B decrypts it with **their private key**:

$$M = D(PR_b, C)$$

Features:

Confidentiality: Only B can decrypt because only B knows PR_b .

No Authentication: Anyone can use B's public key to send messages, so B can't be sure who sent it.



(c) **Public-key encryption: authentication and signature**

Process:

Source A has a **public/private key pair** (PU_a , PR_a).

A encrypts the message using **their private key**:

$$C = E(PR_a, M)$$

B decrypts it using **A's public key**:

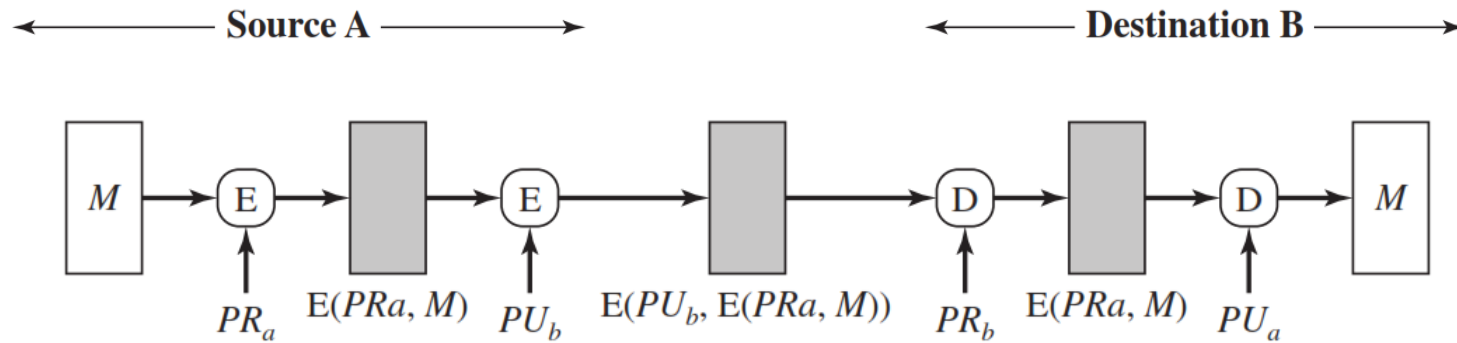
$$M = D(PU_a, C)$$

Features:

Authentication: Since only A knows PR_a , successful decryption with PU_a proves the message is from A.

Digital Signature: It also acts as a signature, verifying A's identity.

No Confidentiality: Anyone can decrypt using PU_a (public key).



(d) Public-key encryption: confidentiality, authentication, and signature

Process:

Combines both (b) and (c):

A first encrypts the message using **their private key (PR_a)** → provides **authentication**.

$$E(PR_a, M)$$

Then encrypts that result using **B's public key (PU_b)** → provides **confidentiality**.

$$E(PU_b, E(PR_a, M))$$

B decrypts first using **their private key (PR_b)**, then using **A's public key (PU_a)**:

$$M = D(PU_a, D(PR_b, E(PU_b, E(PR_a, M))))$$

Features:

Confidentiality: Only B can decrypt the outer layer (PU_b/PR_b).

Authentication & Signature: Decryption with PU_a confirms it was sent by A.

Integrity: Any modification to the message will be detected because decryption will fail.

Message Authentication Code

- An alternative authentication technique involves the use of a secret key to generate a small fixed-size block of data, known as a **cryptographic checksum** or **MAC**
- MAC is appended to the message. This technique assumes that two communicating parties, say A and B, share a common secret key K.
- When A has a message to send to B, it calculates the MAC as a function of the message and the key

$$\text{MAC} = C (K , M)$$

Purpose

Integrity → ensures the message was not altered during transmission.

Authentication → ensures the message really came from the sender who knows the secret key.

Message Authentication Code

Purpose

Integrity → ensures the message was not altered during transmission.

Authentication → ensures the message really came from the sender who knows the secret key.

How it works

Both sender (A) and receiver (B) share a **secret key K**.

A uses a MAC function C to compute:

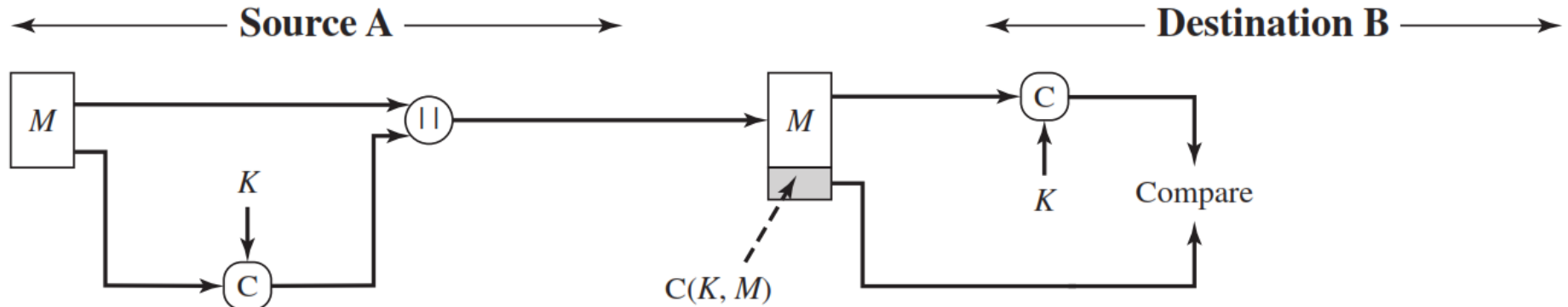
$$MAC = C(K, M)$$

Then A sends both the message **M** and the **MAC** to B.

B uses the same secret key K to compute its own MAC for the received message and **compares** it with the received MAC.

If both match → message is authentic and unmodified.

Message Authentication Code



(a) Message authentication

Process

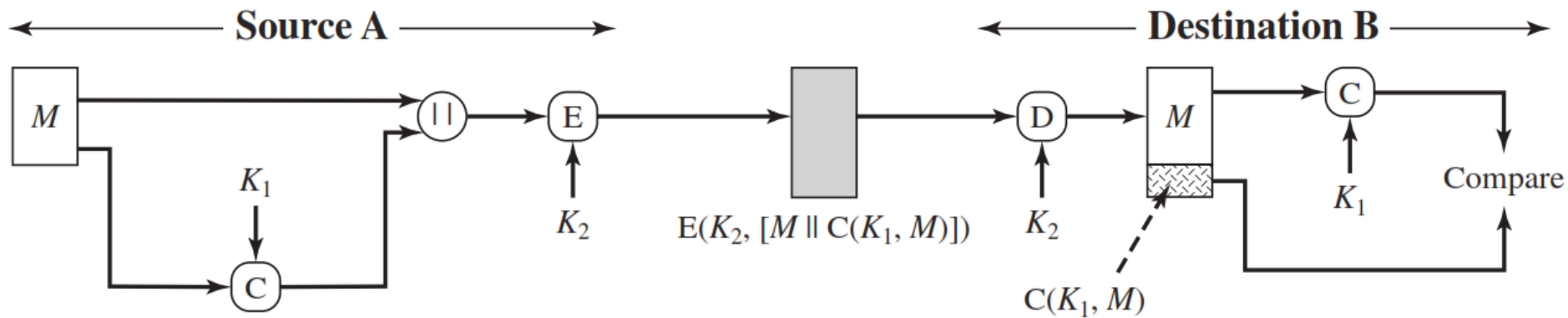
1. Source A computes: $C(K, M)$
2. A sends both M and $C(K, M)$ to B.
3. B recomputes $C(K, M)$ using the same key K and compares it.

Purpose

Authenticity: Only someone with key K can generate the correct MAC.

Integrity: If message changes, MAC comparison fails.

No Confidentiality: Message M is sent in plaintext.



(b) Message authentication and confidentiality; authentication tied to plaintext

Two keys are used:

K_1 :For MAC generation. K_2 :For encryption/decryption.

Process

Compute MAC:

$$C(K_1, M)$$

Concatenate message and MAC:

$$[M \parallel C(K_1, M)]$$

Encrypt the entire block:

$$E(K_2, [M \parallel C(K_1, M)])$$

Send to destination.

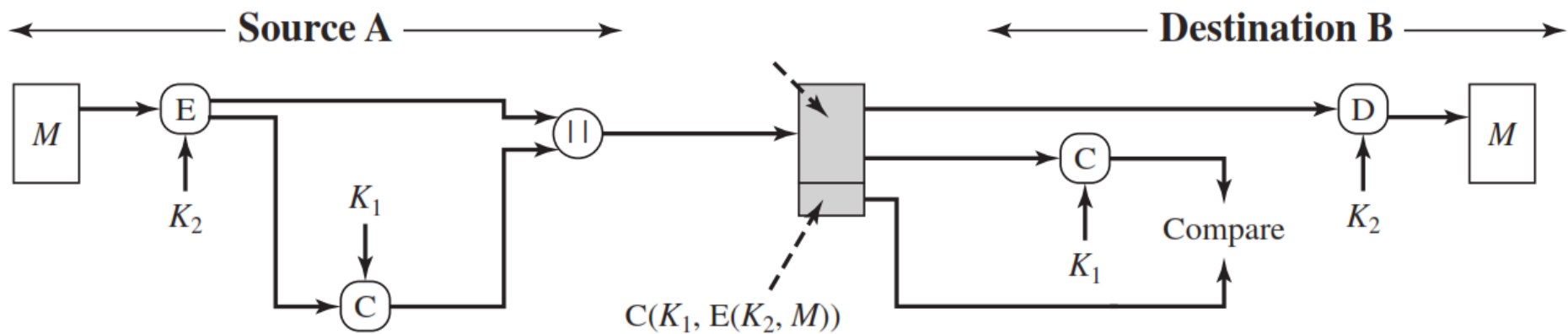
Destination decrypts $\rightarrow$ verifies MAC.

Purpose

Confidentiality (via encryption with K_2)

Authentication & Integrity (via MAC with K_1)

MAC is tied to plaintext because MAC is computed before encryption.



(c) Message authentication and confidentiality; authentication tied to ciphertext

Two keys again:

K_2 :For encryption/decryption. K_1 :For MAC computation.

Process

Encrypt the message:

$$E(K_2, M)$$

Compute MAC over the **ciphertext**:

$$C(K_1, E(K_2, M))$$

Send both:

$$[E(K_2, M) || C(K_1, E(K_2, M))]$$

Receiver verifies MAC first, then decrypts.

Purpose

Confidentiality: via encryption (same as before).

Authentication: via MAC on ciphertext (any tampering with ciphertext is detected before decryption).

MAC is tied to ciphertext, giving extra protection against attacks that try to alter the ciphertext directly.

Case	Process	Keys Used	Confidentiality	Authentication	MAC Tied To
(a)	Plaintext + MAC	K	✗	✓	Plaintext
(b)	Encrypt $[M MAC]$	K_1, K_2	✓	✓	Plaintext
(c)	MAC over ciphertext	K_1, K_2	✓	✓	Ciphertext

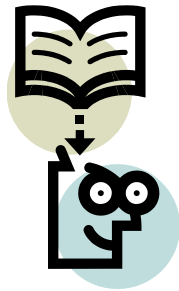
Message Authentication code - Cont...

- The receiver is assured that the message is from the alleged sender.
- Because no one else knows the secret key, no one else could prepare a message with a proper MAC.
- A MAC function is similar to encryption. One difference is that the MAC algorithm need not be reversible, as it must be for decryption.
- In general, the MAC function is a many-to-one function. The domain of the function consists of messages of some arbitrary length, whereas the range consists of all possible MACs and all possible keys.
- If an n -bit MAC is used, then there are 2^n possible MACs

UNIT-7

Hash Algorithms Digital Signature





Lecture 33

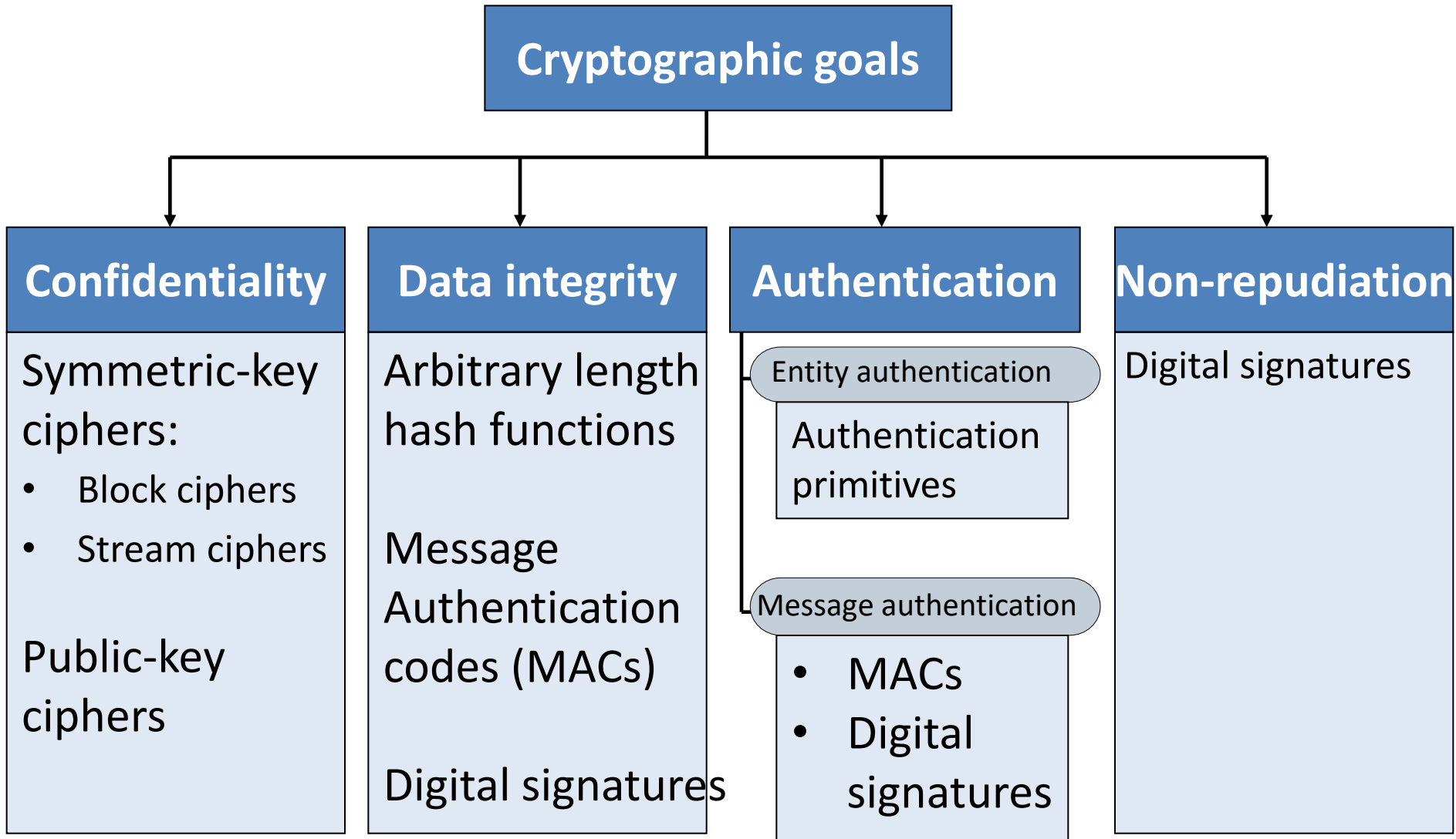


Digital Signature

Outline

- Digital Signatures
- Hash Algorithms

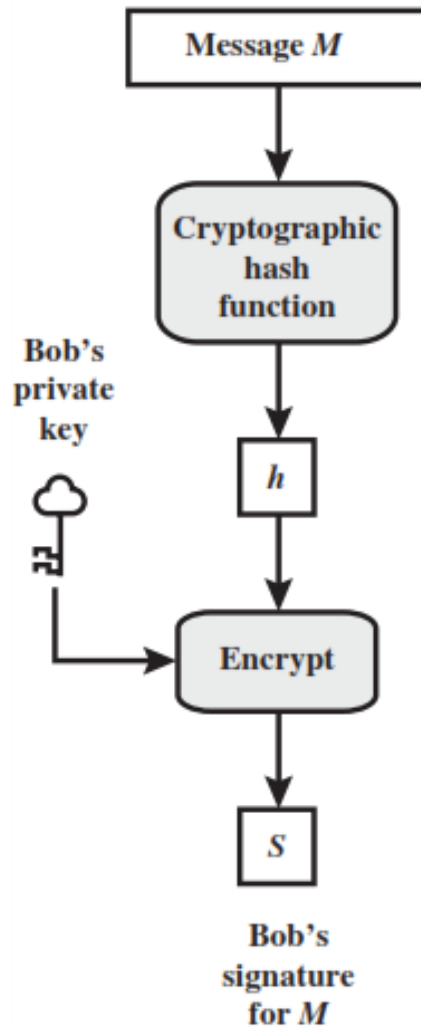
Cryptographic Goals



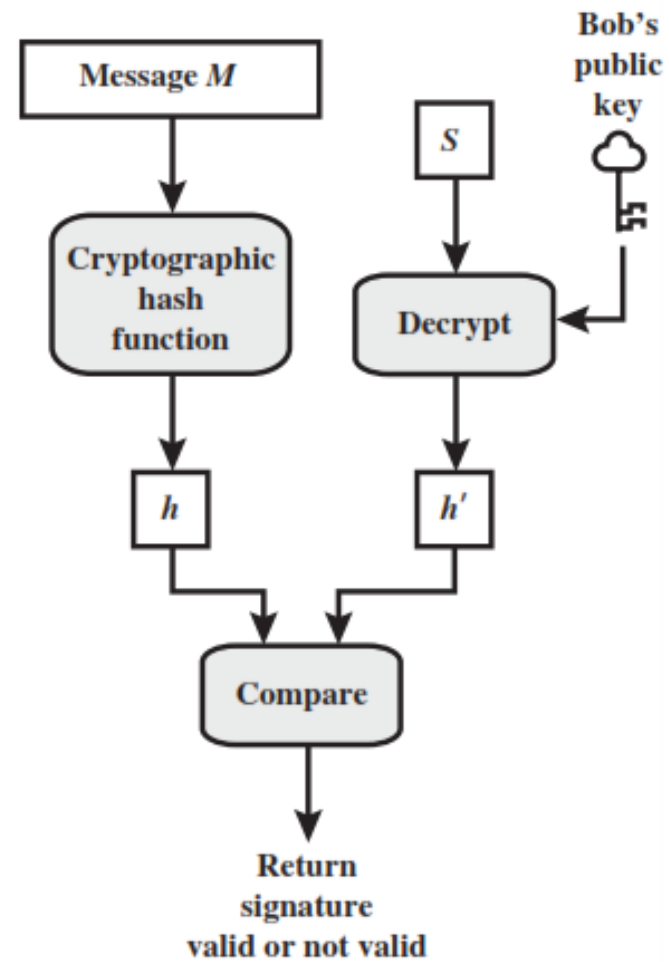
Digital Signature

- A **digital signature** is an authentication mechanism that enables the creator of a message to attach a code that acts as a **signature**.
- Typically the **signature** is formed by taking the hash of the message and encrypting the message with the creator's private key.
- The **signature** guarantees the source and integrity of the message.
- The **digital signature standard (DSS)** is an NIST standard that uses the secure hash algorithm (SHA).

Bob



Alice



Hash code, MAC and Digital Signature

Hash Code

- A **hash** of the message, if appended to the message itself, only protects against accidental changes to the message, as an attacker who modifies the message can simply calculate a new hash and use it instead of the original one. So this **only gives integrity**.

MAC

- A message authentication code (MAC) (sometimes also known as keyed hash) protects against message forgery by anyone who doesn't know the secret.
- This means that the receiver can forge any message – thus we have both **integrity** and **authentication** (as long as the receiver doesn't have a split personality), **but not non-repudiation**.

Hash code, MAC and Digital Signature

Digital Signature

- A **digital signature** is created with a private key, and verified with the corresponding public key of an asymmetric key-pair.
- Only the holder of the private key can create this signature, and normally anyone knowing the public key can verify it.

Attacks and Forgeries

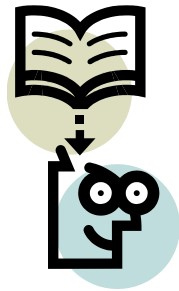
- **Key-only attack:** C only knows A's public key.
- **Known message attack:** C is given access to a set of messages and their signatures.
- **Generic chosen message attack:** C chooses a list of messages before attempting to break A's signature scheme, independent of A's public key. C then obtains from A valid signatures for the chosen messages. The attack is generic, because it does not depend on A's public key; the same attack is used against everyone.
- **Directed chosen message attack:** Similar to the generic attack, except that the list of messages to be signed is chosen after C knows A's public key but before any signatures are seen.
- **Adaptive chosen message attack:** C is allowed to use A as an "oracle." This means the A may request signatures of messages that depend on previously obtained message–signature pairs.

Attacks and Forgeries

- **Total break:** C determines A's private key.
- **Universal forgery:** C finds an efficient signing algorithm that provides an equivalent way of constructing signatures on arbitrary messages.
- **Selective forgery:** C forges a signature for a particular message chosen by C.
- **Existential forgery:** C forges a signature for at least one message. C has no control over the message. Consequently, this forgery may only be a minor nuisance to A.

Digital Signature Requirements

1. The signature must be a **bit pattern** that depends on the message being signed.
2. The signature must use some information **unique** to the sender to prevent both forgery and denial.
3. It must be relatively **easy to produce** the digital signature.
4. It must be relatively **easy to recognize** and **verify** the digital signature.
5. It must be computationally **infeasible to forge** a digital signature, either by constructing a new message for an existing digital signature or by constructing a fraudulent digital signature for a given message.
6. It must be **practical to retain a copy** of the digital signature in storage.



Lecture 35



Digital Signature Algorithm

Digital Signature Algorithm

Global Public-Key Components

- p prime number where $2^{L-1} < p < 2^L$
for $512 \leq L \leq 1024$ and L a multiple of 64;
i.e., bit length of between 512 and 1024 bits
in increments of 64 bits
- q prime divisor of $(p - 1)$, where $2^{N-1} < q < 2^N$
i.e., bit length of N bits
- $g = h(p - 1)/q \bmod p$,
where h is any integer with $1 < h < (p - 1)$
such that $h^{(p-1)/q} \bmod p > 1$

Key & Secret Number Generation

User's Private Key

x random or pseudorandom integer with $0 < x < q$

User's Public Key

$y = g^x \bmod p$

User's Per-Message Secret Number

k random or pseudorandom integer with $0 < k < q$

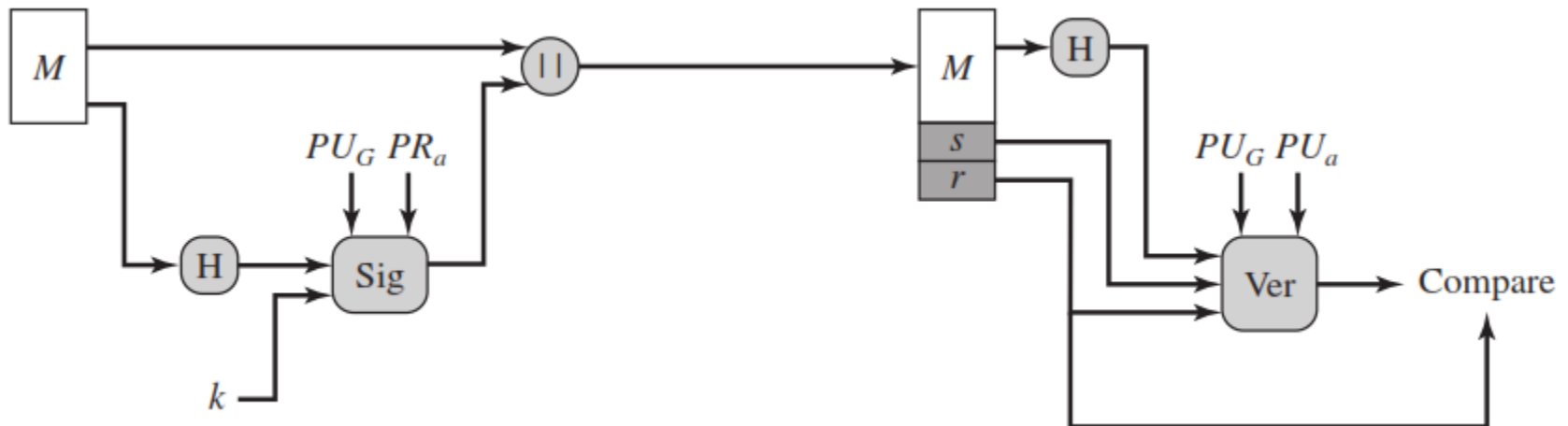
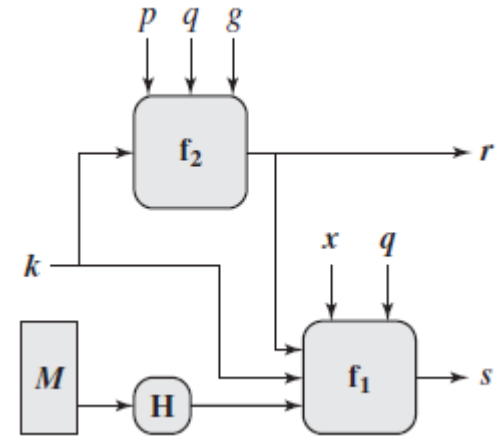
Signing Process

Signing

$$r = (g^k \bmod p) \bmod q$$

$$s = [k^{-1} (H(M) + xr)] \bmod q$$

$$\text{Signature} = (r, s)$$



Digital Signature Algorithm

Verifying

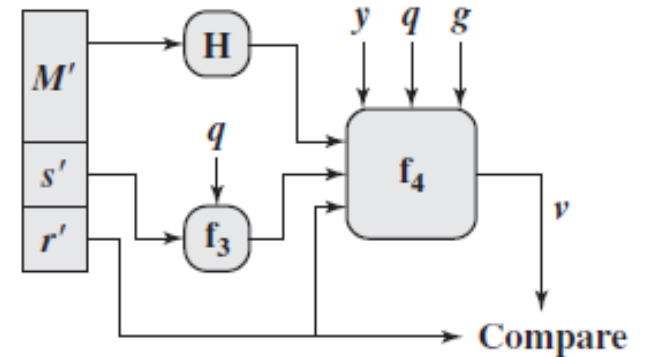
$$w = (s')^{-1} \bmod q$$

$$u_1 = [H(M')w] \bmod q$$

$$u_2 = (r')w \bmod q$$

$$v = [(g^{u_1} y^{u_2}) \bmod p] \bmod q$$

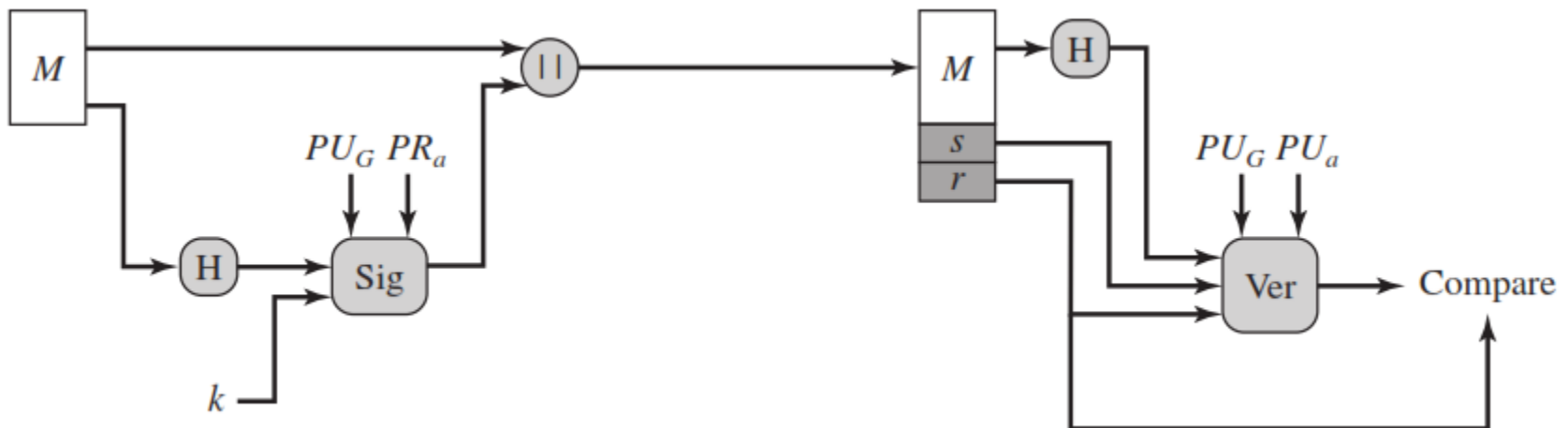
$$\text{TEST: } v = r'$$



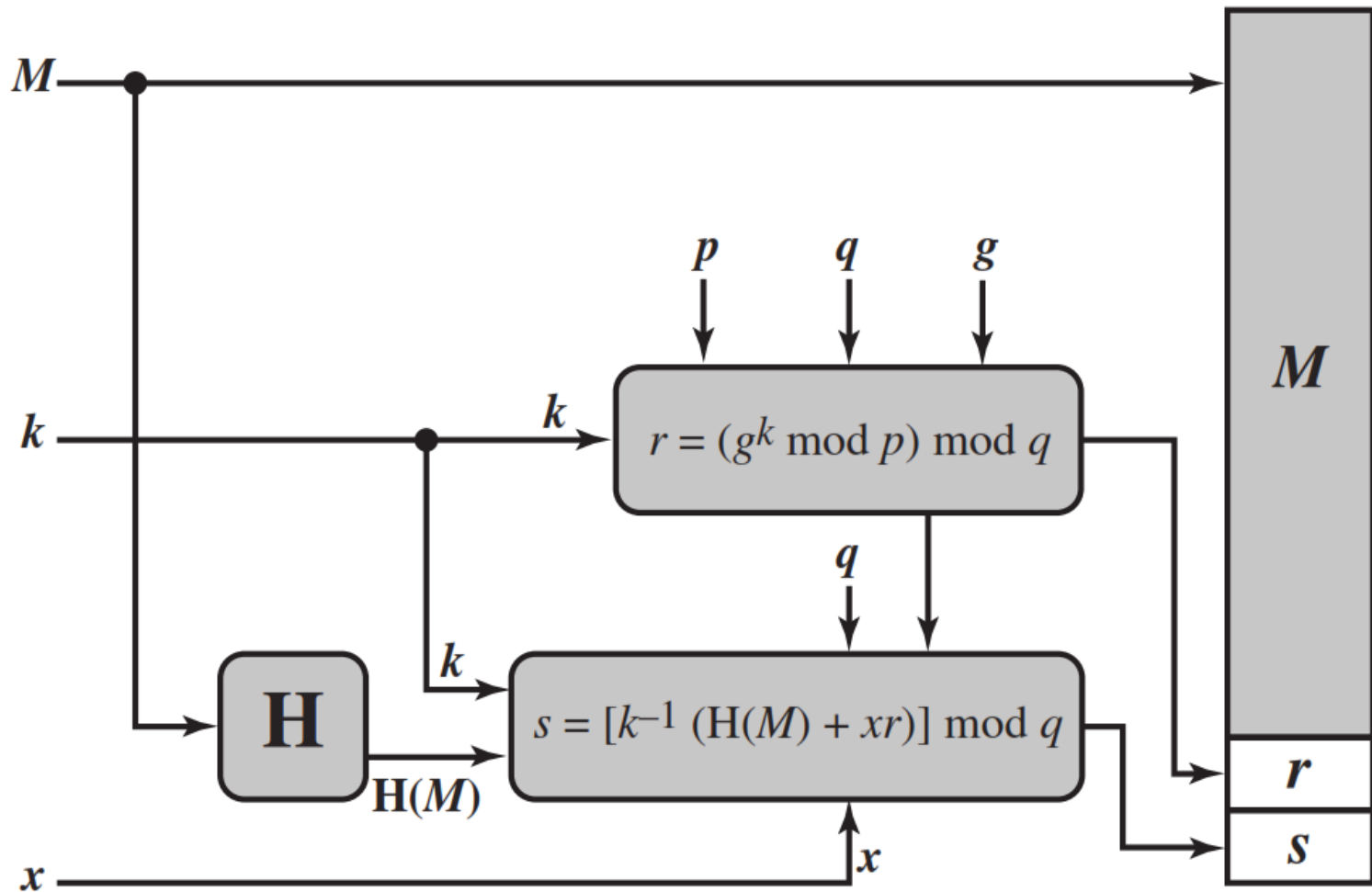
M = message to be signed

$H(M)$ = hash of M using SHA-1

M', r', s' = received versions of M, r, s



DSA Signing



(a) Signing

ElGamal Digital Signatures

Key Generation

Select $X_A : 1 < X_A < q-1$

Calculate $Y_A = \alpha^{X_A} \bmod q$

Private Key = X_A

Public Key = $\{q, \alpha, Y_A\}$

Signing a Message

Choose random integer K ,

$1 \leq K \leq q-1$ and $\text{GCD}(K, q-1) = 1$

Compute $S_1 = \alpha^K \bmod q$

Compute $K^{-1} \bmod (q-1)$

Compute $S_2 = K^{-1} (m - X_A * S_1) \bmod (q-1)$

Signature = (S_1, S_2)

Verifying a Signature

Compute $V_1 = \alpha^m \bmod q$

Compute $V_2 = (Y_A)^{S_1} * (S_1)^{S_2} \bmod q$

The signature is valid if $V_1 = V_2$

How $V_1 = V_2$?

$$\alpha^m \bmod q = (Y_A)^{S_1} * (S_1)^{S_2} \bmod q$$

$$\alpha^m \bmod q = (\alpha^{X_A})^{S_1} * (\alpha^K)^{S_2} \bmod q$$

$$\alpha^m \bmod q = \alpha^{X_A S_1} * \alpha^{K S_2} \bmod q$$

$$\alpha^{m - X_A S_1} \bmod q = \alpha^{K S_2} \bmod q$$

$$m - X_A S_1 \equiv K S_2 \bmod (q-1)$$

$$m - X_A S_1 \equiv K K^{-1} (m - X_A * S_1) \bmod (q-1)$$

ElGamal Signature Example

- Use field $GF(19)$ $q=19$ and $a=10$
- Alice computes her key:
 - A chooses $x_A=16$ & computes $y_A=10^{16} \bmod 19 = 4$
- Alice signs message with hash $m=14$ as $(3, 4)$:
 - choosing random $K=5$ which has $\gcd(18, 5)=1$
 - computing $S_1 = 10^5 \bmod 19 = 3$
 - finding $K^{-1} \bmod (q-1) = 5^{-1} \bmod 18 = 11$
 - computing $S_2 = 11(14 - 16 \cdot 3) \bmod 18 = 4$
- Any user B can verify the signature by computing
 - $V_1 = 10^{14} \bmod 19 = 16$
 - $V_2 = 4^3 \cdot 3^4 = 5184 = 16 \bmod 19$
 - since $16 = 16$ signature is valid

Schnorr Digital Signatures

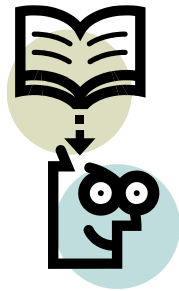
- Also uses exponentiation in a finite (Galois)
 - security based on discrete logarithms
- Minimizes message dependent computation
 - multiplying a $2n$ -bit integer with an n -bit integer
- Main work can be done in idle time
- Have using a prime modulus p
 - $p-1$ has a prime factor q of appropriate size
 - typically p 1024-bit and q 160-bit numbers

Schnorr Key Setup

- choose suitable primes p, q
- choose α such that $\alpha^q = 1 \pmod q$
- (α, p, q) are global parameters for all
- each user (eg. A) generates a key
 - chooses a secret key (number): $0 < s_A < q$
 - compute their **public key**: $v_A = \alpha^{-s_A} \pmod q$

Schnorr Signature

- User signs message by
 - choosing random r with $0 < r < q$ and computing $x = \alpha^r \bmod p$
 - concatenate message with x and hash result to computing: $e = H(M \parallel x)$
 - computing: $y = (r + se) \bmod q$
 - signature is pair (e, y)
- Any other user can verify the signature as follows:
 - computing: $x' = \alpha^y v^e \bmod p$
 - verifying that: $e = H(M \parallel x')$



Lecture 37



Network & System Security

Network & System Security

Security Basics

CIA Triad Diagram

Malicious Software Overview

Security Basics

- Network Security vs System Security
 - Security Goals: Confidentiality, Integrity, Availability
 - Threats and Vulnerabilities
 - Attacker Types

CIA Triad Diagram



The diagram consists of three light blue rounded rectangular boxes arranged vertically. Each box has a thin dark blue border and a subtle drop shadow. The top box contains the word 'Confidentiality', the middle box contains 'Integrity', and the bottom box contains 'Availability'.

Confidentiality

Integrity

Availability

Malicious Software Overview

- Definition of Malware
 - Malware Behavior
 - Propagation Techniques

Virus, Worms & Trojans

Computer Viruses

Worms

Virus vs Worm vs Trojan

Trojans

Computer Viruses

- Definition & Behavior

- Components: Infection Mechanism, Trigger, Payload
- Types: Boot, File, Macro Virus
- Detection Techniques

Worms

- Definition & Propagation
 - Self-replicating nature
 - Network-level infection
 - Examples

Virus vs Worm vs Trojan

Virus

Worm

Trojan

Trojans

- Masquerading Behavior
 - Backdoors & Keyloggers
 - Prevention & Detection

Advanced Malware Types & Defense

Malware Classification

Viruses

Worms

Trojans

Spyware

Ransomware

Other Malware

- Spyware
 - Adware
 - Ransomware
 - Rootkits

Defense Techniques

- Antivirus & Anti-malware Tools
 - Sandboxing
 - Patch Management
 - User Awareness

IPSec (IP & Network Layer Security)

Overview

- Need for IP Layer Security
 - Security Associations (SA)
 - Key Management

IPSec Protocols

- Authentication Header (AH)
 - Encapsulating Security Payload (ESP)
 - Transport Mode vs Tunnel Mode

IPSec Building Blocks

AH

ESP

SA

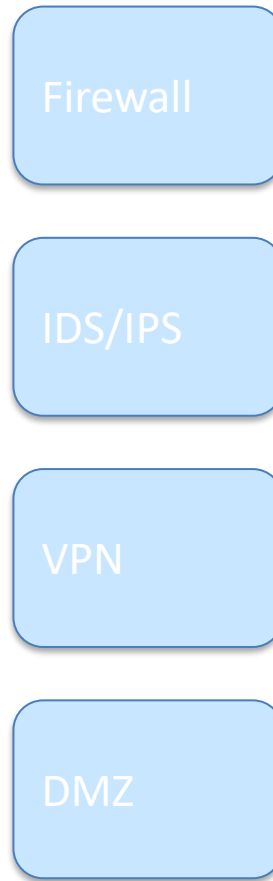
Firewall Concepts

- Packet Filtering
 - Stateful Firewalls
 - Next-Generation Firewalls

IDS / IPS

- Intrusion Detection System
 - Intrusion Prevention System
 - Signature-based vs Anomaly-based Detection

Network Security Architecture



VPN Technologies

- Tunneling Concepts
 - Site-to-Site VPN
 - Remote Access VPN

Web Threats

- SQL Injection
 - XSS
 - Session Hijacking
 - Cookie Theft

Web Security Layers

Input Validation

Authentication

Session Security

Output Encoding

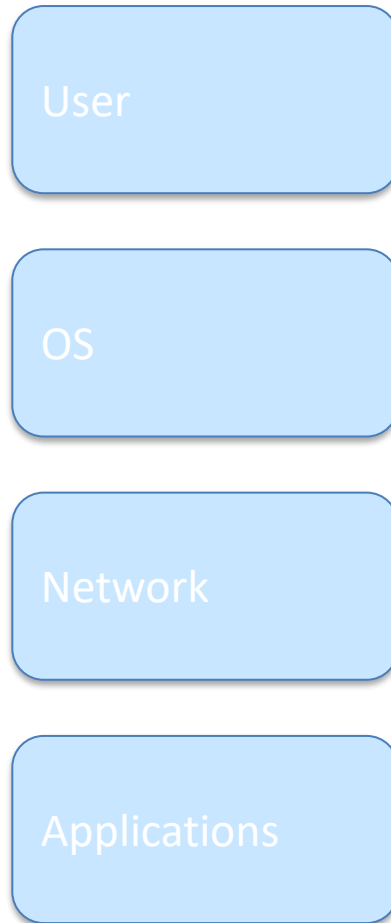
Email Security

- Phishing & Spoofing
 - PGP
 - S/MIME
 - SPF, DKIM, DMARC

System Security Basics

- Authentication Mechanisms
 - Access Control Models
 - Least Privilege Principle
 - Operating System Hardening

System Security Layers



Security Tools

- Packet Analyzers (Wireshark)
 - Vulnerability Scanners (Nmap, Nessus)
 - Antivirus / EDR Tools